

Rapport de Projet de Fin d'Études

Présenté à :

**L'Institut Supérieur d'Informatique
et de Multimédia de Sfax**

Élaboré par :

**Nouha BOUGHNIM
Wiem BZIOUICH**

**InvisiThreat : Plateforme DevSecOps de détection
automatisée de vulnérabilités par analyses SAST et DAST**

Présenté devant le Jury :

Président : M./Mme *Prénom NOM*

Rapporteur : M./Mme *Prénom NOM*

Examineur : M./Mme *Prénom
NOM*

Encadrant académique :

Mme *Najiba TAGOUGUI*

Encadrant professionnel :

Mr *Houssemeddine HASSAYOUNE*

Organisme d'accueil : **Mobelite**



Année Universitaire : **2025 - 2026**

Dédicace

*À nos parents,
dont le soutien indéfectible, les sacrifices silencieux
et la confiance sans faille ont rendu possible
chaque étape de ce parcours.*

*À nos enseignants de l'ISIMS,
qui nous ont transmis bien plus que des connaissances :
une rigueur, une méthode et le goût de l'excellence.*

*À nos encadrantes et encadrants,
Mme Najiba TAGOUGUI et M. Houssemeddine HASSAYOUNE,
pour leur disponibilité, leurs orientations éclairées
et la confiance accordée tout au long de ce projet.*

*À nos collègues et amis,
compagnons de longues nuits de travail,
dont l'encouragement mutuel a transformé
chaque difficulté en opportunité d'apprentissage.*

*À toute personne qui croit que la sécurité informatique
n'est pas une contrainte, mais une responsabilité.*

Nouha & Wiem

Remerciements

La réalisation de ce projet de fin d'études n'aurait pas été possible sans le concours bienveillant de nombreuses personnes à qui nous tenons à exprimer notre sincère gratitude.

Nous adressons en premier lieu nos remerciements les plus chaleureux à notre encadrante académique, **Mme Najiba TAGOUGUI**, pour la qualité de son encadrement, la pertinence de ses conseils scientifiques et sa disponibilité constante tout au long de ce travail. Sa rigueur intellectuelle et son soutien méthodologique ont été déterminants dans la conduite et la réussite de ce projet.

Nous remercions également notre encadrant professionnel, **M. Houssemeddine HASSAYOUNE**, ingénieur chez Mobelite, pour nous avoir accueillis au sein de son équipe, partagé son expertise terrain en matière de sécurité applicative et orienté nos choix techniques avec une vision résolument pragmatique. Sa connaissance des enjeux DevSecOps a été une source d'inspiration continue.

Nos remerciements vont aussi à la direction et aux équipes de **Mobelite** pour avoir mis à notre disposition un environnement de travail stimulant, un accès aux ressources nécessaires et un cadre professionnel propice à l'innovation.

Nous tenons à remercier les membres du jury, qui ont accepté d'évaluer ce travail et dont les remarques contribueront à son enrichissement. Leur expertise représente pour nous une reconnaissance précieuse de l'effort fourni.

Nous exprimons notre profonde gratitude à l'ensemble du corps enseignant de l'**Institut Supérieur d'Informatique et de Multimédia de Sfax**, dont la formation solide nous a dotés des compétences techniques et analytiques nécessaires pour mener à bien un projet de cette envergure.

Enfin, nous remercions nos familles respectives pour leur soutien moral indéfectible, leur patience et leur encouragement tout au long de ces années d'études. Ce travail leur appartient autant qu'à nous.

Liste des Abréviations

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DAST	Dynamic Application Security Testing
JWT	JSON Web Token
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
SAST	Static Application Security Testing
2FA	Two-Factor Authentication
TOTP	Time-Based One-Time Password
LLM	Large Language Model
CLI	Command-Line Interface
OAuth	Open Authorization
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
SQL	Structured Query Language
UI	User Interface
UX	User Experience
ZAP	Zed Attack Proxy

Table des matières

Dédicace	i
Remerciements	ii
Liste des Abréviations	iii
Introduction Générale	1
1 Cadre général du projet	2
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.3 Présentation du projet InvisiThreat	3
1.3.1 Contexte général	3
1.3.2 Définition de la mission	3
1.3.3 Problématique	4
1.3.4 Objectifs du projet	4
1.3.5 Périmètre du projet	4
1.4 Étude et critique de l'existant	5
1.4.1 Panorama des solutions existantes	5
1.4.1.1 SonarQube	5
1.4.1.2 Burp Suite Professional	5
1.4.1.3 Checkmarx	5
1.4.1.4 Snyk	6
1.4.1.5 OWASP ZAP	6
1.4.2 Critiques et recommandations	6
1.4.2.1 Synthèse comparative	6
1.4.2.2 Critique de l'existant	7
1.4.3 Solution proposée	7
1.5 Méthodologie de développement	8
1.5.1 Choix méthodologique : Scrum	8

1.5.1.1	Étude comparative des méthodologies	8
1.5.1.2	Valeurs fondamentales de l'Agile	8
1.5.1.3	Présentation de Scrum	8
1.5.1.4	Les trois piliers de Scrum	9
1.5.2	Rôles et cérémonies	9
1.5.2.1	Artefacts de Scrum	10
1.6	Conclusion	10
2	Sprint 0 – Initialisation et étude technique	11
2.1	Introduction	11
2.2	Identification des acteurs	11
2.3	Analyse des besoins	12
2.3.1	Besoins fonctionnels	12
2.3.2	Besoins non fonctionnels	14
2.3.3	Diagramme global des cas d'utilisation	14
2.4	Gestion du projet avec Scrum	15
2.4.1	Product Backlog	15
2.4.1.1	Structuration en epics	15
2.4.1.2	Technique de priorisation du Product Backlog	15
2.4.1.3	Synthèse du Product Backlog	16
2.4.2	Planification des sprints	20
2.5	Étude technique	21
2.5.1	Environnement matériel	21
2.5.2	Environnement logiciel	21
2.5.3	Technologies et outils utilisés	22
2.5.4	Architecture du système	24
2.5.4.1	Architecture logicielle	24
2.5.4.2	Architecture physique	24
2.6	Diagramme de déploiement	25
2.7	Conclusion	25
3	Sprint 1 – Sécurité et gouvernance	27
3.1	Introduction	27
3.2	Objectifs du sprint et planification des tâches	27
3.2.1	Objectifs attendus	27
3.2.2	Sprint Backlog	28

3.3	Technologies et concepts utilisés	28
3.3.1	JWT et FastAPI Security (python-jose, OAuth2PasswordBearer) . .	28
3.3.2	Authentification forte TOTP (pyotp)	29
3.3.3	Contrôle d'accès RBAC (bcrypt)	29
3.4	Analyse et spécification des besoins	29
3.4.1	Diagramme de cas d'utilisation du sprint	29
3.4.2	Description détaillée des cas d'utilisation	31
3.5	Modélisation conceptuelle	32
3.5.1	Diagramme de classes	32
3.5.2	Diagrammes de séquence	35
3.6	Réalisation	36
3.6.1	Authentification JWT	36
3.6.2	RBAC et gestion des rôles	36
3.6.3	2FA TOTP et audit logs	36
3.7	Test et validation	37
3.7.1	Tests réalisés	37
3.7.2	Validation des fonctionnalités	37
3.7.3	Gestion des versions et commits	38
3.8	Conclusion	38
4	Sprint 2 – Projets et intégrations	39
4.1	Introduction	39
4.2	Objectifs du sprint et planification des tâches	39
4.2.1	Objectifs attendus	39
4.2.2	Sprint Backlog	40
4.3	Technologies et concepts utilisés	40
4.3.1	GitHub REST API v3 et OAuth2	40
4.3.2	Chiffrement Fernet (cryptography)	41
4.3.3	SQLAlchemy Alembic (migrations)	41
4.4	Analyse et spécification des besoins	41
4.4.1	Diagramme de cas d'utilisation du sprint	41
4.4.2	Description détaillée des cas d'utilisation	42
4.5	Modélisation conceptuelle	43
4.5.1	Diagramme de classes	43
4.5.2	Diagrammes de séquence	46
4.6	Réalisation	47

4.6.1	Gestion des projets et des dépôts	47
4.6.2	Intégrations OAuth et clés API	47
4.7	Test et validation	47
4.7.1	Tests réalisés	47
4.7.2	Validation	47
4.7.3	Gestion des versions et commits	48
4.8	Conclusion	48
5	Sprint 3 – SAST et DAST	49
5.1	Introduction	49
5.2	Objectifs du sprint et planification des tâches	49
5.2.1	Objectifs attendus	49
5.2.2	Sprint Backlog	50
5.3	Technologies et concepts utilisés	50
5.3.1	Semgrep (analyse statique polyglotte)	50
5.3.2	Bandit (inspection de sécurité Python)	51
5.3.3	Celery et Redis (orchestration asynchrone)	51
5.3.4	Socket.IO — python-socketio (temps réel)	51
5.3.5	OWASP ZAP (analyse dynamique, mode daemon)	51
5.4	Analyse et spécification des besoins	52
5.4.1	Diagramme de cas d'utilisation du sprint	52
5.4.2	Description détaillée des cas d'utilisation	53
5.5	Modélisation conceptuelle	54
5.5.1	Diagramme de classes	54
5.5.2	Diagrammes de séquence	56
5.6	Réalisation	57
5.6.1	Pipeline SAST (Semgrep + Bandit)	57
5.6.2	Pipeline DAST (OWASP ZAP)	58
5.6.3	Suivi temps réel (Socket.IO)	58
5.7	Test et validation	58
5.7.1	Tests réalisés	58
5.7.2	Validation	59
5.7.3	Gestion des versions et commits	59
5.8	Conclusion	59
5.9	Introduction	59
5.10	Objectifs du sprint et planification des tâches	60

5.10.1	Objectifs attendus	60
5.10.2	Sprint Backlog	61
5.11	Technologies et concepts utilisés	61
5.11.1	Semgrep (analyse statique polyglotte)	61
5.11.2	Bandit (inspection de sécurité Python)	62
5.11.3	Celery et Redis (orchestration asynchrone)	62
5.12	Analyse et spécification des besoins	62
5.12.1	Diagramme de cas d'utilisation du sprint	62
5.12.2	Description détaillée des cas d'utilisation	63
5.13	Modélisation conceptuelle	64
5.13.1	Diagramme de classes	64
5.13.2	Diagrammes de séquence	65
5.14	Réalisation	66
5.14.1	Intégration SAST (Semgrep, Bandit)	66
5.15	Test et validation	67
5.15.1	Tests réalisés	67
5.15.2	Validation	67
5.15.3	Gestion des versions et commits	67
5.16	Conclusion	67
6	Sprint 4 – Workflow de vulnérabilités	69
6.1	Introduction	69
6.2	Objectifs du sprint et planification des tâches	69
6.2.1	Objectifs attendus	69
6.2.2	Sprint Backlog	70
6.3	Technologies et concepts utilisés	71
6.3.1	Workflow de gestion d'états (State Machine)	71
6.3.2	Système de notifications (Notification)	71
6.3.3	SQLAlchemy — Alembic (migrations Sprint 4)	71
6.4	Analyse et spécification des besoins	71
6.4.1	Diagramme de cas d'utilisation du sprint	71
6.4.2	Description détaillée des cas d'utilisation	72
6.5	Modélisation conceptuelle	73
6.5.1	Diagramme de classes	73
6.5.2	Diagramme de séquence	75
6.6	Réalisation	76

6.6.1	Workflow d'états et assignation	76
6.6.2	Commentaires et notifications	76
6.7	Test et validation	77
6.7.1	Tests réalisés	77
6.7.2	Validation	77
6.7.3	Gestion des versions et commits	77
6.8	Conclusion	77
6.9	Introduction	78
6.10	Objectifs du sprint et planification des tâches	78
6.10.1	Objectifs attendus	78
6.10.2	Sprint Backlog	79
6.11	Technologies et concepts utilisés	80
6.11.1	OWASP ZAP (API REST daemon)	80
6.11.2	python-socketio (Socket.IO temps réel)	80
6.11.3	Ollama (LLM local — Mistral/Qwen)	80
6.11.4	ReportLab (génération PDF)	81
6.12	Analyse et spécification des besoins	81
6.12.1	Diagramme de cas d'utilisation du sprint	81
6.12.2	Description détaillée des cas d'utilisation	82
6.13	Modélisation conceptuelle	84
6.13.1	Diagramme de classes	84
6.13.2	Diagrammes de séquence	88
6.14	Réalisation	89
6.14.1	Intégration DAST (OWASP ZAP)	89
6.14.2	Orchestration et workflow de vulnérabilités	89
6.14.3	Assistant IA (Ollama)	89
6.15	Test et validation	90
6.15.1	Tests réalisés	90
6.15.2	Validation	91
6.15.3	Gestion des versions et commits	91
6.16	Conclusion	91
7	Sprint 5 – Dashboard, Recommandation LLM et Rapport	92
7.1	Introduction	92
7.2	Objectifs du sprint et planification des tâches	92
7.2.1	Objectifs attendus	92

7.2.2	Sprint Backlog	93
7.3	Technologies et concepts utilisés	93
7.3.1	Ollama — LLM local (Mistral, Qwen)	93
7.3.2	ReportLab (génération PDF)	94
7.3.3	Agrégations PostgreSQL (tableau de bord)	94
7.4	Analyse et spécification des besoins	94
7.4.1	Diagramme de cas d'utilisation du sprint	94
7.4.2	Description détaillée des cas d'utilisation	95
7.5	Modélisation conceptuelle	97
7.5.1	Diagramme de classes	97
7.5.2	Diagramme de séquence	100
7.6	Réalisation	100
7.6.1	Tableau de bord analytique	100
7.6.2	Assistance IA locale (Ollama)	100
7.6.3	Génération de rapports (ReportLab + JSON)	101
7.7	Test et validation	101
7.7.1	Tests réalisés	101
7.7.2	Validation	102
7.7.3	Gestion des versions et commits	102
7.8	Conclusion	102
CONCLUSION GÉNÉRALE		103
BIBLIOGRAPHIE		105

0

Table des figures

1.1	Logo de l'entreprise Mobelite.	2
1.2	Logo SonarQube	5
1.3	Logo Burp Suite	5
1.4	Logo Checkmarx	5
1.5	Logo Snyk	6
1.6	Logo OWASP ZAP	6
1.7	Cycle Scrum appliqué au projet InvisiThreat.	9
2.1	Diagramme de cas d'utilisation global d'InvisiThreat.	15
2.2	Architecture logicielle en couches d'InvisiThreat.	24
2.3	Architecture physique – déploiement Docker Compose d'InvisiThreat.	25
2.4	Diagramme de déploiement Docker Compose d'InvisiThreat.	25
3.1	Diagramme de cas d'utilisation – Sprint 1.	30
3.2	Diagramme de classes – Sprint 1 (entités User, Role, AuthToken, AuditLog).	33
3.3	Diagramme de séquence DS-1a – S'authentifier (JWT + TOTP optionnel).	35
3.4	Diagramme de séquence DS-1b – Attribution de rôle (RBAC).	35
4.1	Diagramme de cas d'utilisation – Sprint 2.	42
4.2	Diagramme de classes – Sprint 2 (entités Project, GitHubRepository, ProjectMember, UserAPIKey).	44
4.3	Diagramme de séquence DS-2a – Créer un projet.	46
4.4	Diagramme de séquence DS-2b – Connecter un dépôt GitHub (chiffrement Fernet du token).	46
5.1	Diagramme de cas d'utilisation – Sprint 3 (SAST et DAST).	52
5.2	Diagramme de classes – Sprint 3 (Scan, Vulnerability, RiskScore avec enums).	55
5.3	Diagramme de séquence DS-3a – Lancement SAST (Celery worker · Semgrep · Bandit · Socket.IO).	57
5.4	Diagramme de séquence DS-3b – Lancement DAST (OWASP ZAP · normalisation · score de risque).	57

5.5	Diagramme de cas d'utilisation – Sprint 3.	63
5.6	Diagramme de classes – Sprint 3 (entités Scan, Vulnerability avec enums). . . .	64
5.7	Diagramme de séquence DS-3a – Lancer un scan SAST (orchestration Celery/Redis).	66
5.8	Diagramme de séquence DS-3b – Suivi temps réel Socket.IO (scopé par scan_id). .	66
6.1	Diagramme de cas d'utilisation – Sprint 4 (Workflow de vulnérabilités).	72
6.2	Diagramme de classes – Sprint 4 (VulnerabilityTask, VulnerabilityTaskComment, Notification).	74
6.3	Diagramme de séquence DS-4 – Sync tâches · Assignment · Statut · Notification. .	76
6.4	Diagramme de cas d'utilisation – Sprint 4.	81
6.5	Diagramme de classes – Sprint 4 (entités RiskScore, VulnerabilityTask, VulnerabilityTaskComment, Recommendation, Notification, SecurityReport). .	85
6.6	Diagramme de séquence DS-4a – Scan DAST via OWASP ZAP.	88
6.7	Diagramme de séquence DS-4b – Workflow de vulnérabilité (assignment, transition d'état).	88
6.8	Diagramme de séquence DS-4c – Recommandation IA via Ollama (action Assist with AI).	89
7.1	Diagramme de cas d'utilisation – Sprint 5 (Dashboard · Recommandation LLM · Rapport).	95
7.2	Diagramme de classes – Sprint 5 (Recommendation, SecurityReport, DashboardStats).	98
7.3	Diagramme de séquence DS-5 – Dashboard · Recommandation LLM (Ollama) · Génération de rapport (ReportLab).	100

0

Liste des tableaux

1.1	Fiche d'identité de Mobelite	3
1.2	Comparatif des solutions de sécurité applicative	6
1.3	Comparaison des méthodologies de développement	8
2.1	Acteurs du système InvisiThreat et leurs responsabilités	12
2.2	Product Backlog – Gouvernance et gestion des accès	17
2.3	Product Backlog – Scans et orchestration	18
2.4	Product Backlog – Workflow, IA, reporting et CI/CD	19
2.5	Planification et livrables des sprints InvisiThreat	20
2.6	Configuration matérielle des postes de travail	21
2.7	Environnement logiciel utilisé pour le développement	22
2.8	Stack technique d'InvisiThreat – technologies et justifications	23
3.1	Sprint Backlog – Sprint 1	28
3.2	Description du cas d'utilisation – S'authentifier	31
3.3	Description du cas d'utilisation – Attribuer un rôle	32
3.4	Dictionnaire de données – Sprint 1	34
3.5	Tableau de test – S'authentifier (Sprint 1)	37
3.6	Tableau de test – Attribution de rôle (Sprint 1)	37
4.1	Sprint Backlog – Sprint 2	40
4.2	Description du cas d'utilisation – Créer un projet	42
4.3	Description du cas d'utilisation – Connecter un dépôt GitHub	43
4.4	Dictionnaire de données – Sprint 2	45
4.5	Tableau de test – Créer un projet (Sprint 2)	47
5.1	Sprint Backlog – Sprint 3	50
5.2	Description du cas d'utilisation – Lancer un scan SAST	53
5.3	Description du cas d'utilisation – Lancer un scan DAST	54
5.4	Dictionnaire de données – Sprint 3	56

5.5	Tableau de test – Scan SAST complet (Sprint 3)	58
5.6	Tableau de test – Scan DAST et score de risque (Sprint 3)	59
5.7	Sprint Backlog – Sprint 3	61
5.8	Description du cas d'utilisation – Lancer un scan SAST	63
5.9	Description du cas d'utilisation – Consulter le rapport SAST normalisé	64
5.10	Dictionnaire de données – Sprint 3	65
5.11	Tableau de test – Lancer un scan SAST (Sprint 3)	67
6.1	Sprint Backlog – Sprint 4	70
6.2	Description du cas d'utilisation – Créer et assigner une tâche	72
6.3	Description du cas d'utilisation – Transition d'état	73
6.4	Dictionnaire de données – Sprint 4	75
6.5	Tableau de test – Cycle complet de remédiation (Sprint 4)	77
6.6	Sprint Backlog – Sprint 4	79
6.7	Description du cas d'utilisation – Lancer un scan DAST	82
6.8	Description du cas d'utilisation – Workflow de vulnérabilité	83
6.9	Description du cas d'utilisation – Recommandation IA (Ollama)	84
6.10	Dictionnaire de données – Sprint 4	87
6.11	Tableau de test – Scan DAST et recommandation IA (Sprint 4)	90
6.12	Tableau de test – Workflow de vulnérabilité (Sprint 4)	90
7.1	Sprint Backlog – Sprint 5	93
7.2	Description du cas d'utilisation – Consulter le tableau de bord	95
7.3	Description du cas d'utilisation – Recommandation IA (Ollama)	96
7.4	Description du cas d'utilisation – Générer un rapport	97
7.5	Dictionnaire de données – Sprint 5	99
7.6	Tableau de test – Recommandation LLM (Sprint 5)	101
7.7	Tableau de test – Génération de rapport PDF (Sprint 5)	102

0

Introduction Générale

À l'ère de la transformation numérique, la sécurité des systèmes d'information est devenue un enjeu stratégique majeur pour les organisations de toute taille. L'explosion du nombre d'applications web, conjuguée à la sophistication croissante des cyberattaques, expose quotidiennement les entreprises à des risques considérables : vol de données sensibles, interruption de services, atteinte à la réputation et pertes financières. Selon les rapports récents des organismes spécialisés, le nombre de vulnérabilités découvertes chaque année ne cesse de croître, tandis que les délais d'exploitation par des acteurs malveillants se réduisent drastiquement.

Face à cette réalité, les approches traditionnelles de sécurité — reposant sur des audits ponctuels et des interventions manuelles — montrent leurs limites. Elles peinent à s'adapter au rythme imposé par les cycles de développement modernes, où le code évolue en continu et les mises en production se succèdent à cadence soutenue. Il en résulte une fenêtre de vulnérabilité persistante, durant laquelle les failles applicatives demeurent exposées sans être détectées ni corrigées.

C'est dans ce contexte qu'émerge la nécessité d'intégrer la sécurité au cœur même du cycle de développement logiciel, selon le paradigme **DevSecOps**. Cette approche vise à automatiser la détection des vulnérabilités dès les premières étapes du développement, rendant ainsi la sécurité continue, proactive et mesurable. Cependant, malgré l'existence d'outils d'analyse isolés, les équipes techniques se heurtent souvent à l'absence d'une plateforme unifiée capable d'orchestrer ces analyses, d'en consolider les résultats et d'en restituer une vision claire et priorisée.

C'est précisément pour répondre à ce besoin, identifié au sein de l'entreprise **Mobelite**, qu'a été initié le projet **InvisiThreat**. La solution est une plateforme DevSecOps intégrée combinant **SAST** et **DAST** dans un pipeline unifié. Elle orchestre les scans de manière asynchrone (*Celery/Redis*), normalise les résultats, applique un **scoring de risque**, gère un **workflow de vulnérabilités** et diffuse des **notifications temps réel**. L'intégration **GitHub** (OAuth) et l'usage de **clés API** rendent possible le déclenchement des analyses depuis un CLI ou un pipeline CI/CD.

L'assistance intelligente constitue un élément central : un module IA local (*Ollama*, modèles *Mistral/Qwen*) est exposé dans l'interface via l'action *Assist with AI*. Il explique les vulnérabilités, simplifie les messages techniques, propose des recommandations et guide le développeur vers la correction.

Le développement de ce projet a été conduit selon une méthodologie **Scrum**, favorisant une progression itérative, une collaboration continue avec les parties prenantes et une capacité d'adaptation aux besoins évolutifs du terrain.

Chapitre

1

Cadre général du projet

1.1 Introduction

La sécurité applicative s'impose aujourd'hui comme une préoccupation centrale pour toute organisation reposant sur des systèmes informatiques. L'essor des architectures orientées services et l'adoption des pratiques DevOps ont profondément reconfiguré le paysage des menaces numériques : la sécurité doit désormais être intégrée de manière continue et automatisée à chaque stade du pipeline de livraison.

Ce chapitre pose le cadre général du projet **InvisiThreat** : présentation de l'organisme d'accueil, problématique de sécurité, étude critique de l'existant, solution proposée et méthodologie de développement retenue.

1.2 Présentation de l'organisme d'accueil

Mobelite est une entreprise spécialisée dans la conception et le développement d'applications web et mobiles à forte valeur ajoutée. Elle accompagne ses clients dans la transformation numérique en proposant des solutions sur mesure fondées sur des architectures orientées services.



FIGURE 1.1 – Logo de l'entreprise Mobelite.

Son environnement technique repose sur Git, Docker, des pipelines CI/CD et des API REST consommées par des frontends React ou des clients mobiles — une stack efficace qui génère néanmoins une surface d'attaque étendue. Avant InvisiThreat, les audits de sécurité étaient conduits de manière ponctuelle et hétérogène, sans outillage unifié ni traçabilité structurée, ce qui rendait difficile la mesure de l'évolution du risque. Ce constat a motivé la création d'un outil interne capable de **standardiser, automatiser et centraliser** la détection des failles à l'échelle du portefeuille applicatif.

TABLE 1.1 – Fiche d’identité de Mobelite

Nom de l’entreprise	Mobelite
Secteur d’activité	Conseil en stratégie mobile, développement web et mobile, développement hybride, développement iOS et Android, développement Kotlin, méthodes agiles (SCRUM), test et assurance qualité (QA), SEO
Date de création	2012
Siège social	Monastir, Tunisie
Taille de l’entreprise	51 à 200 employés
Téléphone	+216 31 313 737
Site web	https://www.mobelite.fr
Adresse e-mail	Non communiquée publiquement

1.3 Présentation du projet InvisiThreat

1.3.1 Contexte général

L’accélération des cycles de livraison logicielle impose une approche radicalement différente des audits périodiques traditionnels. C’est dans ce contexte qu’a émergé le paradigme **DevSecOps**, qui intègre la sécurité dès la conception et à chaque étape du pipeline. Deux familles d’analyses en constituent le socle :

- **SAST** (*Static Application Security Testing*) : inspection du code source pour identifier des patterns dangereux, sans exécuter l’application.
- **DAST** (*Dynamic Application Security Testing*) : test de l’application en cours d’exécution pour détecter les failles d’exposition réelles.

À ces deux familles d’analyses s’ajoute un **assistant IA** basé sur des modèles LLM locaux (Ollama, Mistral/Qwen) chargé de contextualiser les alertes, de formuler des recommandations exploitables et d’accompagner la remédiation via l’action *Assist with AI*. Ce module fonctionne via Ollama, un serveur d’inférence local supportant les modèles Mistral et Qwen, garantissant la confidentialité totale du code analysé.

1.3.2 Définition de la mission

Dans le cadre de notre projet de fin d’études pour l’obtention du diplôme Ingénieur à l’Institut supérieur d’informatique et de multimédia de Sfax (ISIMS), Mobelite nous a confié la conception et le développement de la plateforme **InvisiThreat**. Cette plateforme vise à centraliser et automatiser les analyses de sécurité applicative (SAST, DAST), à enrichir les résultats par intelligence artificielle et à s’intégrer nativement dans les pipelines DevOps de l’entreprise.

1.3.3 Problématique

Les pratiques de sécurité traditionnelles reposent sur des audits manuels périodiques, structurellement inadaptés aux cadences des pipelines CI/CD modernes. Chaque déploiement peut introduire de nouvelles vulnérabilités sans que les mécanismes de contrôle classiques ne les interceptent à temps.

La multiplication des outils de scan isolés aggrave la situation : formats hétérogènes (SARIF, JSON propriétaire, HTML), nomenclatures de sévérité incompatibles, corrélation manuelle laborieuse. L'absence d'historique centralisé par projet empêche par ailleurs toute mesure objective de la maturité sécurité dans le temps.

La problématique centrale est donc : **comment orchestrer de manière cohérente et automatisée des analyses SAST et DAST, enrichir leurs résultats par intelligence artificielle, normaliser les alertes selon une nomenclature unifiée et fournir aux équipes une vision priorisée, contextualisée et historisée du risque applicatif, intégrable dans un pipeline DevOps existant ?**

1.3.4 Objectifs du projet

InvisiThreat poursuit des objectifs complémentaires :

1. **Centraliser** l'orchestration des analyses SAST et DAST via un point d'entrée unique.
2. **Normaliser** les résultats vers un modèle interne commun et calculer un **scoring de risque** exploitable.
3. **Enrichir** les vulnérabilités via un assistant IA local fournissant des explications et recommandations contextualisées.
4. **Gérer** un **workflow de vulnérabilités** (états, assignation, commentaires) et assurer la **traçabilité** par audit logs.
5. **Intégrer** GitHub (OAuth) et des **clés API** pour déclencher les scans depuis un CLI ou un pipeline CI/CD.
6. **Notifier** en temps réel et fournir des exports exploitables (PDF/JSON) pour l'audit.
7. **Historiser** les scans par projet et par version afin de suivre l'évolution du risque.

1.3.5 Périmètre du projet

InvisiThreat couvre les dimensions suivantes :

- **SAST** : moteurs **Semgrep** (analyse polyglotte par règles) et **Bandit** (inspection Python).
- **DAST** : orchestration via l'**API REST d'OWASP ZAP** en mode daemon.
- **Assistant IA** : modèles LLM locaux via **Ollama** avec action *Assist with AI*.
- **Gouvernance** : JWT, RBAC, 2FA TOTP, audit logs et workflow de vulnérabilités.
- **Intégrations** : GitHub OAuth, clés API et CLI scanner.
- **Stack technique** : backend FastAPI, workers Celery, Redis, PostgreSQL, frontend React, notifications temps réel via Socket.IO, conteneurisation Docker Compose.

Le projet se concentre exclusivement sur la sécurité applicative web et ne couvre pas les infrastructures réseau ni les composants matériels.

1.4 Étude et critique de l'existant

1.4.1 Panorama des solutions existantes

Une analyse des principaux outils du marché a été conduite selon trois catégories : SAST, DAST et plateformes intégrées.

1.4.1.1 SonarQube

Plateforme d'inspection continue de la qualité et de la sécurité du code, largement adoptée en CI/CD. La version Community offre une couverture des règles de sécurité réduite, sans capacités DAST ni enrichissement intelligent. Les éditions Enterprise restent coûteuses pour les structures intermédiaires.



FIGURE 1.2 – Logo SonarQube

1.4.1.2 Burp Suite Professional

Référence de l'industrie en analyse dynamique web. Son moteur de scan actif est reconnu pour sa précision. Cependant, les licences annuelles sont élevées, l'automatisation complète est réservée à l'édition Pro et l'outil ne dispose d'aucune capacité SAST ni d'historique de scans exploitable.



FIGURE 1.3 – Logo Burp Suite

1.4.1.3 Checkmarx

Solution SAST entreprise offrant une analyse approfondie du code source multi-langages. Son coût prohibitif, son fonctionnement en boîte noire et l'absence native de DAST et d'intelligence artificielle limitent son adéquation au contexte du projet.



FIGURE 1.4 – Logo Checkmarx

1.4.1.4 Snyk

Plateforme SaaS combinant des capacités de sécurité du code et de vérification des dépendances avec une forte orientation vers la sécurité applicative. Sa nature cloud impose une transmission du code vers des serveurs externes (contrainte de confidentialité), et elle ne propose ni DAST ni enrichissement par IA.



FIGURE 1.5 – Logo Snyk

1.4.1.5 OWASP ZAP

Outil open source de référence pour l'analyse dynamique. Puissant et extensible via API REST, il ne couvre néanmoins pas l'analyse statique et ne propose aucun historique centralisé ni enrichissement intelligent des alertes.



FIGURE 1.6 – Logo OWASP ZAP

1.4.2 Critiques et recommandations

1.4.2.1 Synthèse comparative

Le tableau 1.2 présente une comparaison des solutions selon les critères clés du projet.

TABLE 1.2 – Comparatif des solutions de sécurité applicative

Outil	SAST	DAST	Open Source	Enrichissement IA	Historique
SonarQube	✓	✗	Partiel	✗	Partiel
Burp Suite	✗	✓	✗	✗	Limité
Checkmarx	✓	✗	✗	✗	✓
Snyk	Partiel	✗	Partiel	✗	✓
OWASP ZAP	✗	✓	✓	✗	✗
InvisiThreat	✓	✓	✓	✓	✓

1.4.2.2 Critique de l'existant

L'analyse met en évidence une lacune commune à l'ensemble de l'écosystème : aucune solution ne couvre simultanément l'analyse statique, l'analyse dynamique, la restitution unifiée des résultats et leur enrichissement par intelligence artificielle, au sein d'une interface accessible sans infrastructure dédiée coûteuse.

a) Avantages des solutions existantes

- Outils matures avec une large base d'utilisateurs et une documentation étendue.
- Certaines solutions (OWASP ZAP, Semgrep) sont open source et gratuitement accessibles.
- Intégrations CI/CD disponibles pour les principales plateformes (GitHub, GitLab, Jenkins).

b) Limites des solutions existantes

- Aucune solution ne couvre simultanément SAST et DAST dans une interface unifiée.
- Absence d'enrichissement par intelligence artificielle pour la contextualisation des alertes.
- Formats de résultats hétérogènes (SARIF, JSON, HTML) rendant la corrélation manuelle.
- Coût prohibitif des solutions entreprise ou envoi du code vers des serveurs externes (SaaS).
- Absence d'historique centralisé empêchant le suivi de l'évolution du risque dans le temps.

1.4.3 Solution proposée

Face à ces lacunes, **InvisiThreat** propose une approche intégrée reposant sur cinq principes fondateurs :

- **Pipeline unifié** : un point d'entrée unique pour déclencher les scans SAST (Semgrep, Bandit) et DAST (OWASP ZAP).
- **Normalisation et scoring** : alertes converties vers un modèle commun, sévérité unifiée et score de risque par projet.
- **Enrichissement IA** : contextualisation des alertes et recommandations de remédiation via un assistant LLM local.
- **Workflow et traçabilité** : suivi des vulnérabilités par état, assignation, historique et audit logs.
- **Accessibilité opérationnelle** : notifications temps réel, exports PDF/JSON, GitHub OAuth, clés API et API REST pour l'intégration CI/CD.

1.5 Méthodologie de développement

1.5.1 Choix méthodologique : Scrum

La nature d’InvisiThreat — intégration de moteurs tiers, ajustement itératif des règles de détection, incorporation d’un LLM — impose une approche capable d’absorber les incertitudes techniques et d’intégrer progressivement les retours utilisateurs. Une méthodologie **Agile Scrum** s’est naturellement imposée.

Les approches agiles considèrent qu’une planification exhaustive en amont est peu adaptée aux projets complexes : spécifier l’intégralité d’un produit dès le départ limite la capacité d’adaptation. Elles privilégient donc des cycles courts et répétés, en découpant le projet en mini-projets priorisés selon la valeur et le besoin, afin d’obtenir des retours rapides et d’ajuster la trajectoire en continu.

1.5.1.1 Étude comparative des méthodologies

Le tableau 1.3 compare les principales méthodologies envisagées pour le projet.

TABLE 1.3 – Comparaison des méthodologies de développement

Critère	Cycle en V / RUP Cascade		Scrum
Flexibilité	Faible	Moyenne	Forte
Adaptation au changement	Difficile	Possible	Naturelle
Livraisons	Unique en fin	Phases	Itératives (sprints)
Communication	Formelle	Structurée	Continue
Adapté à ce projet	Non	Partiellement	Oui

1.5.1.2 Valeurs fondamentales de l’Agile

Les méthodes agiles s’appuient sur quatre valeurs fondamentales :

- l’interaction entre acteurs plutôt que les processus et les outils ;
- un produit opérationnel plutôt qu’une documentation pléthorique ;
- la collaboration avec le client plutôt que la négociation contractuelle ;
- la réactivité au changement plutôt que le suivi rigide d’un plan.

1.5.1.3 Présentation de Scrum

Scrum est une méthode agile apparue au début des années 2000, dont le nom est emprunté au rugby et signifie « mêlée ». Elle repose sur un découpage du projet en itérations courtes appelées *sprints*, avec un objectif clair à chaque cycle. Le principe est de livrer en priorité les fonctionnalités à forte valeur, tout en garantissant la qualité et un retour utilisateur rapide.

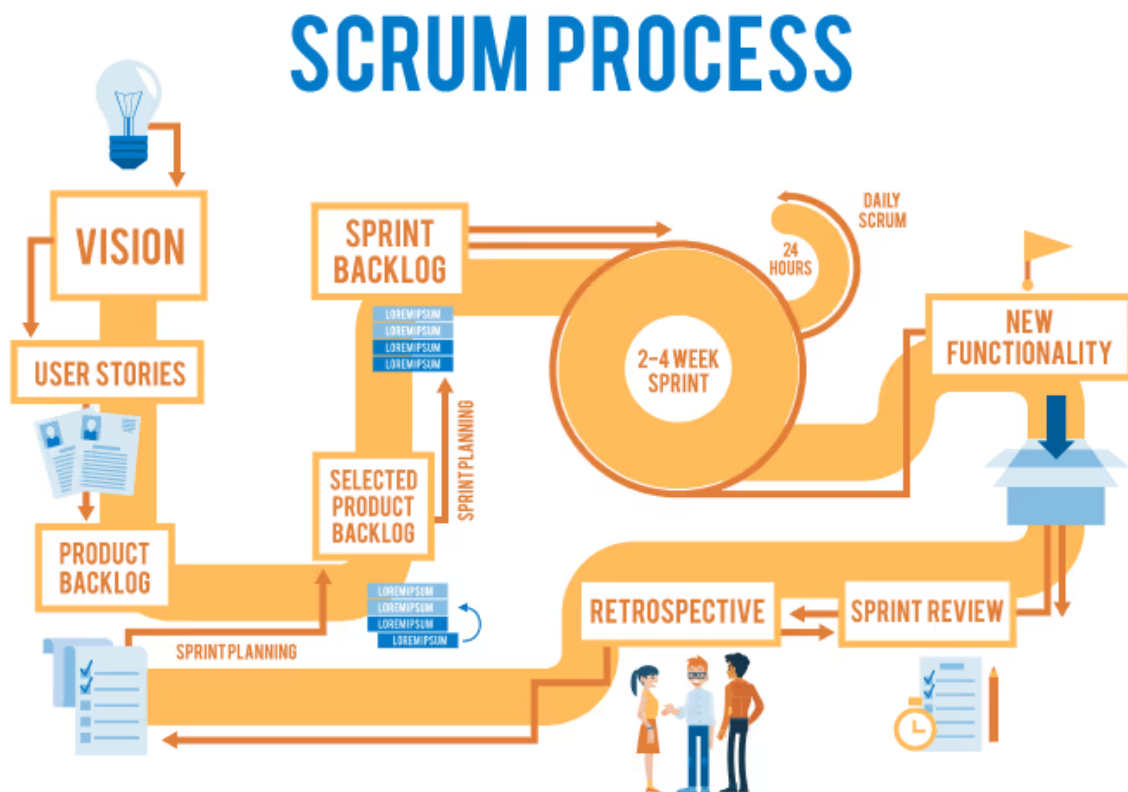


FIGURE 1.7 – Cycle Scrum appliqué au projet InvisiThreat.

1.5.1.4 Les trois piliers de Scrum

Scrum repose sur trois piliers fondamentaux :

- **Transparence** : les aspects importants du processus doivent être visibles et compris par tous les membres de l'équipe.
- **Inspection** : un bilan régulier est réalisé sur les résultats produits afin de détecter tout écart indésirable.
- **Adaptation** : lorsqu'un écart est constaté, le processus est ajusté par des actions correctives ciblées.

1.5.2 Rôles et cérémonies

Scrum définit trois rôles, associés au projet comme suit :

- **Product Owner** (priorisation du backlog selon la valeur et le risque technique) : *****.
- **Scrum Master** (garant du cadre et facilitateur) : *****.
- **Équipe de développement** (auto-organisée, responsable de la qualité des livrables) : Wiem Bziouich, Nouha Boughnim.

Les cérémonies structurantes sont le *Sprint Planning*, le *Daily Scrum*, la *Sprint Review* et la *Sprint Retrospective*, assurant une visibilité continue sur l'avancement et les obstacles.

1.5.2.1 Artefacts de Scrum

- **Product Backlog** : liste priorisée de toutes les fonctionnalités (user stories) du produit, sous la responsabilité du Product Owner.
- **Sprint Backlog** : sous-ensemble du Product Backlog sélectionné pour un sprint donné, avec la liste des tâches associées.
- **Burndown Chart** : courbe graphique permettant de suivre la progression de l'équipe par rapport à l'objectif du sprint.
- **TaskBoard** : tableau (To Do / In Progress / Done) visible par tous les acteurs, reflétant l'état des tâches en temps réel.

1.6 Conclusion

Ce chapitre a posé le cadre général du projet InvisiThreat : identification des lacunes des pratiques de sécurité existantes chez Mobelite, analyse critique des solutions du marché et présentation de la plateforme proposée combinant SAST, DAST et enrichissement par LLM au sein d'un environnement unifié, traçable et intégrable dans les workflows DevOps. La méthodologie Scrum retenue garantit une progression maîtrisée et une validation régulière des modules critiques.

Le chapitre suivant est consacré à l'analyse et à la conception détaillée du système : spécification des besoins fonctionnels et non fonctionnels, identification des acteurs et modélisation UML.

Chapitre

2

Sprint 0 – Initialisation et étude technique

2.1 Introduction

Dans la continuité du chapitre précédent, le **Sprint 0** constitue la phase de fondation du projet InvisiThreat. Il a pour objectif de formaliser les bases nécessaires à un développement maîtrisé : identification des acteurs, spécification des besoins fonctionnels et non fonctionnels, structuration du Product Backlog, planification des sprints et étude technique initiale portant sur les environnements, les technologies retenues et l'architecture du système.

2.2 Identification des acteurs

Avant d'entamer la présentation des diagrammes, il convient d'identifier les acteurs en interaction avec le système. En modélisation UML, un acteur désigne un rôle joué par une entité externe — utilisateur humain, dispositif matériel ou système tiers — qui communique directement avec le système étudié. Il peut consulter ou modifier l'état du système en échangeant des messages porteurs de données.

L'identification préalable des acteurs permet de structurer les responsabilités, de délimiter les périmètres d'accès et de justifier les mécanismes d'autorisation et de traçabilité qui seront mis en place. InvisiThreat implique cinq acteurs distincts, dont les rôles sont synthétisés dans le tableau 2.1.

TABLE 2.1 – Acteurs du système InvisiThreat et leurs responsabilités

Acteur	Rôle principal
Admin	Gouvernance globale de la plateforme, gestion des comptes utilisateurs, configuration des intégrations CI/CD et supervision des politiques de sécurité.
Développeur	Déclenchement des scans sur ses projets rattachés, consultation des résultats et application des correctifs identifiés.
Security Manager	Pilotage de la posture sécurité, priorisation des alertes selon le risque, production et diffusion de rapports de sécurité.
Viewer	Accès en lecture seule aux tableaux de bord et aux rapports, sans pouvoir intervenir sur les configurations ou les états des vulnérabilités.
Systèmes externes	Pipelines CI/CD, dépôts Git et moteur OWASP ZAP interagissant avec la plateforme via l'API REST pour déclencher des scans ou récupérer des résultats.

2.3 Analyse des besoins

L'analyse des besoins d'InvisiThreat se situe à l'intersection de contraintes métier, techniques et sécuritaires. Sur le plan métier, l'enjeu central est de garantir une continuité de livraison logicielle sans compromettre la sécurité, tout en offrant aux équipes une visibilité claire et exploitable sur le niveau de risque par projet. Sur le plan technique, la solution doit s'intégrer à l'écosystème DevOps existant (pipelines CI/CD, dépôts Git, environnements conteneurisés) et traiter des volumes d'alertes croissants sans dégrader les temps de réponse.

Les besoins des équipes sécurité et développement convergent vers une même exigence : disposer d'un processus structuré de gestion des vulnérabilités, fondé sur la traçabilité, la priorisation et l'automatisation. Cela suppose un modèle de données commun pour les alertes, une historisation par version applicative, et des mécanismes d'orchestration découplant les analyses lourdes en tâches asynchrones. L'efficacité de la plateforme dépend également de sa capacité à réduire le bruit (faux positifs) et à transformer les résultats bruts en décisions de remédiation pilotables — d'où l'importance d'une couche d'enrichissement par intelligence artificielle.

2.3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les capacités que la plateforme doit offrir pour couvrir l'ensemble du cycle de gestion des vulnérabilités. Chaque fonctionnalité est motivée par un objectif métier précis, un mécanisme technique sous-jacent et un apport explicite en termes de sécurité ou de gouvernance.

- **Authentification JWT** : authentification stateless reposant sur des jetons d'accès à courte durée de vie et des mécanismes de rafraîchissement, afin d'équilibrer sécurité et performance. La contrainte principale porte sur la gestion des sessions révoquées et la synchronisation temporelle des services.

- **Authentification forte (2FA TOTP)** : second facteur d'authentification basé sur des codes temporels pour les comptes à privilèges, réduisant l'exposition aux compromissions par vol d'identifiants.
- **Contrôle d'accès par rôles (RBAC)** : séparation des responsabilités via une matrice de droits évaluée à chaque requête, limitant l'accès excessif et facilitant l'audit des permissions.
- **Journalisation des actions sensibles (Audit logs)** : enregistrement horodaté des opérations critiques — connexion, lancement de scan, changement d'état de vulnérabilité — afin de satisfaire aux exigences de conformité et d'investigation.
- **Gestion des projets et des dépôts** : centralisation des projets, dépôts Git et URLs cibles, avec paramètres de scan configurables et stockage sécurisé des secrets (tokens, clés).
- **Intégration GitHub OAuth** : rattachement des dépôts et branches aux projets pour automatiser la collecte de métadonnées et simplifier l'authentification des équipes de développement.
- **Clés API** : authentification technique des appels provenant de pipelines CI/CD ou d'un CLI, sans exposition d'identifiants utilisateur.
- **Orchestration asynchrone des scans** : planification, exécution en file d'attente, suivi de progression et arrêt des analyses, avec gestion de la concurrence pour garantir la stabilité opérationnelle en charge élevée.
- **Analyse statique SAST** : détection de patterns de code vulnérables via Semgrep et Bandit, avec normalisation des sorties hétérogènes vers un modèle commun. La maîtrise des faux positifs constitue l'enjeu principal.
- **Extensibilité des moteurs** : capacité à brancher de nouveaux scanners et à normaliser leurs sorties vers le même modèle commun, sans modifier le cœur métier.
- **Analyse dynamique DAST** : pilotage du moteur OWASP ZAP par API REST, avec gestion du contexte applicatif et de l'authentification. La durée d'exécution et la disponibilité des environnements cibles constituent les contraintes majeures.
- **Normalisation des vulnérabilités** : harmonisation des sévérités, classifications (CWE/OWASP), preuves et localisations issus de sources hétérogènes, afin de produire une vision cohérente du risque.
- **Scoring de risque** : calcul d'un score agrégé par projet et par version, permettant de prioriser les efforts de remédiation selon des critères objectifs.
- **Workflow de gestion des vulnérabilités** : gestion des états (ouvert, confirmé, corrigé, faux positif), assignation à des responsables et suivi par commentaires, pour structurer et tracer la remédiation.
- **Enrichissement par IA locale (LLM)** : contextualisation des alertes, proposition de correctifs et assistance à la priorisation via l'action *Assist with AI*, exécutée sur un modèle local garantissant la confidentialité du code.
- **Génération de rapports et exports** : production de livrables PDF et JSON filtrables par période, projet et niveau de sévérité, à destination des équipes de direction et d'audit.
- **Tableau de bord et visualisation** : consolidation des métriques de sécurité, des tendances et des taux de remédiation, avec agrégations optimisées pour maintenir la fluidité de l'interface.
- **API REST et intégration CI/CD** : déclenchement et suivi des scans depuis les pipelines de livraison, avec contrats d'API stables, gestion des quotas et authentification robuste.
- **Notifications et alerting** : diffusion par email, webhooks ou canal temps réel lors de la détection de vulnérabilités critiques, avec mécanismes anti-saturation configurables.

- **Historisation et traçabilité des scans** : conservation de l'historique par projet et par version pour permettre les comparaisons longitudinales, les audits et le suivi de la maturité sécurité dans le temps.

2.3.2 Besoins non fonctionnels

Un besoin non fonctionnel est une condition qui spécifie les critères permettant de juger le fonctionnement d'un système, plutôt que ses comportements spécifiques. À la différence des besoins fonctionnels qui définissent *ce que* le système fait, les besoins non fonctionnels définissent *comment* il le fait, en termes de performance, de fiabilité, de sécurité et de maintenabilité.

- **Performance et latence** : les opérations de consultation doivent rester fluides ; les traitements intensifs (scans, normalisation, IA) sont systématiquement déportés vers des workers asynchrones afin de ne pas impacter la réactivité de l'API.
- **Scalabilité** : la montée en charge est assurée par une multiplication horizontale des workers d'orchestration, sans dégradation de la qualité de service pour les utilisateurs actifs.
- **Disponibilité et continuité de service** : supervision des conteneurs et politique de redémarrage automatique pour maintenir la disponibilité dans les pipelines CI/CD, même en cas de panne d'un service secondaire.
- **Sécurité et confidentialité** : chiffrement des échanges (TLS), protection des secrets (tokens, clés API), cloisonnement des données par projet et journalisation exhaustive des accès aux ressources sensibles.
- **Maintenabilité** : architecture modulaire, code documenté et couverture de tests suffisante pour faciliter l'évolution des moteurs de scan et des règles métier sans régressions.
- **Extensibilité** : intégration de nouveaux scanners ou formats d'alertes via une architecture par connecteurs et un modèle de données ouvert, sans remaniement du cœur applicatif.
- **Ergonomie et adoption** : parcours utilisateurs cohérents, statuts explicites et interface claire, afin de minimiser la friction opérationnelle et favoriser l'adoption par les équipes.
- **Traçabilité et auditabilité** : journaux complets incluant l'identité de l'acteur, le contexte de l'action et l'horodatage, pour répondre aux exigences de conformité et d'investigations post-incident.
- **Portabilité** : déploiement entièrement conteneurisé (Docker Compose), garantissant la reproductibilité des environnements et la compatibilité avec les principaux gestionnaires de dépôts et systèmes CI/CD du marché.
- **Résilience et fiabilité** : traitement idempotent des tâches et mécanismes de reprise automatique, assurant la continuité fonctionnelle en cas de pannes partielles (indisponibilité d'un scanner ou du module IA).

2.3.3 Diagramme global des cas d'utilisation

Les cas d'utilisation représentent les séquences d'actions réalisées par le système qui produisent un résultat observable pour un acteur donné. Ils permettent de modéliser les attentes des utilisateurs, de délimiter le périmètre fonctionnel du système et d'améliorer la compréhension de ses interactions avec l'environnement extérieur.

Le diagramme 2.1 présente les interactions entre les cinq acteurs identifiés et les principales fonctionnalités d’InvisiThreat. Les relations *include* matérialisent des étapes systématiquement nécessaires — par exemple, l’authentification est incluse dans tout déclenchement de scan — tandis que les relations *extend* représentent des comportements conditionnels, comme l’export PDF qui étend la consultation des résultats selon le rôle et le contexte.

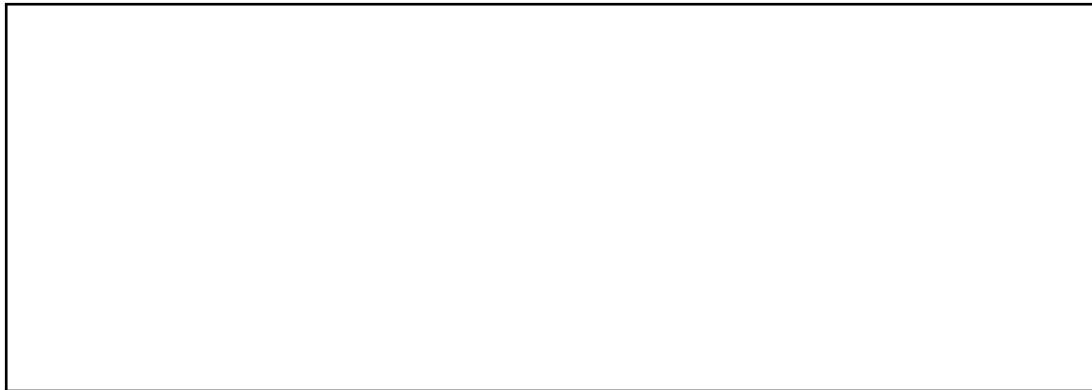


FIGURE 2.1 – Diagramme de cas d’utilisation global d’InvisiThreat.

2.4 Gestion du projet avec Scrum

Le Sprint 0 structure le Product Backlog et prépare la planification des sprints à venir. La priorisation s’appuie sur la criticité sécurité, les dépendances techniques et la valeur métier de chaque fonctionnalité.

2.4.1 Product Backlog

Le Product Backlog formalise la traduction des besoins en unités de valeur livrables. Dans une logique Scrum, il organise le périmètre autour d’epics, de features et de user stories, tout en respectant une priorisation fondée sur le risque et la valeur attendue.

2.4.1.1 Structuration en epics

Les epics regroupent le backlog par grands domaines fonctionnels. L’epic *Authentication et gouvernance* couvre la sécurité des accès et la gestion des rôles. L’epic *Gestion des projets* englobe la configuration des cibles et des dépôts. L’epic *Scans SAST/DAST* porte sur l’orchestration technique des analyses, tandis que l’epic *Résultats, workflow et IA* regroupe la normalisation, le scoring, la remédiation assistée et les exports. Cette structuration favorise la lisibilité du backlog et facilite la planification incrémentale.

2.4.1.2 Technique de priorisation du Product Backlog

La priorisation du Product Backlog conditionne directement la valeur livrée à chaque sprint. Trois critères complémentaires ont été retenus :

- **Business Value** : priorité aux fonctionnalités apportant le maximum de valeur à l'utilisateur final (détection précoce des vulnérabilités, visibilité du risque par projet).
- **Risk Minimizing** : traitement anticipé des éléments les plus complexes ou les plus incertains sur le plan technique (orchestration asynchrone, intégration du module LLM local, pilotage de ZAP par API).
- **MoSCoW** : classification des items selon quatre niveaux — **M** (*Must have*, obligatoire pour la livraison), **S** (*Should have*, fortement souhaitable), **C** (*Could have*, optionnel à valeur ajoutée), **W** (*Would have*, reporté aux versions futures).

2.4.1.3 Synthèse du Product Backlog

Les tableaux suivants présentent l'ensemble des user stories issues du cahier des charges, organisées par domaine fonctionnel et ordonnées selon leur niveau de priorité MoSCoW.

TABLE 2.2 – Product Backlog – Gouvernance et gestion des accès

ID	Feature	US	User Story	Priorité
Module Gouvernance et Accès				
1	Authentification	1.1	En tant qu'administrateur, je veux m'authentifier afin d'accéder à la plateforme et à l'ensemble des fonctionnalités de gouvernance.	M
1	Authentification	1.2	En tant que développeur, je veux m'authentifier afin d'accéder uniquement à mes projets rattachés.	M
2	2FA TOTP	2.1	En tant qu'administrateur, je veux activer l'authentification à deux facteurs afin de renforcer la protection de mon compte.	M
3	RBAC	3.1	En tant qu'administrateur, je veux définir des rôles et des matrices de permissions afin de cloisonner les accès selon les responsabilités.	M
4	Audit logs	4.1	En tant que Security Manager, je veux consulter le journal des actions sensibles afin d'assurer la traçabilité et la conformité.	M
Module Projets et Intégrations				
5	Gestion des projets	5.1	En tant qu'administrateur, je veux créer un projet et définir ses URLs cibles afin de centraliser la configuration des analyses.	M
5	Gestion des projets	5.2	En tant qu'administrateur, je veux configurer les paramètres de scan par projet afin d'adapter les analyses au contexte applicatif.	S
6	Gestion des membres	6.1	En tant qu'administrateur, je veux inviter des membres et leur attribuer des rôles afin de structurer la collaboration sur la plateforme.	S
7	GitHub OAuth	7.1	En tant que développeur, je veux connecter un dépôt GitHub via OAuth afin d'automatiser la collecte des métadonnées de branches.	S
8	Clés API	8.1	En tant que système CI/CD, je veux m'authentifier via une clé API afin de déclencher des scans sans exposer d'identifiants utilisateur.	S

TABLE 2.3 – Product Backlog – Scans et orchestration

ID	Feature	US	User Story	Priorité
Module Scans SAST / DAST et Orchestration				
9	SAST Semgrep/Bandit	9.1	En tant que développeur, je veux lancer une analyse statique afin de détecter les patterns de code vulnérables avant tout déploiement.	M
10	Règles internes	10.1	En tant que Security Manager, je veux appliquer des règles regex personnalisées afin d'adapter la détection aux spécificités du code interne.	S
11	CLI scanner	11.1	En tant que système CI/CD, je veux déclencher une analyse via la ligne de commande afin de l'intégrer directement dans la chaîne de livraison.	S
12	Orchestration asynchrone	12.1	En tant que système, je veux que les scans soient mis en file d'attente via Celery/Redis afin de garantir la stabilité sous charge élevée.	M
13	DAST OWASP ZAP	13.1	En tant que développeur, je veux lancer une analyse dynamique via OWASP ZAP afin de détecter les failles d'exposition réelles de l'application.	M
14	Normalisation	14.1	En tant que Security Manager, je veux consulter des alertes normalisées selon une sévérité unifiée afin de prioriser la remédiation efficacement.	M

TABLE 2.4 – Product Backlog – Workflow, IA, reporting et CI/CD

ID	Feature	US	User Story	Priorité
Module Workflow, IA et Reporting				
16	Assistant IA local	16.1	En tant que développeur, je veux obtenir une explication et une recommandation de correction via <i>Assist with AI</i> afin d'accélérer la remédiation.	M
17	Scoring de risque	17.1	En tant que Security Manager, je veux consulter un score de risque par projet afin de prioriser les efforts de sécurité selon des critères objectifs.	M
18	Workflow vulnérabilités	18.1	En tant que développeur, je veux changer l'état d'une vulnérabilité et l'assigner à un responsable afin de structurer le processus de remédiation.	M
19	Dashboard global	19.1	En tant que Security Manager, je veux visualiser un tableau de bord consolidé afin d'avoir une vue d'ensemble du niveau de risque sur l'ensemble des projets.	S
20	Détail des vulnérabilités	20.1	En tant que développeur, je veux consulter les preuves et le contexte d'une alerte afin de comprendre son impact et de la reproduire.	M
21	Génération de rapports	21.1	En tant que Security Manager, je veux générer un rapport PDF ou JSON par projet afin de le partager lors des revues de sécurité.	S
22	Historique comparatif	22.1	En tant que Security Manager, je veux comparer les résultats de scan entre versions afin de mesurer la progression de la maturité sécurité.	S
Module CI/CD et Notifications				
23	Déclenchement CI/CD	23.1	En tant que système CI/CD, je veux déclencher automatiquement un scan sur commit ou merge request afin d'intégrer la sécurité au pipeline de livraison.	M
24	Notifications temps réel	24.1	En tant que développeur, je veux recevoir des notifications instantanées lors de la détection de vulnérabilités critiques afin de réagir sans délai.	S
25	Politiques d'alerte	25.1	En tant que Security Manager, je veux configurer des seuils d'alerte par sévérité afin d'adapter le niveau de notification au contexte du projet.	S
26	Intégration Slack	26.1	En tant que Security Manager, je veux recevoir les alertes critiques dans un canal Slack afin de centraliser les notifications dans l'outil de communication d'équipe.	W

2.4.2 Planification des sprints

La planification de sprint répond à deux questions fondamentales : qu'est-ce qui peut être livré à l'issue de ce sprint ? Et de quelle façon le travail sélectionné sera-t-il conduit ? Pour InvisiThreat, la durée totale du projet a été estimée à **quatre mois**, découpée en cinq sprints de deux à trois semaines chacun, conformément aux préconisations Scrum pour des projets de taille intermédiaire.

La répartition des responsabilités au sein de l'équipe est la suivante :

- **Stagiaire 1** : développement backend, module IA/LLM, DevSecOps et orchestration des scans.
- **Stagiaire 2** : développement frontend, tableaux de bord et intégration CI/CD.
- **Encadrant professionnel** : supervision technique et validation des choix d'architecture.
- **Encadrant académique** : suivi méthodologique et validation scientifique.

Le planning détaillé est présenté au tableau 2.5. La progression suit une logique de construction en couches : fondations techniques (Sprint 0), gouvernance des accès (Sprint 1), gestion des projets (Sprint 2), analyses de sécurité SAST et DAST (Sprint 3), workflow collaboratif de remédiation (Sprint 4), puis restitution analytique et assistance IA (Sprint 5).

TABLE 2.5 – Planification et livrables des sprints InvisiThreat

Sprint	Objectif		Contenu	Livrables	Durée
0	Cadrage fondations	et	Initialisation du dépôt, configuration Docker Compose, modélisation de la base de données, mise en place des squelettes FastAPI et React.	Socle technique opérationnel, schéma PostgreSQL validé, services démarrables.	2 semaines
1	Sécurité gouvernance	et	Authentification JWT, RBAC, 2FA TOTP, audit logs, politiques de sécurité.	Module d'authentification sécurisé et journalisation opérationnelle.	3 semaines
2	Projets intégrations	et	CRUD projets et membres, intégration GitHub OAuth, gestion des clés API, notifications de base.	Gestion de projets fonctionnelle, intégration GitHub et clés API opérationnelles.	3 semaines
3	SAST et DAST		Intégration Semgrep et Bandit, CLI scanner, normalisation des alertes SAST; intégration OWASP ZAP en mode daemon, workers Celery/Redis, suivi temps réel Socket.IO, scoring de risque.	Pipeline SAST opérationnel, pipeline DAST asynchrone, score de risque calculé.	3 semaines
4	Workflow de vulnérabilités	de	Synchronisation des tâches de remédiation, assignation aux développeurs, machine d'états (open → in_progress → fixed → verified), commentaires collaboratifs, notifications temps réel.	Système de gestion des tâches opérationnel, notifications Socket.IO fonctionnelles.	3 semaines
5	Dashboard, Recommandation LLM et Rapport		Tableau de bord agrégé (métriques, tendances); assistant IA local (<i>Assist with AI</i>) via Ollama (Mistral/Qwen); génération de rapports de sécurité PDF/JSON (ReportLab).	Dashboard opérationnel, recommandations IA persistées, rapports exportables.	3 semaines

2.5 Étude technique

2.5.1 Environnement matériel

Le projet a été réalisé sur deux postes de travail homogènes, un par stagiaire, afin de garantir des conditions de développement et de test reproductibles. Ce dimensionnement répond à la nature des traitements effectués : l'analyse statique nécessite un accès rapide au système de fichiers et une charge CPU stable, tandis que l'analyse dynamique sollicite davantage la mémoire et la latence réseau pour piloter les scans actifs. L'isolation des conteneurs — notamment OWASP ZAP — dans des processus dédiés limite la contention avec l'API et l'interface, et contribue à stabiliser les temps de réponse observés lors des tests.

TABLE 2.6 – Configuration matérielle des postes de travail

Élément	PC 1 – Wiem Bziouich	PC 2 – Nouha Boughnim
Processeur	Intel Core i7	Intel Core i7
RAM installée	16 Go	16 Go
Système	Windows 11 Pro 64 bits	Windows 11 Pro 64 bits
Stockage	SSD 512 Go	SSD 512 Go

2.5.2 Environnement logiciel

Le tableau 2.7 regroupe l'ensemble des outils utilisés par l'équipe pour le développement, les tests, la modélisation et la gestion du projet. Ces outils constituent l'environnement de travail au sens ingénierie logicielle, distinct de la stack technique déployée en production.

TABLE 2.7 – Environnement logiciel utilisé pour le développement

Logo	Outil	Description et rôle dans le projet
	VS Code	IDE principal pour le développement Python et JavaScript, avec extensions de linting, d'intégration Git et de test des APIs REST.
	Git / GitHub	Contrôle de version distribué, collaboration par branches et revue de code pour assurer la traçabilité des évolutions.
	Docker Desktop	Exécution locale des conteneurs pour reproduire fidèlement l'environnement de production et éviter les conflits de dépendances.
	Postman	Test des endpoints REST, validation des payloads et reproduction des scénarios de scan pour vérifier les contrats d'API.
	Draw.io	Modélisation des diagrammes UML : cas d'utilisation, déploiement et architecture logique.
	Trello / Jira / ClickUp	Gestion du Product Backlog, suivi de l'avancement des sprints et tableaux Kanban partagés entre les membres de l'équipe.
	Chrome / Firefox DevTools	Validation de l'interface React, inspection des requêtes réseau et analyse des performances côté client.
	venv / pip	Isolation des dépendances Python par environnement virtuel, garantissant la cohérence entre les deux postes de développement.
	Node.js / npm	Gestion des dépendances et build du frontend React, avec contrôle précis des versions des bibliothèques côté client.

2.5.3 Technologies et outils utilisés

Le tableau 2.8 présente la stack technique déployée pour assurer la détection, la normalisation et la restitution des résultats de sécurité. Chaque choix est justifié par des exigences de performance, de sécurité et de maintenabilité, et accompagné d'une mise en perspective par rapport aux alternatives.

TABLE 2.8 – Stack technique d’InvisiThreat – technologies et justifications

Logo	Technologie	Rôle, justification et contraintes
 python™	Python FastAPI Backend API	/ Expose une API REST asynchrone capable de traiter des requêtes concurrentes sans blocage lors des scans longs. FastAPI apporte un typage strict et une validation automatique des schémas (Pydantic), ce qui réduit les erreurs d’intégration par rapport à Flask. Sa structure légère est mieux adaptée qu’un framework Django pour un service d’orchestration découplé.
	React Frontend	Bibliothèque de construction d’interfaces riches et réactives, adaptée à la visualisation temps réel des états de scan. Son écosystème étendu et sa flexibilité de composition des composants le distinguent de Vue ou Angular pour des tableaux de bord complexes. La gestion d’état est assurée via hooks et contextes React.
	PostgreSQL Base de données	Assure la persistance transactionnelle (ACID) et les agrégations avancées nécessaires à l’historisation des scans et aux analyses de tendances par sévérité. Offre une richesse fonctionnelle supérieure à MySQL pour les requêtes analytiques. La contrainte principale est l’optimisation des index sur de grands volumes d’alertes.
	Redis Broker & cache	Utilisé comme broker de messages pour Celery et stockage temporaire des états de scan (progression, files d’attente, verrous distribués). Sa nature en mémoire garantit des temps d’accès très faibles et décharge PostgreSQL pour les données éphémères. La stratégie d’expiration des clés constitue la contrainte principale.
	Celery Orchestration async.	Exécute les analyses SAST et DAST dans des workers isolés sans bloquer l’API principale. Fournit un mécanisme de file d’attente, de retries configurables et de suivi d’exécution adapté aux tâches longues. La supervision des workers et la gestion fine de la concurrence constituent les contraintes opérationnelles majeures.
	Socket.IO Temps réel	Assure la diffusion en temps réel des états de scan et des notifications critiques vers le frontend, éliminant la latence d’une interrogation périodique (polling). La gestion des connexions persistantes et la montée en charge représentent les contraintes principales.
	Docker / Docker Compose Conteneurisation	Constitue le socle de déploiement de l’ensemble des services, garantissant la reproductibilité des environnements et la séparation des composants critiques. Réduit la surcharge par rapport aux machines virtuelles et s’intègre naturellement aux pratiques DevSecOps. La

2.5.4 Architecture du système

2.5.4.1 Architecture logicielle

L'architecture logicielle d'InvisiThreat repose sur une séparation claire des responsabilités en quatre couches fonctionnelles :

- **Couche API** : gestion des projets, des accès utilisateurs, du déclenchement des scans et de l'exposition des résultats via des endpoints REST documentés.
- **Couche d'orchestration** : exécution asynchrone des analyses longues (SAST, DAST) via des workers Celery, avec file d'attente Redis et suivi de progression.
- **Couche de persistance** : stockage normalisé des vulnérabilités, de l'historique des scans et des artefacts d'audit dans PostgreSQL.
- **Couche de présentation** : interface React pour la visualisation, le workflow de remédiation et les notifications temps réel via Socket.IO.

Le flux principal suit une chaîne continue : déclenchement d'un scan, exécution asynchrone via les workers, normalisation des alertes vers le modèle commun, calcul du scoring de risque, enrichissement par le module IA, puis exposition des résultats via les endpoints REST et le canal temps réel.

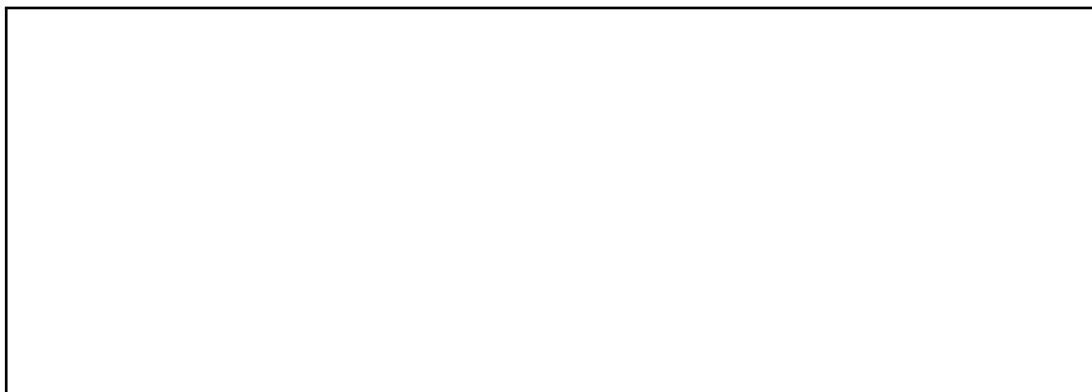


FIGURE 2.2 – Architecture logicielle en couches d'InvisiThreat.

2.5.4.2 Architecture physique

Sur le plan physique, InvisiThreat est déployé via **Docker Compose** selon une organisation en trois niveaux :

- **Niveau client** : navigateur React servi par un conteneur Nginx, communiquant avec le backend via HTTP/REST et Socket.IO.
- **Niveau serveur d'application** : conteneurs FastAPI, workers Celery, broker Redis et module IA Ollama, isolés et communicants par réseau Docker interne.
- **Niveau données** : conteneur PostgreSQL pour la persistance, avec volumes montés pour la durabilité des données entre redémarrages.

InvisiThreat adopte une architecture de **monolithe modulaire** côté backend, enrichie par une orchestration asynchrone. Les moteurs de scan (ZAP, Semgrep, Bandit) et le module IA sont

isolés dans des conteneurs dédiés, sans imposer la complexité opérationnelle d'une architecture microservices complète. La scalabilité horizontale est assurée par la multiplication des workers Celery ; la tolérance aux pannes est renforcée car la défaillance d'un scanner n'interrompt pas les services de consultation ; et l'observabilité (logs structurés, métriques, audit trail) garantit une traçabilité de bout en bout.

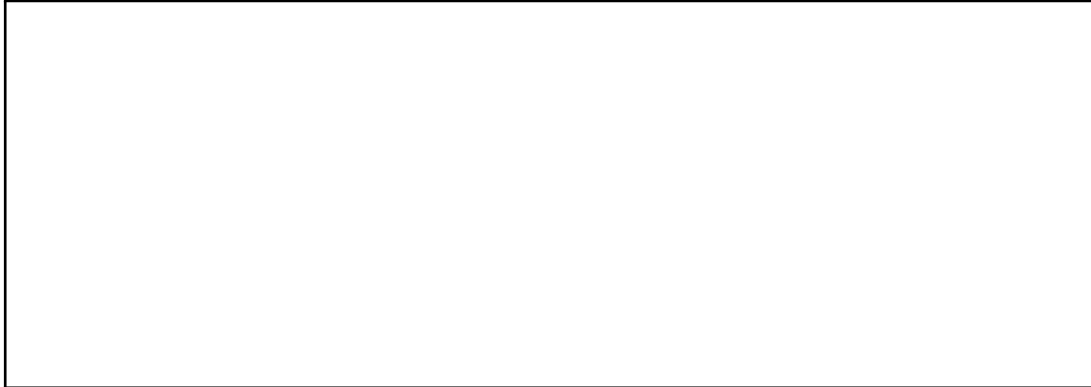


FIGURE 2.3 – Architecture physique – déploiement Docker Compose d'InvisiThreat.

2.6 Diagramme de déploiement

Le diagramme de déploiement illustre l'organisation conteneurisée de la plateforme via Docker Compose, les flux réseau entre services et l'exposition des ports applicatifs (API FastAPI, frontend React, moteur ZAP, module Ollama). Il permet de visualiser l'isolation des composants critiques et les dépendances de communication entre niveaux.



FIGURE 2.4 – Diagramme de déploiement Docker Compose d'InvisiThreat.

2.7 Conclusion

Ce chapitre a posé les fondations méthodologiques et techniques du projet InvisiThreat. L'identification des acteurs et la spécification exhaustive des besoins fonctionnels et non fonctionnels ont permis de délimiter précisément le périmètre de la plateforme. La structuration du Product Backlog selon une priorisation MoSCoW et la planification en cinq sprints garantissent une progression maîtrisée, alignée sur la valeur métier et la gestion du risque

technique. Enfin, l'étude technique a confirmé la cohérence de la stack retenue avec les exigences DevSecOps : architecture modulaire, orchestration asynchrone, enrichissement par IA locale et déploiement intégralement conteneurisé.

Le chapitre suivant présente le **Sprint 1**, consacré à la sécurité et à la gouvernance : authentification JWT, contrôle d'accès par rôles, double facteur TOTP et journalisation des actions sensibles.

Chapitre

3

Sprint 1 – Sécurité et gouvernance

3.1 Introduction

Ce sprint vise à mettre en place le socle de sécurité et de gouvernance de la plateforme InvisiThreat. L'objectif est de garantir un accès sécurisé, une traçabilité complète des actions sensibles et un modèle d'autorisation cohérent pour les différents profils d'utilisateurs. Les modules développés au cours de ce sprint constituent le fondement sur lequel s'appuient l'ensemble des fonctionnalités des sprints ultérieurs.

3.2 Objectifs du sprint et planification des tâches

3.2.1 Objectifs attendus

À l'issue de ce sprint, il convient de disposer d'une couche d'authentification opérationnelle, d'un système de contrôle d'accès par rôles et d'une traçabilité exhaustive des opérations sensibles. Plus précisément, les objectifs fixés sont les suivants :

- mettre en place l'authentification JWT avec gestion des sessions (access token 30 min, refresh token révocable);
- définir et implémenter un modèle RBAC pour les rôles admin, developer, security_manager et viewer;
- intégrer l'authentification forte 2FA TOTP (via pyotp) pour les comptes à privilèges;
- assurer la traçabilité complète via des audit logs horodatés.

3.2.2 Sprint Backlog

TABLE 3.1 – Sprint Backlog – Sprint 1

ID	User Story	ID Tâche	Tâche	Priorité
S1-01	En tant qu'utilisateur, je peux m'authentifier et obtenir un jeton JWT.	T1-01	Implémenter l'endpoint POST /auth/login avec génération d'access token (30 min) et refresh token.	M
		T1-02	Implémenter l'endpoint POST /auth/refresh avec validation et révocation du refresh token.	M
S1-02	En tant qu'administrateur, je peux gérer les rôles et permissions (RBAC).	T1-03	Modéliser et persister les entités Role et User avec SQLAlchemy.	M
		T1-04	Implémenter les dépendances FastAPI de vérification du rôle sur les endpoints protégés.	M
S1-03	En tant qu'administrateur, je peux activer/désactiver la 2FA TOTP.	T1-05	Générer un secret TOTP par utilisateur (pyotp.random_base32) et exposer le QR code.	M
		T1-06	Vérifier le code TOTP à chaque connexion si 2FA activée (pyotp.TOTP.verify).	M
S1-04	En tant que Security Manager, je peux consulter l'audit des actions sensibles.	T1-07	Créer l'entité AuditLog et le middleware d'enregistrement automatique.	M
		T1-08	Exposer l'endpoint GET /audit-logs paginé avec filtres par acteur et par type d'action.	M

3.3 Technologies et concepts utilisés

3.3.1 JWT et FastAPI Security (python-jose, OAuth2PasswordBearer)

Description. Le JSON Web Token (JWT) est un standard ouvert (RFC 7519) permettant l'échange sécurisé d'informations entre parties sous la forme d'un objet JSON signé. FastAPI intègre nativement le schéma OAuth2PasswordBearer qui extrait automatiquement le jeton *Bearer* de l'en-tête HTTP Authorization.

Utilisation dans InvisiThreat. L'access token JWT a une durée de validité de **30 minutes**, signé avec l'algorithme HS256 via la bibliothèque python-jose. Le refresh token, d'une durée plus longue, est stocké dans la table auth_tokens (modèle AuthToken) sous forme de hash

SHA-256, révocable individuellement ou en masse à la déconnexion. Toute requête authentifiée traverse le middleware de vérification du jeton avant d’atteindre l’endpoint cible.

Justification. L’approche stateless du JWT élimine la nécessité d’un stockage côté serveur pour la session active, allégeant la charge sur la base de données en lecture. La durée limitée de l’access token (30 min) réduit la fenêtre d’exploitation en cas de fuite, tandis que le mécanisme de révocation du refresh token garantit la déconnexion effective.

3.3.2 Authentification forte TOTP (pyotp)

Description. Le Time-based One-Time Password (TOTP) est défini par la RFC 6238. Il génère un code à six chiffres valable pendant 30 secondes, calculé à partir d’un secret partagé et de l’horodatage courant. La bibliothèque Python pyotp implémente ce standard et est compatible avec Google Authenticator et Authy.

Utilisation dans InvisiThreat. Lorsqu’un utilisateur active la 2FA, un secret aléatoire (`pyotp.random_base32()`) est généré, chiffré et stocké dans le champ `totp_secret` du modèle `User`. À chaque connexion avec 2FA activée, le serveur vérifie le code saisi via `pyotp.TOTP(secret).verify(code)`.

Justification. Le TOTP constitue le second facteur le plus répandu et le mieux supporté sur mobile. Son implémentation locale (sans SMS, sans service tiers) préserve la confidentialité et élimine les coûts opérationnels.

3.3.3 Contrôle d’accès RBAC (bcrypt)

Description. Le Role-Based Access Control (RBAC) est un modèle de contrôle d’accès qui associe des permissions à des rôles, et des rôles à des utilisateurs. `bcrypt` est un algorithme de hachage adaptatif utilisé pour stocker les mots de passe de manière résistante aux attaques par force brute.

Utilisation dans InvisiThreat. Quatre rôles sont définis : `admin` (gouvernance globale), `developer` (déclenchement de scans), `security_manager` (pilotage du risque) et `viewer` (lecture seule). Les mots de passe sont hachés avec `bcrypt` (facteur de coût configurable) avant persistance dans `hashed_password`. Les dépendances FastAPI vérifient le rôle à chaque requête entrante.

Justification. Le RBAC applique le principe du moindre privilège : chaque rôle n’accède qu’aux fonctionnalités nécessaires à sa mission. `bcrypt` résiste aux attaques par tables arc-en-ciel et son facteur de coût est ajustable pour suivre l’évolution des capacités de calcul.

3.4 Analyse et spécification des besoins

3.4.1 Diagramme de cas d’utilisation du sprint

Le diagramme de cas d’utilisation du Sprint 1 (figure 3.1) représente les interactions entre les acteurs du système et les fonctionnalités de sécurité et de gouvernance. Les deux acteurs principaux sont l’**Utilisateur** (tous rôles confondus, pour les cas d’accès général) et l’**Administrateur** (pour la gestion des rôles et de la 2FA). Le cas *S’authentifier* est inclus dans

tous les autres cas via une relation *include*, matérialisant ainsi l'obligation de posséder une session valide. Le cas *Activer la 2FA* étend *S'authentifier* via une relation *extend* conditionnelle, déclenchée uniquement si le flag `totp_enabled` est positionné.

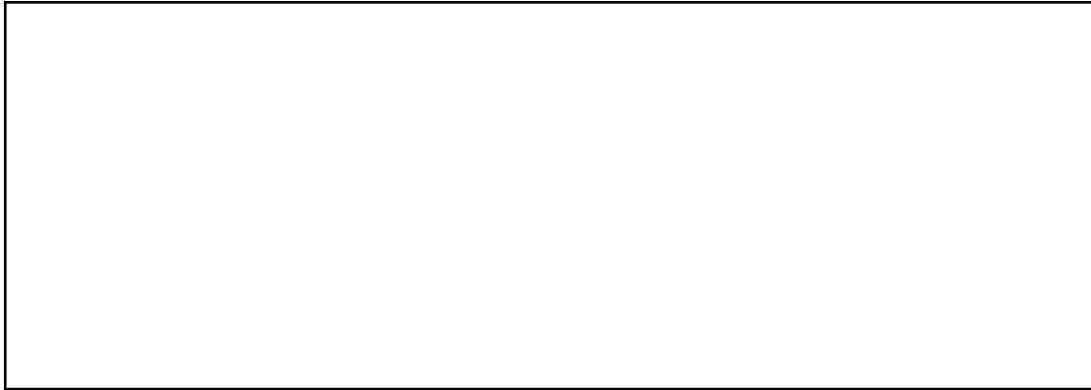


FIGURE 3.1 – Diagramme de cas d'utilisation – Sprint 1.

3.4.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : S'authentifier

TABLE 3.2 – Description du cas d'utilisation – S'authentifier

Titre	S'authentifier
Acteur	Utilisateur (tous rôles : admin, developer, security_manager, viewer)
Résumé	Permet à un utilisateur de se connecter à la plateforme InvisiThreat en soumettant ses identifiants et en obtenant un jeton JWT d'accès (30 min) ainsi qu'un refresh token.
Préconditions	L'utilisateur possède un compte actif (<code>is_active = true</code>) et vérifié (<code>is_verified = true</code>) dans la base de données.
Postconditions	L'utilisateur est authentifié. Un access token JWT (30 min) et un refresh token sont émis. Un enregistrement AuthToken est persisté. Un AuditLog de type <i>LOGIN</i> est créé.
Scénario nominal	1. L'utilisateur soumet son adresse e-mail et son mot de passe via le formulaire de connexion. 2. Le système vérifie l'e-mail dans la base et compare le mot de passe au hash bcrypt stocké. 3. Si la 2FA est activée (<code>totp_enabled = true</code>), le système demande le code TOTP. 4. L'utilisateur saisit le code temporaire à six chiffres. 5. Le système vérifie le code via <code>pyotp.TOTP.verify()</code>. 6. Le système génère un access token JWT signé HS256 (expiration 30 min) et un refresh token. 7. Le refresh token est haché (SHA-256) et persisté dans <code>auth_tokens</code>. 8. Le système retourne les deux tokens et enregistre un AuditLog de connexion.
Scénario alternatif	2.a Identifiants incorrects : le système retourne HTTP 401 avec un message d'erreur générique (sans préciser si l'e-mail ou le mot de passe est erroné) et incrémente le compteur d'échecs. 4.a Code TOTP expiré ou incorrect : le système retourne HTTP 401 et invite l'utilisateur à saisir le code suivant. 2.b Compte inactif ou non vérifié : le système retourne HTTP 403 avec un message indiquant que le compte est suspendu ou en attente de vérification.

b) Cas d'utilisation : Attribuer un rôle (RBAC)**TABLE 3.3 – Description du cas d'utilisation – Attribuer un rôle**

Titre	Attribuer un rôle (RBAC)
Acteur	Administrateur (admin)
Résumé	Permet à un administrateur de modifier le rôle d'un utilisateur existant, en lui assignant l'un des quatre rôles disponibles (admin, developer, security_manager, viewer).
Préconditions	L'administrateur est authentifié. L'utilisateur cible existe dans le système.
Postconditions	Le champ <code>role_id</code> de l'utilisateur cible est mis à jour en base. Un <code>AuditLog</code> de type <code>ROLE_ASSIGN</code> est créé avec l'identité de l'administrateur et le nouveau rôle.
Scénario nominal	1. L'administrateur accède au panneau de gestion des utilisateurs. 2. Il sélectionne l'utilisateur cible et choisit le nouveau rôle dans la liste déroulante. 3. Il confirme la modification. 4. Le système met à jour <code>role_id</code> dans la table <code>users</code> via <code>SQLAlchemy</code>. 5. Le système enregistre un <code>AuditLog</code> horodaté avec <code>action = ROLE_ASSIGN</code>, <code>resource_type = User</code> et <code>resource_id = user.id</code>. 6. Le système notifie l'utilisateur cible du changement de rôle.
Scénario alternatif	2.a Rôle identique au rôle actuel : le système retourne un message d'information sans effectuer de modification ni créer d'audit log. 4.a Erreur de persistance : le système retourne <code>HTTP 500</code> et annule la transaction (rollback <code>SQLAlchemy</code>).

3.5 Modélisation conceptuelle

3.5.1 Diagramme de classes

Le diagramme de classes du Sprint 1 (figure 3.2) modélise les entités impliquées dans la sécurité et la gouvernance. La classe centrale `User` est associée à `Role` par une relation de composition (`role_id` clé étrangère) : un utilisateur possède exactement un rôle. `AuthToken` est liée à `User` par une association `1..N` : un utilisateur peut posséder plusieurs refresh tokens actifs (sessions multiples), chacun révoquant individuellement. `AuditLog` est liée à `User` par une association de journalisation (`1..N`) : chaque entrée référence l'acteur responsable de l'action enregistrée.

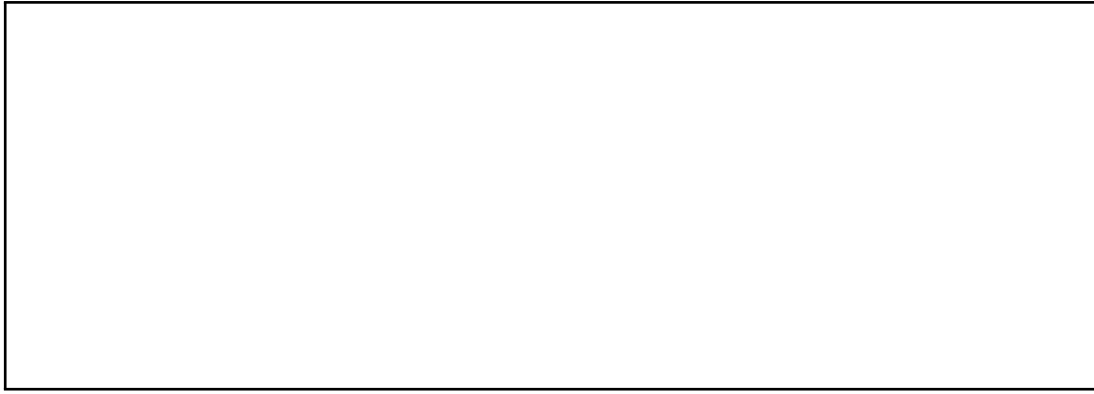


FIGURE 3.2 – Diagramme de classes – Sprint 1 (entités User, Role, AuthToken, AuditLog).

a) Dictionnaire de données

TABLE 3.4 – Dictionnaire de données – Sprint 1

Classe	Code	Désignation	Type
User	id	Identifiant unique de l'utilisateur (UUID)	UUID
User	nom	Nom complet de l'utilisateur	String
User	email	Adresse e-mail (identifiant de connexion, unique)	String
User	hashed_password	Mot de passe haché avec bcrypt	String
User	is_active	Indicateur d'activation du compte	Boolean
User	is_verified	Indicateur de vérification de l'adresse e-mail	Boolean
User	totp_secret	Secret TOTP chiffré (null si 2FA non activée)	String (nullable)
User	totp_enabled	Indicateur d'activation de la 2FA TOTP	Boolean
User	role_id	Référence au rôle RBAC de l'utilisateur	UUID (FK → Role)
User	trial_scans_remaining	Nombre de scans restants en période d'essai	Integer
Role	id	Identifiant unique du rôle	UUID
Role	name	Nom du rôle (admin / developer / security_manager / viewer)	String (enum)
AuthToken	id	Identifiant unique du refresh token	UUID
AuthToken	user_id	Référence à l'utilisateur propriétaire du token	UUID (FK → User)
AuthToken	token_hash	Hash SHA-256 du refresh token (stocké, jamais le token en clair)	String
AuthToken	expires_at	Date et heure d'expiration du token	DateTime
AuthToken	revoked	Indicateur de révocation du token	Boolean
AuditLog	id	Identifiant unique de l'entrée de journal	UUID
AuditLog	user_id	Référence à l'acteur ayant déclenché l'action	UUID (FK → User)
AuditLog	action	Type d'action journalisée (<i>LOGIN</i> , <i>ROLE_ASSIGN</i> , <i>SCAN_START</i> , etc.)	String
AuditLog	resource_type	Type de ressource affectée (<i>User</i> , <i>Scan</i> , <i>Project</i> , etc.)	String
AuditLog	resource_id	Identifiant de la ressource affectée	UUID
AuditLog	timestamp	Horodatage UTC de l'action	DateTime

3.5.2 Diagrammes de séquence

a) Diagramme DS-1a : S'authentifier avec JWT et TOTP

Le diagramme de séquence DS-1a (figure 3.3) modélise le flux d'authentification selon la notation BCE (*Boundary–Control–Entity*). La frontière **LoginPage** (composant React) collecte les identifiants et les transmet à **AuthController** (module FastAPI /auth/login), qui orchestre la vérification. La couche entité est composée de **User** (vérification des identifiants et du TOTP), **AuthToken** (persistance du refresh token) et **AuditLog** (journalisation de la connexion). La base de données PostgreSQL est représentée comme entité de stockage externe.

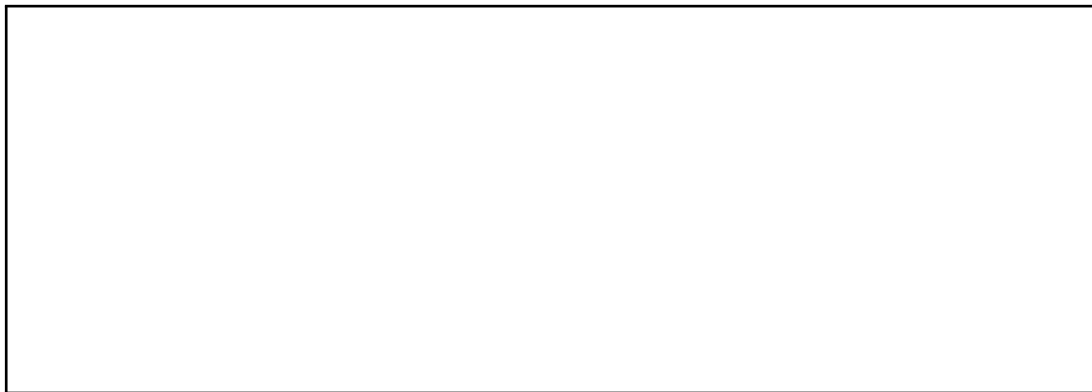


FIGURE 3.3 – Diagramme de séquence DS-1a – S'authentifier (JWT + TOTP optionnel).

b) Diagramme DS-1b : Attribution de rôle par l'administrateur

Le diagramme de séquence DS-1b (figure 3.4) décrit l'attribution de rôle selon la notation BCE. **AdminPanel** (boundary — interface React d'administration) transmet la requête à **AuthController** (control — endpoint FastAPI PUT /users/{id}/role). La dépendance d'autorisation vérifie que l'appelant possède le rôle admin avant toute opération. Les entités **User** et **AuditLog** (SQLAlchemy) sont mises à jour en une seule transaction atomique.



FIGURE 3.4 – Diagramme de séquence DS-1b – Attribution de rôle (RBAC).

3.6 Réalisation

3.6.1 Authentification JWT

La couche d'authentification repose sur un mécanisme stateless : l'access token JWT (durée de vie 30 minutes, signé HS256) encapsule l'identifiant de l'utilisateur et son rôle dans le payload. À chaque requête, le middleware FastAPI decode et valide le token sans consulter la base de données pour l'access token. La rotation du refresh token (stocké sous forme de hash SHA-256 dans `auth_tokens`) est assurée à chaque renouvellement, invalidant le token précédent. La déconnexion révoque explicitement le refresh token en positionnant `revoked = true`.

3.6.2 RBAC et gestion des rôles

Les quatre rôles (`admin`, `developer`, `security_manager`, `viewer`) sont persistés dans la table `roles`. La vérification du rôle est implémentée comme une dépendance FastAPI injectable (`Depends`) sur chaque endpoint protégé. La matrice de permissions garantit que, par exemple, seul `admin` peut modifier les rôles et que `viewer` ne peut qu'accéder aux ressources en lecture.

3.6.3 2FA TOTP et audit logs

L'activation de la 2FA génère un secret `pyotp.random_base32()` persisté chiffré dans `totp_secret`. Un QR code est exposé à l'utilisateur pour enregistrement dans son application TOTP (Google Authenticator, Authy). Chaque action critique (connexion, changement de rôle, lancement de scan, révocation de token) génère automatiquement un enregistrement `AuditLog` via un middleware dédié, garantissant la traçabilité sans modifier la logique métier des endpoints.

3.7 Test et validation

3.7.1 Tests réalisés

TABLE 3.5 – Tableau de test – S’authentifier (Sprint 1)

Cas de Test	TC-S1-01 – Authentification réussie avec 2FA
Auteur du cas de test	Développeur / Testeur QA
Description	Vérifier que l’authentification JWT avec TOTP retourne un access token valide et crée un AuditLog.
Précondition	Un utilisateur avec <code>totp_enabled = true</code> existe dans la base. Un code TOTP valide est généré à partir du secret partagé.
Scénario	1. Envoyer POST <code>/auth/login</code> avec e-mail et mot de passe corrects. 2. Répondre à la demande TOTP avec un code généré via <code>pyotp.TOTP(secret).now()</code>. 3. Recevoir la réponse HTTP 200 avec <code>access_token</code> et <code>refresh_token</code>. 4. Décoder le JWT et vérifier la présence des champs <code>sub (user_id)</code> et <code>exp</code> (expiration à +30 min). 5. Vérifier la création d’un enregistrement AuditLog de type LOGIN dans la base.
Postcondition	L’access token est valide, le refresh token est persisté dans <code>auth_tokens</code> , l’audit log est créé.

TABLE 3.6 – Tableau de test – Attribution de rôle (Sprint 1)

Cas de Test	TC-S1-02 – Attribution de rôle par l’administrateur
Auteur du cas de test	Administrateur / Testeur QA
Description	Vérifier qu’un administrateur peut modifier le rôle d’un utilisateur et que l’action est correctement auditée.
Précondition	Un utilisateur authentifié avec le rôle admin dispose d’un access token valide. Un utilisateur cible avec le rôle viewer existe dans la base.
Scénario	1. Envoyer PUT <code>/users/{id}/role</code> avec <code>Authorization: Bearer <admin_token></code> et <code>{"role": "developer"}</code> dans le corps. 2. Vérifier la réponse HTTP 200 avec le profil utilisateur mis à jour. 3. Vérifier que <code>role_id</code> de l’utilisateur cible est modifié en base. 4. Vérifier la création d’un AuditLog avec <code>action = ROLE_ASSIGN</code> et les identifiants corrects.
Postcondition	Le rôle de l’utilisateur cible est mis à jour. L’audit log retrace l’action avec l’identité de l’administrateur et le nouveau rôle.

3.7.2 Validation des fonctionnalités

La validation s’est appuyée sur des scénarios de bout en bout exécutés via Pytest et la suite `httpx.AsyncClient` de FastAPI. Les scénarios couvrent : la création d’un utilisateur,

l'attribution d'un rôle, les tentatives d'accès aux endpoints protégés selon différents rôles et la vérification des entrées d'audit log produites. Les tests de régression garantissent que toute modification ultérieure de la couche d'autorisation n'introduit pas de permissions non intentionnelles.

3.7.3 Gestion des versions et commits

Le développement du Sprint 1 a été versionné sur la branche `feature/sprint1-auth-rbac`, avec des messages de commit structurés (`feat:`, `fix:`, `test:`) conformes à la convention *Conventional Commits*. Les merge requests vers `develop` ont fait l'objet d'une revue de code avant intégration, avec vérification automatique de la couverture de tests via le pipeline CI/CD GitHub Actions.

3.8 Conclusion

Le Sprint 1 a posé le socle de gouvernance et de sécurité de la plateforme InvisiThreat. Les modules d'authentification JWT (30 min, refresh révocable), de contrôle d'accès par rôles (`admin`, `developer`, `security_manager`, `viewer`), d'authentification forte TOTP (`pyotp`) et de journalisation exhaustive constituent une base solide pour les sprints fonctionnels suivants. L'ensemble de ces mécanismes satisfait aux exigences OWASP relatives à la gestion des sessions, au contrôle d'accès et à la traçabilité.

Chapitre

4

Sprint 2 – Projets et intégrations

4.1 Introduction

Ce sprint est consacré à la gestion des projets et aux intégrations nécessaires pour connecter InvisiThreat à l'écosystème existant. L'objectif est de permettre la création de projets structurés, la connexion de dépôts GitHub via OAuth, la génération et la gestion de clés API pour les pipelines CI/CD, ainsi que la gestion des membres par projet. L'ensemble de ces fonctionnalités repose sur un chiffrement rigoureux des secrets et une séparation stricte des périmètres par projet.

4.2 Objectifs du sprint et planification des tâches

4.2.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit permettre de créer et configurer des projets, de relier des dépôts GitHub (avec chiffrement du token d'accès), de générer des clés API pour l'authentification technique et d'ajouter des membres avec des rôles spécifiques au projet. Plus précisément :

- permettre la création et la configuration des projets (language, analysis_type, visibility);
- intégrer les dépôts GitHub via OAuth avec chiffrement Fernet du token d'accès;
- générer et révoquer des clés API pour l'authentification depuis les pipelines CI/CD;
- gérer les membres d'un projet (rôles owner/member/viewer).

4.2.2 Sprint Backlog

TABLE 4.1 – Sprint Backlog – Sprint 2

ID	User Story	ID Tâche	Tâche	Priorité
S2-01	En tant que développeur ou administrateur, je peux créer un projet et définir ses paramètres (langage, type d'analyse, visibilité).	T2-01	Implémenter l'endpoint POST /projects avec validation Pydantic v2 et persistance SQLAlchemy.	M
		T2-02	Exposer GET /projects paginé avec filtres par propriétaire et visibilité.	M
S2-02	En tant que développeur, je peux connecter un dépôt GitHub via OAuth et conserver le token de manière sécurisée.	T2-03	Implémenter le flux GitHub OAuth (callback + échange de code contre token d'accès).	S
		T2-04	Chiffrer le token GitHub avec cryptography.fernet.Fernet avant persistance dans encrypted_token.	M
S2-03	En tant que développeur ou administrateur, je veux générer une clé API afin d'authentifier les appels depuis un pipeline CI/CD sans exposer mes identifiants.	T2-05	Générer une clé API aléatoire, hacher avec SHA-256, persister dans user_api_keys.	S
		T2-06	Exposer les endpoints POST /api-keys et DELETE /api-keys/{id} (révocation).	S
S2-04	En tant que propriétaire de projet, je peux ajouter des membres et leur attribuer un rôle au sein du projet.	T2-07	Implémenter POST /projects/{id}/members avec gestion des rôles owner/member/viewer.	S

4.3 Technologies et concepts utilisés

4.3.1 GitHub REST API v3 et OAuth2

Description. GitHub expose une API REST (version 3) permettant d'accéder aux dépôts, branches, commits et métadonnées d'organisation via un token d'accès personnel (PAT) ou un token OAuth. Le flux OAuth2 Authorization Code échange un code temporaire contre un token d'accès après consentement de l'utilisateur.

Utilisation dans InvisiThreat. Le flux OAuth GitHub est déclenché lorsqu'un développeur clique sur "Connecter GitHub". Après consentement, GitHub retourne un code qui est échangé contre un token d'accès côté serveur. Ce token est utilisé pour récupérer les métadonnées du dépôt (repo_url, default_branch) et déclencher les scans SAST sur le contenu du dépôt.

Justification. Le flux OAuth évite l'exposition de credentials statiques et permet la révocation du consentement depuis l'interface GitHub sans modifier la configuration d'InvisiThreat.

4.3.2 Chiffrement Fernet (cryptography)

Description. Fernet est un schéma de chiffrement symétrique authentifié (AES-128-CBC + HMAC-SHA256) fourni par la bibliothèque Python cryptography. Il garantit à la fois la confidentialité et l'intégrité du message chiffré.

Utilisation dans InvisiThreat. Le token GitHub OAuth, qui donne accès aux dépôts privés, est une donnée particulièrement sensible. Avant toute persistance dans la colonne `encrypted_token` de `github_repositories`, il est chiffré avec une clé Fernet dérivée d'un secret d'environnement (`FERNET_KEY`). Lors d'un scan, le token est déchiffré en mémoire uniquement pour la durée nécessaire.

Justification. Stocker le token en clair en base de données constituerait une vulnérabilité OWASP A02 (Cryptographic Failures). Fernet apporte le chiffrement authentifié en une seule opération, sans gestion complexe de IV ou de MAC.

4.3.3 SQLAlchemy Alembic (migrations)

Description. Alembic est un outil de migration de schéma de base de données pour SQLAlchemy. Il versionne les modifications du modèle de données et permet leur application ou leur annulation de manière reproductible.

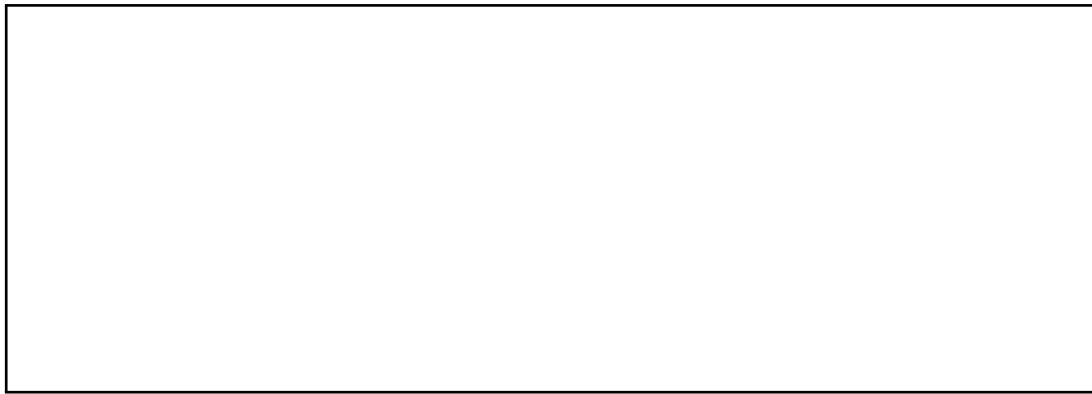
Utilisation dans InvisiThreat. L'ajout des tables `projects`, `github_repositories`, `project_members` et `user_api_keys` est géré via des scripts de migration Alembic versionnés dans le dépôt. Le pipeline CI exécute `alembic upgrade head` à chaque déploiement.

Justification. Alembic garantit la cohérence du schéma entre les environnements de développement, de test et de production, en évitant les migrations manuelles source d'erreurs.

4.4 Analyse et spécification des besoins

4.4.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 2 (figure 4.1) représente les interactions entre les acteurs **Developer** (utilisateur standard) et **Admin** avec les fonctionnalités de gestion de projets et d'intégrations. Le cas *Créer un projet* inclut le cas *S'authentifier* (relation *include*). Le cas *Connecter dépôt GitHub* étend *Créer un projet* et implique une communication avec le système externe **GitHub OAuth**. Le cas *Générer une clé API* est disponible pour `developer` et `admin`, tandis que *Ajouter un membre* est réservé au propriétaire du projet.

**FIGURE 4.1 – Diagramme de cas d'utilisation – Sprint 2.**

4.4.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Créer un projet

TABLE 4.2 – Description du cas d'utilisation – Créer un projet

Titre	Créer un projet
Acteur	Développeur (developer) ou Administrateur (admin)
Résumé	Permet de créer un nouveau projet d'analyse en définissant ses paramètres (nom, description, langage, type d'analyse, visibilité). Le créateur est automatiquement désigné propriétaire.
Préconditions	L'utilisateur est authentifié. Son rôle est developer ou admin.
Postconditions	Un enregistrement Project est créé en base avec owner_id = user.id. Un enregistrement ProjectMember est créé avec le rôle owner. Un AuditLog de type <i>PROJECT_CREATE</i> est enregistré.
Scénario nominal	1. L'utilisateur remplit le formulaire de création (nom, description, langage, type d'analyse : sast/dast/both, visibilité : private/public/internal). 2. Le système valide les données via Pydantic v2 (nom unique par propriétaire). 3. Le système persiste l'entité Project et crée l'entrée ProjectMember (rôle owner) dans la même transaction. 4. Le système retourne le projet créé avec son identifiant UUID.
Scénario alternatif	2.a Nom de projet déjà utilisé par cet utilisateur : le système retourne HTTP 409 avec un message d'erreur explicite. 2.b Données manquantes ou invalides : le système retourne HTTP 422 avec le détail des erreurs de validation Pydantic.

b) Cas d'utilisation : Connecter un dépôt GitHub**TABLE 4.3 – Description du cas d'utilisation – Connecter un dépôt GitHub**

Titre	Connecter un dépôt GitHub
Acteur	Développeur (developer), Administrateur (admin)
Résumé	Permet d'associer un dépôt GitHub à un projet en réalisant le flux OAuth2, puis chiffre et stocke le token d'accès pour les scans ultérieurs.
Préconditions	L'utilisateur est propriétaire ou membre du projet. L'intégration GitHub OAuth est configurée sur la plateforme (client_id, client_secret).
Postconditions	Un enregistrement GitHubRepository est créé avec encrypted_token (Fernet), repo_url et default_branch.
Scénario nominal	1. L'utilisateur clique sur "Connecter GitHub" depuis le panneau de configuration du projet. 2. Le système redirige vers GitHub avec le client_id et un state CSRF. 3. L'utilisateur autorise l'accès sur GitHub. 4. GitHub redirige vers le callback avec le code d'autorisation. 5. Le serveur échange le code contre un token d'accès via l'API GitHub. 6. Le token est chiffré avec Fernet et persisté dans encrypted_token. 7. Les métadonnées du dépôt (URL, branche par défaut) sont récupérées et stockées.
Scénario alternatif	5.a Échange du code échoue (code expiré ou invalide) : le système retourne HTTP 400 et invite à relancer le flux OAuth. 2.a Paramètre state invalide (attaque CSRF détectée) : le système retourne HTTP 403 et journalise l'événement.

4.5 Modélisation conceptuelle

4.5.1 Diagramme de classes

Le diagramme de classes du Sprint 2 (figure 4.2) modélise les entités de gestion des projets. Project est l'agrégat central, lié à User via owner_id (propriétaire) et via ProjectMember (membres du projet). GitHubRepository est lié à Project par une association 0..1 : un projet peut avoir au plus un dépôt GitHub rattaché. UserAPIKey est liée à User par une association 0..N : un utilisateur peut posséder plusieurs clés API actives.

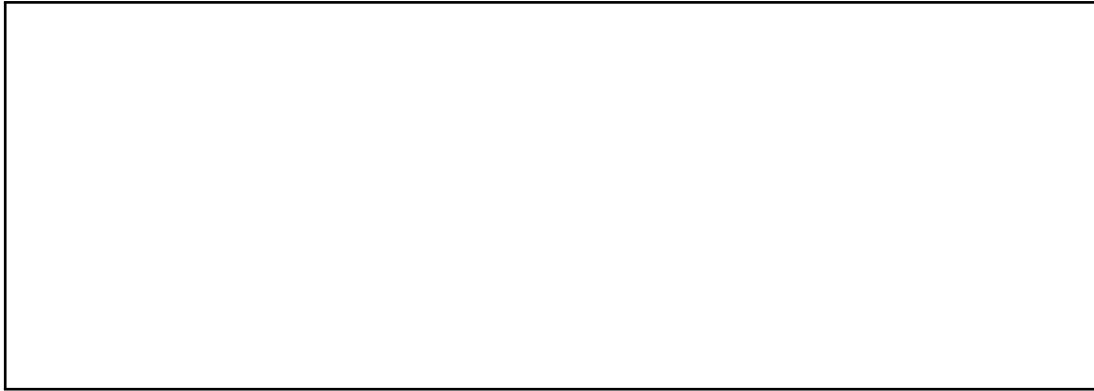


FIGURE 4.2 – Diagramme de classes – Sprint 2 (entités Project, GitHubRepository, ProjectMember, UserAPIKey).

a) Dictionnaire de données

TABLE 4.4 – Dictionnaire de données – Sprint 2

Classe	Code	Désignation	Type
Project	id	Identifiant unique du projet (UUID)	UUID
Project	name	Nom du projet (unique par propriétaire)	String
Project	description	Description textuelle du projet	String (nullable)
Project	owner_id	Référence au propriétaire du projet	UUID (FK → User)
Project	language	Langage principal du projet (enum : python, javascript, java, etc.)	String (enum)
Project	analysis_type	Type d'analyse configuré (enum : sast, dast, both)	String (enum)
Project	visibility	Visibilité du projet (enum : private, public, internal)	String (enum)
ProjectMember	id	Identifiant unique de l'appartenance	UUID
ProjectMember	project_id	Référence au projet	UUID (FK → Project)
ProjectMember	user_id	Référence à l'utilisateur membre	UUID (FK → User)
ProjectMember	role	Rôle dans le projet (owner/member/viewer)	String (enum)
GitHubRepository	id	Identifiant unique du dépôt rattaché	UUID
GitHubRepository	project_id	Référence au projet auquel le dépôt est rattaché	UUID (FK → Project)
GitHubRepository	encrypted_token	Token OAuth GitHub chiffré avec Fernet	String
GitHubRepository	repo_url	URL HTTPS du dépôt GitHub	String
GitHubRepository	default_branch	Branche par défaut utilisée pour les scans	String
UserAPIKey	id	Identifiant unique de la clé API	UUID
UserAPIKey	user_id	Référence à l'utilisateur propriétaire	UUID (FK → User)
UserAPIKey	key_hash	Hash SHA-256 de la clé API (jamais la clé en clair)	String
UserAPIKey	name	Nom descriptif de la clé (ex. : « Pipeline GitLab CI »)	String
UserAPIKey	is_active	Indicateur d'activation de la clé	Boolean
UserAPIKey	last_used_at	Horodatage de la dernière utilisation	DateTime (nullable)

4.5.2 Diagrammes de séquence

a) Diagramme DS-2a : Créer un projet

Le diagramme DS-2a (figure 4.3) décrit le flux de création de projet selon la notation BCE. **ProjectForm** (boundary — formulaire React) transmet les données à **ProjectController** (control — endpoint FastAPI `POST /projects`). La dépendance d'autorisation vérifie le rôle de l'appelant. Le contrôleur persiste les entités **Project** et **ProjectMember** (entity) dans une transaction atomique PostgreSQL.

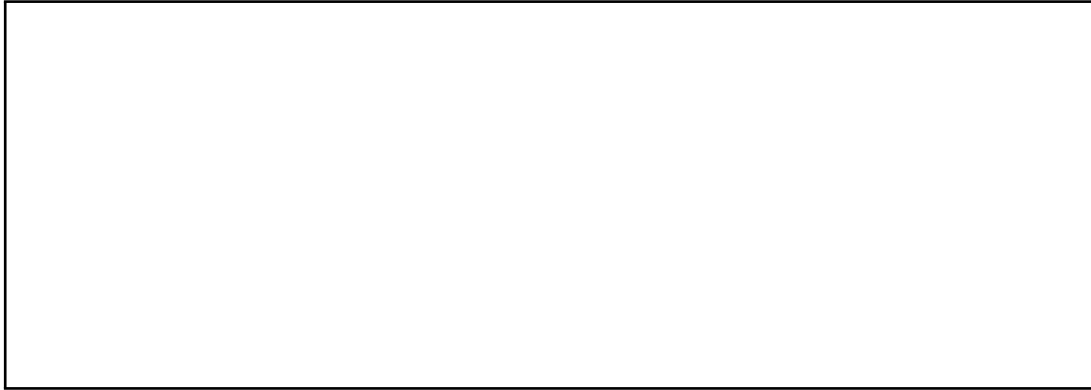


FIGURE 4.3 – Diagramme de séquence DS-2a – Créer un projet.

b) Diagramme DS-2b : Connecter un dépôt GitHub (avec chiffrement Fernet)

Le diagramme DS-2b (figure 4.4) modélise le flux OAuth GitHub. **GitHubForm** (boundary) initie la redirection OAuth. Après retour du code d'autorisation, **ProjectController** (control) délègue l'échange du code à **EncryptionService** (control — module Fernet), qui chiffre le token avant que **GitHubRepository** (entity) ne soit persisté. Ce diagramme met en évidence la séparation de responsabilités entre le contrôle OAuth, le chiffrement et la persistance.



FIGURE 4.4 – Diagramme de séquence DS-2b – Connecter un dépôt GitHub (chiffrement Fernet du token).

4.6 Réalisation

4.6.1 Gestion des projets et des dépôts

Le module projet centralise les métadonnées (nom, dépôt, type d'analyse, visibilité) et constitue le point d'entrée pour tous les scans. La table `projects` est versionnée par Alembic. Les endpoints exposent une API RESTful complète (GET, POST, PUT, DELETE) avec pagination et filtres. Les migrations Alembic garantissent la cohérence du schéma entre environnements.

4.6.2 Intégrations OAuth et clés API

L'intégration GitHub OAuth est sécurisée par un paramètre `state` aléatoire anti-CSRF, vérifié à la réception du callback. Le token obtenu est immédiatement chiffré avec Fernet et jamais exposé en dehors du processus de scan. Les clés API (`UserAPIKey`) sont générées avec `secrets.token_urlsafe(32)`, transmises une seule fois à l'utilisateur lors de la création, puis stockées uniquement sous forme de hash SHA-256.

4.7 Test et validation

4.7.1 Tests réalisés

TABLE 4.5 – Tableau de test – Créer un projet (Sprint 2)

Cas de Test	TC-S2-01 – Création de projet avec paramètres valides
Auteur du cas de test	Développeur / Testeur QA
Description	Vérifier qu'un développeur authentifié peut créer un projet et que les entités <code>Project</code> et <code>ProjectMember</code> (rôle <code>owner</code>) sont correctement persistées.
Précondition	Un utilisateur avec le rôle <code>developer</code> est authentifié (access token valide).
Scénario	- Envoyer <code>POST /projects</code> avec <code>Authorization: Bearer <token></code> et un corps JSON contenant <code>name</code>, <code>language = "python"</code>, <code>analysis_type = "sast"</code>, <code>visibility = "private"</code>. - Vérifier la réponse HTTP 201 contenant l'UUID du projet créé. - Vérifier en base que <code>projects.owner_id</code> correspond à l'UUID de l'utilisateur. - Vérifier qu'un enregistrement <code>ProjectMember</code> avec <code>role = "owner"</code> existe pour ce projet et cet utilisateur.
Postcondition	Le projet est créé. L'utilisateur est désigné propriétaire (<code>owner</code>) dans <code>project_members</code> .

4.7.2 Validation

La validation s'est appuyée sur des scénarios simulant des pipelines CI/CD déclenchant des scans via clé API, ainsi que sur des tests de sécurité vérifiant que le token GitHub ne peut pas être récupéré en clair depuis l'API ni depuis la base de données.

4.7.3 Gestion des versions et commits

Le Sprint 2 a été développé sur la branche `feature/sprint2-projects-integrations`. Les migrations Alembic ont été versionnées en parallèle du code métier. Les tests d'intégration du flux OAuth ont été moqués via `httpx.MockTransport` pour éviter les appels réels à GitHub dans le pipeline CI.

4.8 Conclusion

Le Sprint 2 a permis d'ouvrir la plateforme aux intégrations essentielles. La gestion des projets, le chiffrement des tokens GitHub (Fernet), la génération sécurisée de clés API et la gestion des membres par projet préparent le terrain pour les sprints d'analyse SAST et DAST. Le respect des bonnes pratiques de sécurité (aucun secret en clair, validation CSRF, hash des clés API) garantit la conformité aux recommandations OWASP dès cette phase.

Chapitre

5

Sprint 3 – SAST et DAST

5.1 Introduction

Ce sprint constitue le cœur du moteur d'analyse d'InvisiThreat : il intègre en un même pipeline les deux familles d'analyses de sécurité applicative. La première partie porte sur l'analyse statique (SAST) via Semgrep et Bandit, exécutée de façon asynchrone par des workers Celery avec suivi temps réel Socket.IO. La seconde partie introduit l'analyse dynamique (DAST) via OWASP ZAP, pilotée par l'API REST du daemon ZAP, et calcule un scoring de risque agrégé. La complexité de ce sprint justifie deux diagrammes de séquence distincts (DS-3a et DS-3b).

5.2 Objectifs du sprint et planification des tâches

5.2.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit être capable de lancer des scans SAST et DAST, d'exécuter les moteurs correspondants de façon asynchrone, de normaliser les résultats et de calculer un score de risque. Plus précisément :

- intégrer Semgrep (analyse polyglotte par règles) et Bandit (inspection Python) via workers Celery/Redis ;
- assurer le suivi temps réel de la progression via Socket.IO (room scopée par `scan_id`) ;
- intégrer OWASP ZAP en mode daemon, piloté par API REST, pour l'analyse dynamique ;
- normaliser les alertes SAST et DAST vers le modèle Vulnerability unifié ;
- calculer le score de risque agrégé (RiskScore) après chaque scan.

5.2.2 Sprint Backlog

TABLE 5.1 – Sprint Backlog – Sprint 3

ID	User Story	ID Tâche	Tâche	Priorité
S3-01	En tant que développeur, je peux lancer un scan SAST via Semgrep et Bandit sur un dépôt GitHub.	T3-01	Implémenter POST /scans avec method="github" et enqueue Celery (worker SAST).	M
		T3-02	Développer les workers Celery Semgrep (semgrep -json) et Bandit (bandit -f json).	M
S3-02	En tant que développeur ou Security Manager, je peux suivre la progression du scan en temps réel.	T3-03	Émettre des événements Socket.IO (scan.status_changed, scan.vuln_found) depuis le worker Celery, scopés par scan_id.	M
S3-03	En tant que développeur, je peux lancer un scan DAST sur une URL cible via OWASP ZAP.	T3-04	Implémenter POST /scans avec method="dast" et enqueue Celery (worker DAST).	M
		T3-05	Développer le worker Celery DAST : contexte ZAP, spider, scan actif, collecte des alertes.	M
S3-04	En tant que Security Manager, je peux consulter le rapport normalisé filtré par sévérité et le score de risque du scan.	T3-06	Normaliser les sorties SAST (SARIF/JSON Bandit) et DAST (alertes ZAP) vers Vulnerability avec mapping VulnerabilitySeverity.	M
		T3-07	Calculer et persister le RiskScore (agrégation CVSS pondérée) après chaque scan.	M

5.3 Technologies et concepts utilisés

5.3.1 Semgrep (analyse statique polyglotte)

Description. Semgrep est un moteur d'analyse statique basé sur la correspondance de patterns AST (Abstract Syntax Tree). Il supporte plus de 30 langages et produit des sorties au format SARIF ou JSON exploitables programmatiquement.

Utilisation dans InvisiThreat. Exécuté en sous-processus depuis le worker Celery SAST (semgrep -config auto -json), ses résultats sont normalisés par l'adaptateur SAST. Des règles personnalisées sont configurables via le paramètre custom_rules de la requête POST /scans.

Justification. Semgrep couvre les projets polyglots (Python, JavaScript, Java, TypeScript) du portefeuille Mobelite. La combinaison avec Bandit maximise la couverture sur les projets Python.

5.3.2 Bandit (inspection de sécurité Python)

Description. Bandit inspecte l'AST Python pour détecter des patterns de code dangereux (injections, algorithmes cryptographiques faibles, secrets codés en dur). Il classe ses résultats par sévérité et confiance.

Utilisation dans InvisiThreat. Exécuté via `bandit -r . -f json -q` sur les projets Python. Ses sorties sont fusionnées avec celles de Semgrep, avec déduplication par empreinte (fingerprint).

5.3.3 Celery et Redis (orchestration asynchrone)

Description. Celery est un système de file de tâches distribuées pour Python. Redis, utilisé comme broker de messages, gère les queues des workers SAST et DAST.

Utilisation dans InvisiThreat. À la réception d'un `POST /scans`, l'API crée immédiatement l'entité Scan avec `status="pending"`, puis enqueue la tâche dans Redis. Le worker approprié (SAST ou DAST) consomme la tâche, exécute le moteur, persiste les Vulnerability et met à jour le statut du scan.

Justification. Les scans peuvent durer plusieurs minutes. L'approche asynchrone garantit la réactivité de l'API et permet d'exécuter plusieurs scans en parallèle.

5.3.4 Socket.IO — python-socketio (temps réel)

Description. python-socketio implémente le protocole Socket.IO côté serveur Python, offrant une communication bidirectionnelle temps réel via WebSocket (avec fallback long-polling).

Utilisation dans InvisiThreat. Les workers Celery publient des événements (`scan.status_changed`, `scan.vuln_found`) via un canal Redis. Le gateway Socket.IO (intégré à FastAPI) les diffuse aux clients. Les rooms sont scoppées par `scan_id` pour éviter les émissions croisées entre projets — contrainte principale de l'architecture Socket.IO multi-tenant.

5.3.5 OWASP ZAP (analyse dynamique, mode daemon)

Description. OWASP Zed Attack Proxy, en mode daemon (`zap.sh -daemon`), expose une API REST permettant de piloter les phases de scan : création de contexte, spider (exploration), scan actif (injection de payloads), collecte des alertes.

Utilisation dans InvisiThreat. Le worker Celery DAST crée un contexte ZAP dédié par scan, déclenche le spider sur l'URL cible, lance le scan actif et collecte les alertes. Celles-ci sont normalisées vers le modèle Vulnerability avec mapping des sévérités ZAP (High/Medium/Low/Informational → critical/high/medium/info).

Justification. ZAP est la référence open source pour le DAST web. Son API REST permet une intégration programmatique complète depuis Celery, sans interface graphique.

5.4 Analyse et spécification des besoins

5.4.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 3 (figure 5.5) représente les interactions entre le **Developer**, le **Security Manager** et les systèmes externes **Semgrep**, **Bandit** et **OWASP ZAP**. Le cas *Lancer un scan SAST* et le cas *Lancer un scan DAST* partagent la même infrastructure Celery/Redis via une relation *include* vers *Exécuter le worker asynchrone*. Le cas *Suivre en temps réel* étend les deux cas de scan via Socket.IO. Le cas *Consulter le rapport normalisé* est accessible après `status="completed"` et inclut la consultation du RiskScore.



FIGURE 5.1 – Diagramme de cas d'utilisation – Sprint 3 (SAST et DAST).

5.4.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Lancer un scan SAST

TABLE 5.2 – Description du cas d'utilisation – Lancer un scan SAST

Titre	Lancer un scan SAST
Acteur	Développeur (developer), Administrateur (admin)
Résumé	Soumettre un dépôt de code source pour analyse statique (Semgrep + Bandit) via un job Celery asynchrone, avec suivi temps réel Socket.IO.
Préconditions	L'utilisateur est authentifié, membre du projet. Le projet est configuré avec <code>analysis_type = "sast" ou "both"</code> . Le dépôt est accessible.
Postconditions	Scan créé avec <code>method="github" ou "cli", status="completed"</code> . Entités Vulnerability persistées. Événements Socket.IO émis.
Scénario nominal	1. L'utilisateur soumet <code>POST /scans</code> avec <code>project_id</code> et <code>method="github"</code>. 2. Le système crée le Scan (<code>status="pending"</code>) et publie la tâche dans Redis. 3. Le worker Celery SAST clone le dépôt et passe <code>status="running"</code>. 4. Semgrep puis Bandit sont exécutés; les résultats sont normalisés et persistés. 5. Le scan passe à <code>status="completed"</code>; Socket.IO émet <code>scan.status_changed</code>.
Scénario alternatif	3.a Dépôt inaccessible : <code>status="failed"</code>, erreur dans <code>job_state</code>. 4.a Semgrep timeout : Bandit est poursuivi seul; l'erreur partielle est journalisée.

b) Cas d'utilisation : Lancer un scan DAST**TABLE 5.3 – Description du cas d'utilisation – Lancer un scan DAST**

Titre	Lancer un scan DAST (OWASP ZAP)
Acteur	Développeur (developer), Administrateur (admin)
Résumé	Soumettre une URL cible pour analyse dynamique via OWASP ZAP (daemon), orchestrée par Celery. Le RiskScore est calculé en fin de scan.
Préconditions	L'utilisateur est authentifié, membre du projet. <code>analysis_type = "dast"</code> ou <code>"both"</code> . L'URL cible est accessible depuis le worker. Le daemon ZAP est démarré.
Postconditions	Scan à <code>status="completed"</code> , <code>method="dast"</code> . Vulnerability DAST persistées. RiskScore calculé et persisté.
Scénario nominal	1. L'utilisateur soumet <code>POST /scans</code> avec <code>method="dast"</code> et l'URL cible. 2. Le système crée le Scan (<code>status="pending"</code>) et publie la tâche dans Redis. 3. Le worker Celery DAST crée un contexte ZAP, déclenche le spider, puis le scan actif. 4. Les alertes ZAP sont collectées et normalisées vers Vulnerability. 5. Le RiskScore est calculé (agrégation CVSS pondérée par sévérité). 6. Le scan passe à <code>status="completed"</code>; Socket.IO émet <code>scan.status_changed</code>.
Scénario alternatif	3.a URL cible inaccessible : <code>status="failed"</code>, message dans <code>job_state</code>. 3.b Timeout ZAP : alertes déjà collectées sauvegardées ; <code>status="failed"</code> partiel.

5.5 Modélisation conceptuelle

5.5.1 Diagramme de classes

Le diagramme de classes du Sprint 3 (figure 5.6) modélise les entités du pipeline d'analyse. Scan est l'agrégat central, lié à Project (*N..1*). Vulnerability est lié à Scan (*1..N*) et porte les résultats normalisés, qu'ils proviennent du SAST ou du DAST. RiskScore est lié à Scan (*1..1*) et encapsule le score agrégé calculé en fin d'exécution. Les enums ScanMethod (`cli/github/dast/exe`), ScanStatus (`pending/running/completed/failed`) et VulnerabilitySeverity (`critical/high/medium/low/info`) contraignent les valeurs admissibles.

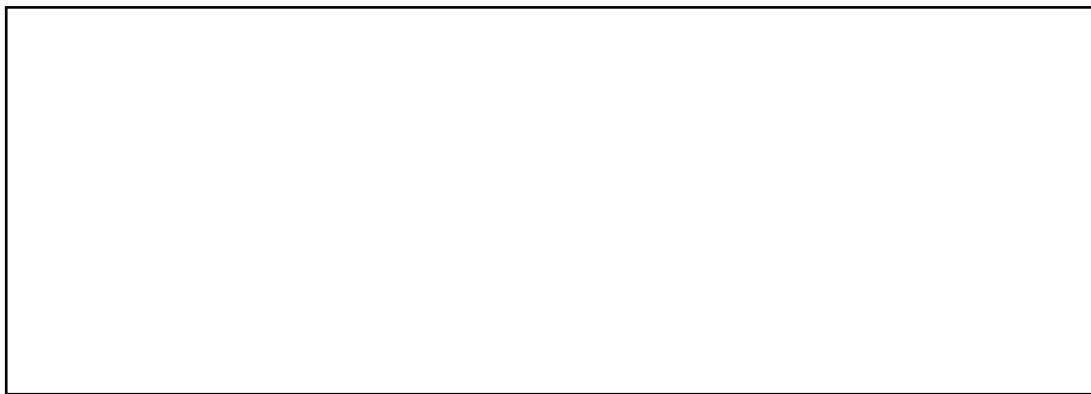


FIGURE 5.2 – Diagramme de classes – Sprint 3 (Scan, Vulnerability, RiskScore avec enums).

a) Dictionnaire de données

TABLE 5.4 – Dictionnaire de données – Sprint 3

Classe	Code	Désignation	Type
Scan	id	Identifiant unique du scan	UUID
Scan	project_id	Référence au projet analysé	UUID (FK → Project)
Scan	method	Méthode de déclenchement (cli/github/dast/exe)	String (enum)
Scan	status	État du scan (pending/running/completed/failed)	String (enum)
Scan	job_state	Métadonnées de progression Celery (JSON)	JSON (nullable)
Scan	created_at	Horodatage de création	DateTime
Vulnerability	id	Identifiant unique de la vulnérabilité	UUID
Vulnerability	scan_id	Référence au scan parent	UUID (FK → Scan)
Vulnerability	title	Titre de la vulnérabilité	String
Vulnerability	description	Description détaillée	Text
Vulnerability	severity	Sévérité normalisée (critical/high/medium/low/info)	String (enum)
Vulnerability	status	État de traitement (open/in_progress/resolved/false_positive/risk_accepted)	String (enum)
Vulnerability	cvss_score	Score CVSS (0.0–10.0)	Float (nullable)
RiskScore	id	Identifiant unique du score de risque	UUID
RiskScore	scan_id	Référence au scan évalué	UUID (FK → Scan)
RiskScore	score	Score agrégé pondéré par CVSS (0.0–10.0)	Float
RiskScore	level	Niveau qualitatif (critical/high/medium/low)	String (enum)

5.5.2 Diagrammes de séquence

a) Diagramme DS-3a : Lancement SAST — Celery worker et Socket.IO

Le diagramme DS-3a (figure 5.7) modélise le flux d'un scan SAST selon la notation BCE. **ScanForm** (boundary — interface React ou CLI) soumet la requête à **ScanController** (control — POST /scans). Après création de l'entité **Scan** (entity), la tâche est publiée dans **Redis Queue**. Le **CeleryWorker SAST** (control) exécute Semgrep et Bandit, persiste les **Vulnerability** (entity) et émet les événements `scan.status_changed` et `scan.vuln_found` vers le **SocketIOGateway** (control), qui les diffuse aux clients abonnés à la room `scan_id`.

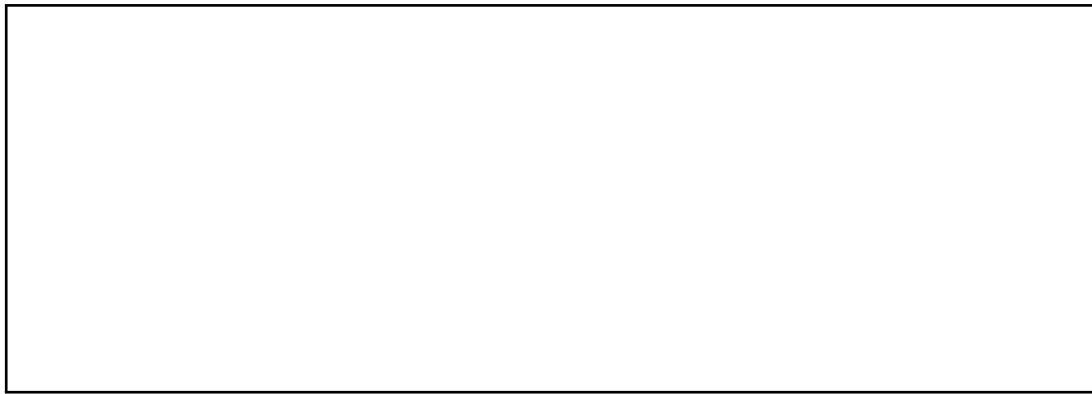


FIGURE 5.3 – Diagramme de séquence DS-3a – Lancement SAST (Celery worker · Semgrep · Bandit · Socket.IO).

b) Diagramme DS-3b : Lancement DAST — OWASP ZAP et score de risque

Le diagramme DS-3b (figure 5.4) décrit le flux DAST selon la notation BCE. **DASTScanForm** (boundary) soumet la requête à **ScanController** (control). La tâche est enqueued dans Redis; le **CeleryWorker DAST** (control) interagit avec le système externe **OWASP ZAP** (API REST) : création du contexte, spider, scan actif, collecte des alertes. Les alertes sont normalisées et persistées comme **Vulnerability** (entity). Le **RiskScoreCalculator** (control) agrège les scores CVSS et persiste l'entité **RiskScore** (entity). Des événements Socket.IO sont émis à chaque étape.

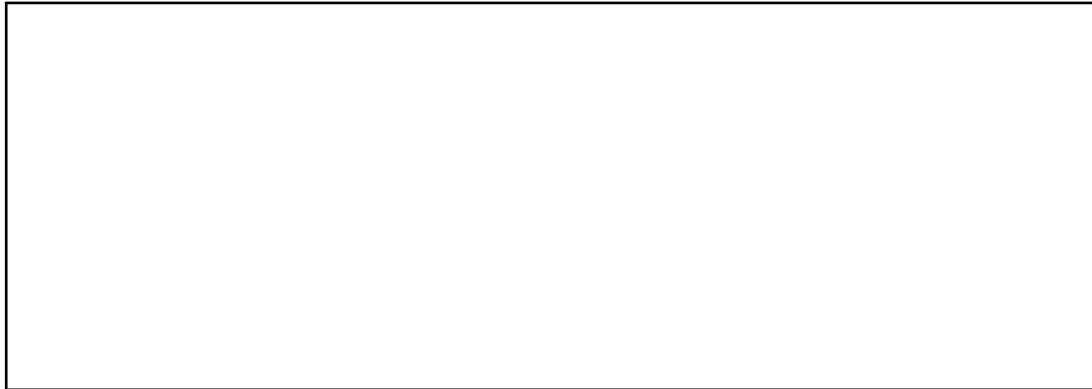


FIGURE 5.4 – Diagramme de séquence DS-3b – Lancement DAST (OWASP ZAP · normalisation · score de risque).

5.6 Réalisation

5.6.1 Pipeline SAST (Semgrep + Bandit)

Les workers Celery SAST exécutent Semgrep (`semgrep -config auto -json`) et Bandit (`bandit -r . -f json -q`) en sous-processus isolés. Les adaptateurs de normalisation (un par moteur) mappent les sévérités natives vers l'enum `VulnerabilitySeverity` et calculent une empreinte (fingerprint) unique par vulnérabilité pour la déduplication lors des scans répétés.

5.6.2 Pipeline DAST (OWASP ZAP)

Le worker Celery DAST démarre une session ZAP via API REST, configure le contexte avec l'URL cible, déclenche le spider puis le scan actif. Les alertes collectées sont normalisées par l'adaptateur DAST (même interface que l'adaptateur SAST). Le RiskScore est calculé en agrégeant les scores CVSS pondérés par sévérité et persisté en base à la fin du scan.

5.6.3 Suivi temps réel (Socket.IO)

Les rooms Socket.IO sont scopées par scan_id : seuls les clients abonnés à la room d'un scan reçoivent ses événements. Les événements scan.status_changed (transitions pending→running→completed) et scan.vuln_found (compteur en temps réel) permettent un suivi granulaire sans polling de l'API REST.

5.7 Test et validation

5.7.1 Tests réalisés

TABLE 5.5 – Tableau de test – Scan SAST complet (Sprint 3)

Cas de Test	TC-S3-01 – Scan SAST sur dépôt Python avec vulnérabilités connues
Auteur du cas de test	Développeur / Testeur QA
Description	Vérifier qu'un scan SAST détecte les vulnérabilités connues et les persiste avec les sévérités correctes.
Précondition	Un projet Python est configuré. Un dépôt de test contenant des vulnérabilités intentionnelles (injection SQL, secret codé en dur) est rattaché. Un worker Celery est opérationnel.
Scénario	1. Envoyer POST /scans (method="github"). 2. Observer les événements Socket.IO : pending → running → completed. 3. Appeler GET /scans/{id}/vulnerabilities?severity=critical,high. 4. Vérifier la présence des vulnérabilités attendues avec sévérités correctes.
Postcondition	Les vulnérabilités connues sont détectées. Le scan passe à completed.

TABLE 5.6 – Tableau de test – Scan DAST et score de risque (Sprint 3)

Cas de Test	TC-S3-02 – Scan DAST sur DVWA avec calcul de RiskScore
Auteur du cas de test	Security Manager / Testeur QA
Description	Vérifier qu'un scan DAST sur DVWA détecte les failles OWASP Top 10 et produit un RiskScore cohérent.
Précondition	Une instance DVWA est accessible à l'URL cible. Worker Celery DAST et daemon ZAP opérationnels.
Scénario	1. Envoyer POST /scans (method="dast", URL DVWA). 2. Observer les événements Socket.IO jusqu'à completed. 3. Appeler GET /scans/{id}/vulnerabilities et vérifier XSS et SQL injection (high/critical). 4. Appeler GET /scans/{id}/risk-score et vérifier score > 7.0 et level = "high".
Postcondition	Vulnérabilités OWASP détectées. RiskScore calculé et persisté.

5.7.2 Validation

La validation bout en bout a porté sur des projets de test Python (SAST) et une instance DVWA (DAST). La cohérence du mapping de sévérités entre les deux moteurs et le modèle interne a été vérifiée par comparaison avec les sorties brutes attendues.

5.7.3 Gestion des versions et commits

Ce sprint a été développé sur la branche `feature/sprint3-sast-dast`. Les workers SAST et DAST ont fait l'objet de tests unitaires avec mocking des sous-processus (Semgrep, Bandit) et de l'API ZAP. Les migrations Alembic ajoutant les tables `scans`, `vulnerabilities` et `risk_scores` ont été versionnées et testées.

5.8 Conclusion

Le Sprint 3 a intégré les deux piliers analytiques d'InvisiThreat : le SAST via Semgrep et Bandit, et le DAST via OWASP ZAP, tous deux orchestrés par Celery/Redis avec suivi temps réel Socket.IO. La normalisation unifiée des résultats et le calcul du RiskScore fournissent une vision priorisée du risque applicatif. Cette base technique solide ouvre la voie au Sprint 4, centré sur le workflow collaboratif de gestion des vulnérabilités.

5.9 Introduction

Ce sprint est dédié à l'intégration des analyses statiques (SAST) dans le pipeline InvisiThreat. Il vise à automatiser la détection précoce des vulnérabilités dans le code source via les moteurs Semgrep et Bandit, à normaliser les résultats vers un modèle interne commun et à exposer les alertes dans une interface filtrable par sévérité. L'infrastructure d'orchestration

asynchrone (Celery/Redis) est introduite dans ce sprint pour prendre en charge les scans longs sans bloquer l'API, et sera étendue au DAST dans le Sprint 4.

5.10 Objectifs du sprint et planification des tâches

5.10.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit permettre de lancer un scan SAST sur un dépôt GitHub ou via le CLI, d'exécuter les moteurs Semgrep et Bandit de façon asynchrone, de normaliser les résultats vers le modèle Vulnerability et de présenter le rapport filtré par sévérité. Plus précisément :

- intégrer les moteurs Semgrep (analyse polyglotte par règles) et Bandit (inspection Python) ;
- mettre en place l'infrastructure d'orchestration asynchrone Celery/Redis ;
- normaliser les sorties hétérogènes (SARIF, JSON Bandit) vers le modèle Vulnerability ;
- exposer le rapport SAST normalisé filtrable par sévérité (critical/high/medium/low/info).

5.10.2 Sprint Backlog

TABLE 5.7 – Sprint Backlog – Sprint 3

ID	User Story	ID Tâche	Tâche	Priorité
S3-01	En tant que développeur, je peux lancer un scan SAST sur un projet via Semgrep et Bandit.	T3-01	Implémenter l'endpoint POST /scans avec method = "cli" ou "github" et création d'un job Celery.	M
		T3-02	Développer les workers Celery pour Semgrep (semgrep -json) et Bandit (bandit -f json).	M
S3-02	En tant que développeur, je peux consulter le rapport SAST normalisé filtré par sévérité.	T3-03	Normaliser les sorties Semgrep (SARIF) et Bandit (JSON) vers le modèle Vulnerability avec mapping des sévérités.	M
		T3-04	Exposer GET /scans/{id}/vulnerabilities avec paramètre de filtre severity.	M
S3-03	En tant que Security Manager, je peux consulter les alertes normalisées d'un scan et les trier par criticité.	T3-05	Implémenter la vue tableau de bord SAST avec agrégation par sévérité (VulnerabilitySeverity enum).	M
S3-04	En tant que développeur, je peux suivre la progression du scan en temps réel.	T3-06	Émettre des événements Socket.IO (namespaces scopés par scan_id) depuis le worker Celery.	S

5.11 Technologies et concepts utilisés

5.11.1 Semgrep (analyse statique polyglotte)

Description. Semgrep est un outil d'analyse statique open source basé sur un moteur de correspondance de patterns AST (Abstract Syntax Tree). Il supporte plus de 30 langages et s'appuie sur des règles YAML définissant les patterns de code vulnérable à détecter. Il peut produire des sorties au format SARIF (Static Analysis Results Interchange Format), standard de l'industrie.

Utilisation dans InvisiThreat. Semgrep est exécuté en sous-processus depuis le worker Celery avec la commande `semgrep -config auto -json`. Les résultats JSON sont parsés et normalisés vers le modèle Vulnerability. Des règles personnalisées configurables via le paramètre `custom_rules` de la requête POST /scans permettent d'affiner la détection selon le contexte du projet.

Justification. Semgrep couvre les projets Python, JavaScript, Java et d'autres langages présents dans le portefeuille Mobelite, contrairement à Bandit qui se limite à Python. La combinaison des deux moteurs maximise la couverture des vulnérabilités.

5.11.2 Bandit (inspection de sécurité Python)

Description. Bandit est un outil d'analyse statique spécialisé dans la détection de problèmes de sécurité dans les projets Python. Il inspecte l'AST Python à la recherche de patterns connus (utilisation d'algorithmes faibles, injections, secrets codés en dur, etc.) et classe les résultats par sévérité et confiance.

Utilisation dans InvisiThreat. Bandit est exécuté via `bandit -r . -f json -q` sur le code Python du projet. Ses résultats JSON sont normalisés et fusionnés avec ceux de Semgrep pour les projets Python, éliminant les doublons par empreinte (fingerprint).

Justification. Bandit complète Semgrep sur les projets Python avec une profondeur d'analyse spécifique au langage et des règles couvrant des vulnérabilités Python-spécifiques (ex. : subprocess shell injection, pickle désérialisation).

5.11.3 Celery et Redis (orchestration asynchrone)

Description. Celery est un système de file de tâches distribuées pour Python. Redis, utilisé ici comme broker de messages, gère la file d'attente des jobs Celery. Cette combinaison permet de déporter les traitements longs (scans de code) vers des workers indépendants sans bloquer l'API principale.

Utilisation dans InvisiThreat. Lors de la création d'un scan (`POST /scans`), l'API crée immédiatement l'entité Scan avec `status = "pending"`, puis enqueue une tâche Celery. Le worker exécute Semgrep et Bandit, normalise les résultats, persiste les Vulnerability en base et met à jour le statut du scan (`running` → `completed` ou `failed`). Des événements Socket.IO sont émis à chaque étape pour le suivi temps réel.

Justification. Les scans SAST sur de grands dépôts peuvent durer plusieurs minutes. L'approche asynchrone garantit que l'API reste réactive et que plusieurs scans peuvent être exécutés en parallèle sans dégradation de performance.

5.12 Analyse et spécification des besoins

5.12.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 3 (figure 5.5) représente les interactions entre le **Developer** (déclenchement du scan, consultation des alertes) et le **Security Manager** (consultation des alertes normalisées, filtrage par criticité). Un acteur **CLI Scanner** représente les appels automatisés depuis un pipeline CI/CD via clé API. La communication avec les systèmes externes **Semgrep** et **Bandit** (exécutés en sous-processus par le worker Celery) est matérialisée par des relations *include* depuis le cas *Lancer un scan SAST*.

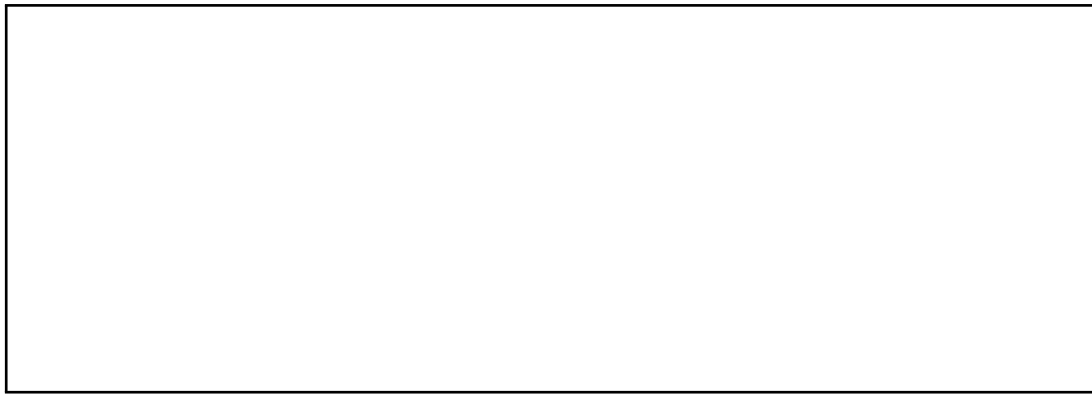


FIGURE 5.5 – Diagramme de cas d'utilisation – Sprint 3.

5.12.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Lancer un scan SAST

TABLE 5.8 – Description du cas d'utilisation – Lancer un scan SAST

Titre	Lancer un scan SAST
Acteur	Développeur (developer), Administrateur (admin)
Résumé	Permet de soumettre un dépôt de code source pour analyse statique par Semgrep et Bandit, via l'interface web ou un appel CLI avec clé API.
Préconditions	L'utilisateur est authentifié et membre du projet. Le projet est configuré avec <code>analysis_type = "sast" ou "both"</code> . Le dépôt est accessible (GitHub OAuth ou upload direct).
Postconditions	Un enregistrement Scan est créé avec <code>status = "pending"</code> , <code>method = "github" ou "cli"</code> . Une tâche Celery est enqueued sur Redis. Des Vulnerability sont persistées après exécution.
Scénario nominal	1. L'utilisateur soumet <code>POST /scans</code> avec <code>project_id</code> et <code>method = "github"</code>. 2. Le système crée l'entité Scan avec <code>status = "pending"</code>. 3. Le système publie la tâche dans la file Redis. 4. Le worker Celery récupère la tâche, clone le dépôt et met à jour <code>status = "running"</code>. 5. Semgrep et Bandit sont exécutés séquentiellement sur le code. 6. Les résultats sont normalisés et persistés comme entités Vulnerability. 7. Le scan passe à <code>status = "completed"</code>. 8. Des événements Socket.IO sont émis pour chaque changement de statut (scopés par <code>scan_id</code>).
Scénario alternatif	4.a Le dépôt est inaccessible (token révoqué ou dépôt supprimé) : le worker met le scan à <code>status = "failed"</code> et enregistre le message d'erreur dans <code>job_state</code>. 5.a Semgrep échoue (timeout, dépôt trop grand) : le worker poursuit avec Bandit seul et journalise l'erreur partielle.

b) Cas d'utilisation : Consulter le rapport SAST normalisé

TABLE 5.9 – Description du cas d'utilisation – Consulter le rapport SAST normalisé

Titre	Consulter le rapport SAST normalisé filtré par sévérité
Acteur	Développeur (developer), Security Manager (security_manager)
Résumé	Permet de consulter les vulnérabilités détectées lors d'un scan SAST, filtrées par niveau de sévérité (critical/high/medium/low/info).
Préconditions	Le scan est dans l'état completed. L'utilisateur est membre du projet.
Postconditions	L'utilisateur obtient la liste paginée des vulnérabilités correspondant au filtre.
Scénario nominal	1. L'utilisateur accède à la page de résultats du scan. 2. Il sélectionne le filtre de sévérité (ex. : severity=critical,high). 3. Le système retourne GET /scans/{id}/vulnerabilities?severity=critical,high. 4. Les vulnérabilités sont affichées avec titre, description, fichier, ligne, score CVSS et CWE.
Scénario alternatif	3.a Aucune vulnérabilité ne correspond au filtre : le système retourne une liste vide avec HTTP 200 et un message informatif.

5.13 Modélisation conceptuelle

5.13.1 Diagramme de classes

Le diagramme de classes du Sprint 3 (figure 5.6) modélise les entités du pipeline SAST. Scan est lié à Project (*N..1*) et constitue le conteneur des résultats d'analyse. Vulnerability est liée à Scan (*1..N*) et représente une faille individuelle détectée. Les enums ScanMethod (cli/github/dast/exe), ScanStatus (pending/running/completed/failed) et VulnerabilitySeverity (critical/high/medium/low/info) sont définis au niveau du modèle de données et contraignent les valeurs admissibles.



FIGURE 5.6 – Diagramme de classes – Sprint 3 (entités Scan, Vulnerability avec enums).

a) Dictionnaire de données

TABLE 5.10 – Dictionnaire de données – Sprint 3

Classe	Code	Désignation	Type
Scan	id	Identifiant unique du scan	UUID
Scan	project_id	Référence au projet analysé	UUID (FK → Project)
Scan	method	Méthode de déclenchement (cli/github/dast/exe)	String (enum)
Scan	status	État du scan (pending/running/completed/failed)	String (enum)
Scan	job_state	Métadonnées de progression du job Celery (JSON)	JSON (nullable)
Scan	created_at	Horodatage de création du scan	DateTime
Vulnerability	id	Identifiant unique de la vulnérabilité	UUID
Vulnerability	scan_id	Référence au scan parent	UUID (FK → Scan)
Vulnerability	title	Titre de la vulnérabilité (issue du moteur de scan)	String
Vulnerability	description	Description textuelle de la vulnérabilité	Text
Vulnerability	severity	Niveau de sévérité (critical/high/medium/low/info)	String (enum)
Vulnerability	status	État de traitement (open/in_progress/resolved/false_positive/risk_accepted)	String (enum)
Vulnerability	cvss_score	Score CVSS de la vulnérabilité (0.0–10.0)	Float (nullable)

5.13.2 Diagrammes de séquence

a) Diagramme DS-3a : Lancer un scan SAST (orchestration Celery/Redis)

Le diagramme DS-3a (figure 5.7) modélise le flux de déclenchement et d'exécution d'un scan SAST selon la notation BCE. **ScanForm** (boundary — interface React ou CLI) soumet la requête à **ScanController** (control — endpoint FastAPI POST /scans). Le contrôleur crée l'entité **Scan** (entity) avec `status = "pending"`, puis enqueue la tâche dans la file **Redis Queue**. Le **CeleryWorker** (control) consomme la tâche, exécute Semgrep et Bandit, persiste les **Vulnerability** (entity) et émet des événements Socket.IO.

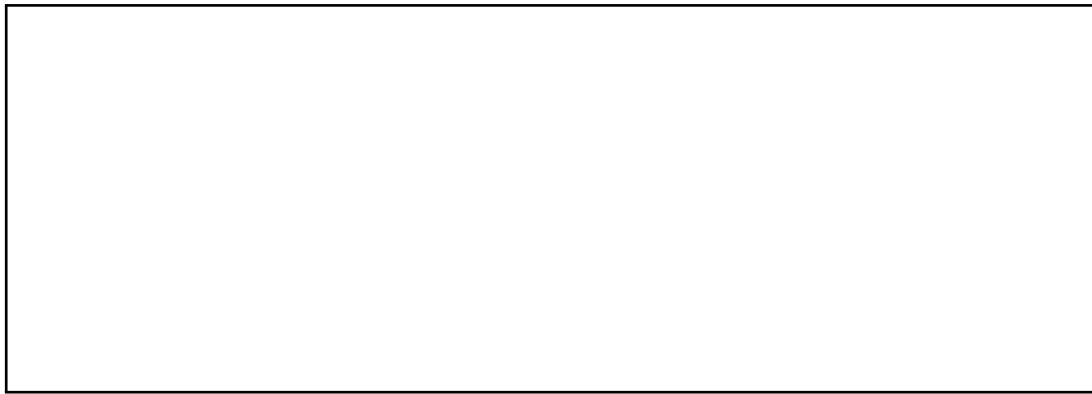


FIGURE 5.7 – Diagramme de séquence DS-3a – Lancer un scan SAST (orchestration Celery/Redis).

b) Diagramme DS-3b : Suivi temps réel via Socket.IO

Le diagramme DS-3b (figure 5.8) décrit le flux de notification temps réel. **ScanProgressUI** (boundary — composant React abonné au namespace Socket.IO /scans/{scan_id}) reçoit les événements `scan.status_changed` et `scan.vulnerability_found` émis par le **SocketIOGateway** (control — python-socketio). La salle (*room*) Socket.IO est scopée par `scan_id` pour éviter les émissions croisées entre projets. Le **CeleryWorker** (control) publie les événements via le bus Redis.



FIGURE 5.8 – Diagramme de séquence DS-3b – Suivi temps réel Socket.IO (scopé par scan_id).

5.14 Réalisation

5.14.1 Intégration SAST (Semgrep, Bandit)

Les workers Celery exécutent Semgrep (`semgrep -config auto -json`) et Bandit (`bandit -r . -f json -q`) en sous-processus isolés. Les sorties sont parsées et normalisées par des adaptateurs dédiés (un par moteur) qui mappent les sévérités natives vers l'enum `VulnerabilitySeverity` et calculent une empreinte (`fingerprint`) unique par vulnérabilité pour détecter les doublons lors des scans répétés. L'infrastructure Celery/Redis introduite dans ce sprint sera étendue au DAST (Sprint 4).

5.15 Test et validation

5.15.1 Tests réalisés

TABLE 5.11 – Tableau de test – Lancer un scan SAST (Sprint 3)

Cas de Test	TC-S3-01 – Scan SAST complet sur dépôt Python
Auteur du cas de test	Développeur / Testeur QA
Description	Vérifier qu'un scan SAST sur un dépôt Python contenant des vulnérabilités connues détecte correctement les failles et les persiste avec les sévérités attendues.
Précondition	Un projet Python est configuré. Un dépôt GitHub de test contenant des vulnérabilités connues (injection SQL, subprocess shell, secret codé en dur) est rattaché au projet. Un worker Celery est opérationnel.
Scénario	1. Envoyer POST /scans avec <code>project_id</code> et <code>method = "github"</code>. 2. Vérifier que la réponse HTTP 201 contient un <code>scan_id</code> et <code>status = "pending"</code>. 3. Observer les événements Socket.IO : <code>scan.status_changed</code> de <code>pending</code> → <code>running</code> → <code>completed</code>. 4. Appeler GET /scans/{id}/vulnerabilities et vérifier que les vulnérabilités connues sont présentes avec les sévérités correctes (<code>critical</code>, <code>high</code>). 5. Vérifier que le mapping des sévérités Bandit (<code>HIGH/MEDIUM/LOW</code>) et Semgrep correspond à l'enum <code>VulnerabilitySeverity</code>.
Postcondition	Les vulnérabilités connues sont détectées et persistées. Le scan passe à <code>completed</code> . Les événements Socket.IO sont reçus dans l'ordre attendu.

5.15.2 Validation

La validation s'est appuyée sur un dépôt Python de test contenant des vulnérabilités intentionnelles (OWASP WebGoat Python adapté). Les résultats Semgrep et Bandit ont été comparés aux sorties attendues pour vérifier la fidélité du mapping de sévérités et l'absence de doublons après normalisation.

5.15.3 Gestion des versions et commits

Le Sprint 3 a été développé sur la branche `feature/sprint3-sast`. Les workers Celery et les adaptateurs de normalisation ont fait l'objet de tests unitaires (mocking des sous-processus Semgrep et Bandit). La couverture de tests est suivie via le pipeline CI.

5.16 Conclusion

Le Sprint 3 a permis d'intégrer la détection SAST et de normaliser les alertes issues de Semgrep et Bandit. L'infrastructure d'orchestration asynchrone Celery/Redis, le suivi temps

réel via Socket.IO et le modèle de données `Scan/Vulnerability` constituent la base technique du pipeline de sécurité. Ces fondations ouvrent la voie au Sprint 4, centré sur la détection dynamique DAST, le scoring de risque et l'assistance IA.

Chapitre

6

Sprint 4 – Workflow de vulnérabilités

6.1 Introduction

Ce sprint met en place le système de gestion collaborative des vulnérabilités détectées lors des scans. Il s'agit de transformer des alertes brutes en tâches de remédiation pilotables : synchronisation des tâches entre les scans, assignation à des responsables, suivi des transitions d'état et diffusion de notifications en temps réel. Ce sprint constitue la couche de gouvernance opérationnelle qui permet aux équipes sécurité et développement de collaborer efficacement autour des résultats des analyses.

6.2 Objectifs du sprint et planification des tâches

6.2.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit permettre de créer des tâches de remédiation à partir des vulnérabilités détectées, de les assigner, de gérer leurs transitions d'état, d'y associer des commentaires et de notifier les parties prenantes en temps réel. Plus précisément :

- synchroniser les tâches `VulnerabilityTask` avec les vulnérabilités issues des scans ;
- implémenter les transitions d'état (`open` → `in_progress` → `fixed` → `verified`) ;
- gérer l'assignation des tâches et les commentaires collaboratifs ;
- émettre des notifications (`Notification`) sur chaque événement clé (assignation, changement d'état) ;
- diffuser les mises à jour en temps réel via `Socket.IO`.

6.2.2 Sprint Backlog

TABLE 6.1 – Sprint Backlog – Sprint 4

ID	User Story	ID Tâche	Tâche	Priorité
S4-01	En tant que Security Manager, je peux créer une tâche de remédiation à partir d'une vulnérabilité détectée et l'assigner à un développeur.	T4-01	Implémenter POST /vulnerability-tasks avec création de VulnerabilityTask liée à une Vulnerability par empreinte.	M
		T4-02	Implémenter l'assignation (assignee_id, due_date) et déclencher la création d'une Notification.	M
S4-02	En tant que développeur, je peux consulter mes tâches assignées et changer leur statut.	T4-03	Exposer GET /vulnerability-tasks?assignee_id=me et PUT /vulnerability-tasks/{id}/status.	M
		T4-04	Valider les transitions d'état autorisées (open→in_progress→fixed→verified) côté API.	M
S4-03	En tant que membre du projet, je peux ajouter un commentaire sur une tâche et notifier les parties prenantes.	T4-05	Implémenter POST /vulnerability-tasks/{id}/comments avec création de VulnerabilityTaskComment.	M
		T4-06	Créer une Notification pour chaque nouveau commentaire à destination du Security Manager et de l'assigné.	S
S4-04	En tant qu'utilisateur, je reçois des notifications temps réel pour les événements qui me concernent.	T4-07	Exposer GET /notifications (paginé, filtre is_read) et PUT /notifications/{id}/read.	M
		T4-08	Émettre des événements Socket.IO (task.assigned, task.status_changed, taskcommented) vers la room de l'utilisateur concerné.	M

6.3 Technologies et concepts utilisés

6.3.1 Workflow de gestion d'états (State Machine)

Description. Un workflow de gestion d'états modélise le cycle de vie d'un objet métier à travers des transitions explicites entre états définis. Chaque transition est validée par une règle métier (qui peut effectuer la transition, dans quelle condition).

Utilisation dans InvisiThreat. Le cycle de vie d'une `VulnerabilityTask` suit quatre états : `open` (tâche créée, non assignée ou non démarrée), `in_progress` (développeur a pris en charge), `fixed` (développeur a appliqué un correctif) et `verified` (Security Manager a validé la correction). Les transitions inverses (`verified` → `open`) sont autorisées si la correction est jugée insuffisante.

Justification. La formalisation des états et des transitions garantit la traçabilité complète du processus de remédiation et empêche les sauts d'état incohérents.

6.3.2 Système de notifications (Notification)

Description. Un système de notifications persistantes stocke en base les messages destinés à chaque utilisateur, avec un indicateur de lecture. Les notifications peuvent être diffusées en push (Socket.IO) ou consultées en pull (endpoint REST).

Utilisation dans InvisiThreat. Chaque événement clé (assignation d'une tâche, changement d'état, nouveau commentaire, fin de scan) crée une ou plusieurs entités `Notification` ciblant les utilisateurs concernés. Le champ `is_read` permet de distinguer les notifications non lues. Les événements Socket.IO (`notification.new`) permettent une mise à jour instantanée du badge dans l'interface sans rechargement.

6.3.3 SQLAlchemy — Alembic (migrations Sprint 4)

Description. Les nouvelles entités introduites dans ce sprint (`vulnerability_tasks`, `vulnerability_task_comments`, `notifications`) nécessitent des migrations de schéma versionnées.

Utilisation dans InvisiThreat. Les scripts Alembic de ce sprint ajoutent les tables correspondantes avec leurs contraintes de clés étrangères, index et enums PostgreSQL natifs pour les colonnes de statut.

6.4 Analyse et spécification des besoins

6.4.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 4 (figure 6.4) représente les interactions entre le **Security Manager** (création et vérification des tâches) et le **Developer** (prise en charge, correction, commentaires). Le cas *Créer une tâche* inclut *S'authentifier* et déclenche *Envoyer*

une notification. Le cas *Changer le statut* est disponible pour les deux acteurs selon leur rôle dans la tâche. Le cas *Consulter les notifications* est transverse à tous les rôles.

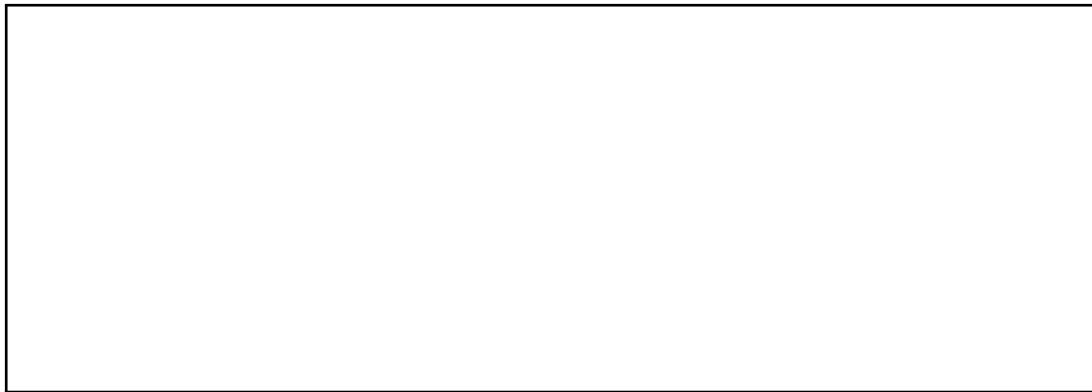


FIGURE 6.1 – Diagramme de cas d'utilisation – Sprint 4 (Workflow de vulnérabilités).

6.4.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Créer et assigner une tâche de remédiation

TABLE 6.2 – Description du cas d'utilisation – Créer et assigner une tâche

Titre	Créer et assigner une tâche de remédiation
Acteur	Security Manager (security_manager)
Résumé	Créer une VulnerabilityTask liée à une vulnérabilité détectée, l'assigner à un développeur et fixer une date limite de correction.
Préconditions	Un scan completed existe avec des vulnérabilités open. L'utilisateur est Security Manager et membre du projet.
Postconditions	VulnerabilityTask créée avec status="open". Notification créée pour l'assigné. Événement Socket.IO task.assigned émis.
Scénario nominal	1. Le Security Manager sélectionne une vulnérabilité depuis le tableau de bord. 2. Il crée une tâche (POST /vulnerability-tasks) en spécifiant assignee_id et due_date. 3. Le système persiste la VulnerabilityTask et crée une Notification pour le développeur assigné. 4. Socket.IO émet task.assigned vers la room de l'utilisateur assigné. 5. Le développeur reçoit la notification et voit la tâche dans son tableau de bord.
Scénario alternatif	2.a assignee_id ne correspond pas à un membre du projet : HTTP 400, tâche non créée. 2.b due_date dans le passé : HTTP 422, erreur de validation Pydantic.

b) Cas d'utilisation : Transition d'état et commentaire**TABLE 6.3 – Description du cas d'utilisation – Transition d'état**

Titre	Changer le statut d'une tâche et ajouter un commentaire
Acteur	Développeur (developer), Security Manager (security_manager)
Résumé	Faire évoluer le statut d'une VulnerabilityTask selon le cycle (open→in_progress→fixed→verified) et documenter l'avancement par des commentaires.
Préconditions	La tâche existe avec un statut compatible. L'utilisateur est l'assigné (pour in_progress/fixed) ou le Security Manager (pour verified).
Postconditions	Statut mis à jour. VulnerabilityTaskComment créé si commentaire. Notification envoyée aux parties prenantes.
Scénario nominal	1. Le développeur change le statut de la tâche à in_progress (PUT /vulnerability-tasks/{id}/status). 2. Il ajoute un commentaire expliquant son approche (POST /vulnerability-tasks/{id}/comments). 3. Après correction, il passe le statut à fixed. 4. Une notification est envoyée au Security Manager. 5. Le Security Manager vérifie la correction et passe à verified.
Scénario alternatif	1.a Transition non autorisée (ex. : open→fixed directement) : HTTP 400, message d'erreur explicite. 5.a Correction insuffisante : le Security Manager repasse à open avec un commentaire expliquant le rejet.

6.5 Modélisation conceptuelle

6.5.1 Diagramme de classes

Le diagramme de classes du Sprint 4 (figure 6.5) modélise les entités du workflow collaboratif. VulnerabilityTask est l'entité centrale, liée à Vulnerability (via fingerprint) et à User (via assignee_id). VulnerabilityTaskComment est lié à VulnerabilityTask (1..N) et à User (auteur). Notification est liée à User (1..N) et référence optionnellement la tâche concernée via resource_id.

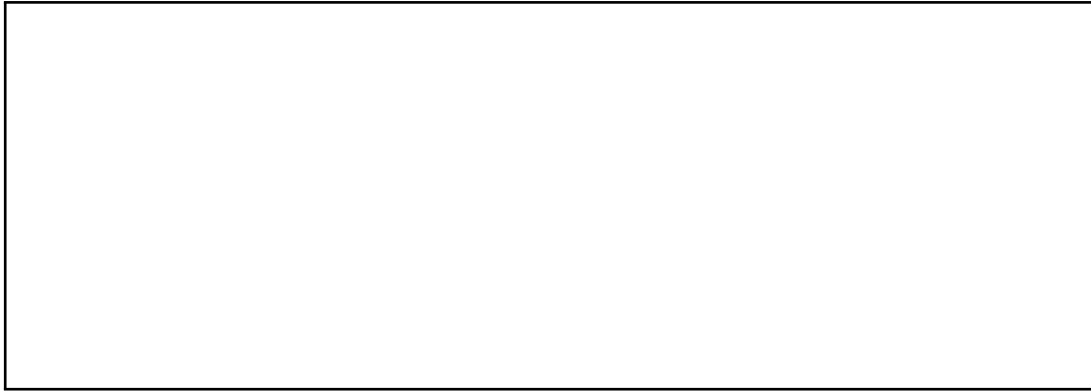


FIGURE 6.2 – Diagramme de classes – Sprint 4 (VulnerabilityTask, VulnerabilityTaskComment, Notification).

a) Dictionnaire de données

TABLE 6.4 – Dictionnaire de données – Sprint 4

Classe	Code	Désignation	Type
VulnerabilityTask	id	Identifiant unique de la tâche	UUID
VulnerabilityTask	project_id	Référence au projet concerné	UUID (FK → Project)
VulnerabilityTask	scan_id	Référence au scan source	UUID (FK → Scan)
VulnerabilityTask	fingerprint	Empreinte de la vulnérabilité (déduplication)	String
VulnerabilityTask	title	Titre descriptif de la tâche	String
VulnerabilityTask	severity	Sévérité héritée de la vulnérabilité source	String (enum)
VulnerabilityTask	status	État de la tâche (open/in_progress/fixed/verified)	String (enum)
VulnerabilityTask	assignee_id	Référence au développeur assigné	UUID (FK → User, nullable)
VulnerabilityTask	due_date	Date limite de correction	Date (nullable)
VulnerabilityTaskComment	id	Identifiant unique du commentaire	UUID
VulnerabilityTaskComment	task_id	Référence à la tâche concernée	UUID (FK → VulnerabilityTask)
VulnerabilityTaskComment	author_id	Référence à l’auteur du commentaire	UUID (FK → User)
VulnerabilityTaskComment	comment	Contenu textuel du commentaire	Text
VulnerabilityTaskComment	created_at	Horodatage de création	DateTime
Notification	id	Identifiant unique de la notification	UUID
Notification	user_id	Référence à l’utilisateur destinataire	UUID (FK → User)
Notification	type	Type (task_assigned, task_status_changed, task_commented, scan_completed)	String
Notification	message	Contenu lisible de la notification	String
Notification	is_read	Indicateur de lecture	Boolean
Notification	created_at	Horodatage de création	DateTime

6.5.2 Diagramme de séquence

a) Diagramme DS-4 : Sync tâches — Assignment • Statut • Notification

Le diagramme DS-4 (figure 6.3) modélise le cycle complet d’une tâche de remédiation selon la notation BCE. **VulnerabilityPanel** (boundary — interface React) soumet les actions

au **VulnWorkflowController** (control — endpoints /vulnerability-tasks). Le contrôleur met à jour l'entité **VulnerabilityTask** (entity) et crée les entités **VulnerabilityTaskComment** et **Notification** (entity) dans une transaction atomique. Le **SocketIOGateway** (control) émet les événements `task.assigned`, `task.status_changed` et `taskcommented` vers les rooms des utilisateurs concernés (`user_{id}`).

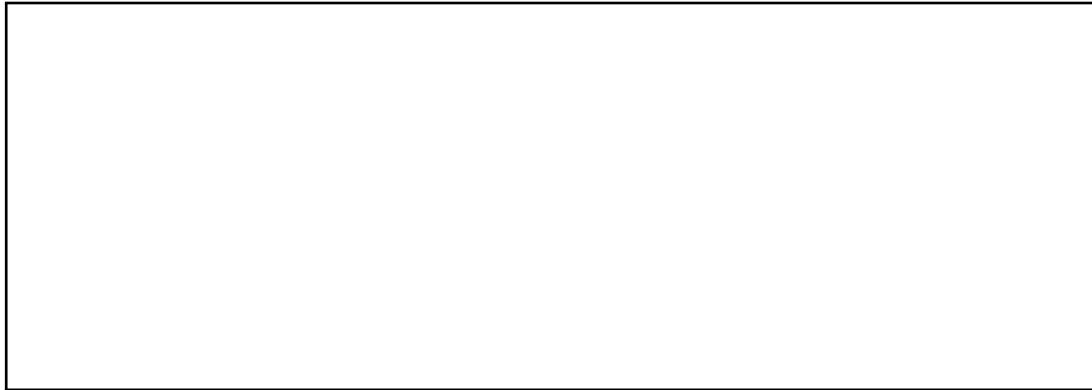


FIGURE 6.3 – Diagramme de séquence DS-4 – Sync tâches · Assignment · Statut · Notification.

6.6 Réalisation

6.6.1 Workflow d'états et assignation

Les transitions d'état de **VulnerabilityTask** sont validées par une matrice de transitions côté API : seules les transitions autorisées (`open→in_progress`, `in_progress→fixed`, `fixed→verified`, `verified→open`) sont acceptées. L'assignation met à jour `assignee_id` et déclenche immédiatement la création d'une **Notification** et l'émission d'un événement **Socket.IO** vers la room `user_{assignee_id}`.

6.6.2 Commentaires et notifications

L'ajout d'un commentaire (**VulnerabilityTaskComment**) notifie automatiquement l'assigné et le Security Manager du projet. Les notifications non lues (`is_read=false`) sont accessibles via `GET /notifications?is_read=false` et affichées dans le badge de l'interface. L'endpoint `PUT /notifications/{id}/read` permet le marquage individuel ; `PUT /notifications/read-all` marque toutes les notifications en lot.

6.7 Test et validation

6.7.1 Tests réalisés

TABLE 6.5 – Tableau de test – Cycle complet de remédiation (Sprint 4)

Cas de Test	TC-S4-01 – Cycle complet open → verified avec notifications
Auteur du cas de test	Security Manager + Développeur / Testeur QA
Description	Vérifier le cycle complet de remédiation d'une vulnérabilité, de la création de tâche à la vérification, avec génération des notifications à chaque étape.
Précondition	Un scan completed avec au moins une vulnérabilité. Un développeur membre du projet.
Scénario	1. Security Manager : POST /vulnerability-tasks (assignation au développeur). 2. Vérifier Notification créée pour le développeur (type="task_assigned"). 3. Développeur : PUT /vulnerability-tasks/{id}/status → in_progress. 4. Développeur : POST /vulnerability-tasks/{id}/comments (explication de la correction). 5. Développeur : PUT → fixed; vérifier notification Security Manager. 6. Security Manager : PUT → verified. 7. Vérifier les événements Socket.IO reçus à chaque étape (task.assigned, task.status_changed).
Postcondition	Tâche à l'état verified. Toutes les notifications créées. Tous les événements Socket.IO émis.

6.7.2 Validation

Les tests d'intégration couvrent les scénarios nominaux et les cas de transition interdite (HTTP 400 attendu). Les notifications en temps réel ont été validées via un client Socket.IO de test abonné aux rooms des deux utilisateurs impliqués.

6.7.3 Gestion des versions et commits

Ce sprint a été développé sur la branche feature/sprint4-vuln-workflow. Les migrations Alembic ajoutant les tables vulnerability_tasks, vulnerability_task_comments et notifications ont été versionnées.

6.8 Conclusion

Le Sprint 4 a mis en place le système collaboratif de gestion des vulnérabilités d'InvisiThreat. La formalisation des transitions d'état, l'assignation des tâches et le système de notifications temps réel transforment les résultats bruts des scans en un processus de remédiation

structuré, traçable et exploitable par les équipes. Le Sprint 5 apportera la dimension analytique et l'assistance IA sur l'ensemble des données collectées.

6.9 Introduction

Ce sprint finalise la chaîne DevSecOps par l'intégration de l'analyse dynamique (DAST) via OWASP ZAP, l'extension de l'orchestration Celery/Redis au DAST, la mise en place du workflow de gestion des vulnérabilités (états, assignation, commentaires), le calcul du scoring de risque, les notifications temps réel et l'assistance IA locale via Ollama. L'objectif est de compléter le pipeline de sécurité et de fournir aux équipes une remédiation priorisée, contextualisée et traçable.

6.10 Objectifs du sprint et planification des tâches

6.10.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit permettre de lancer un scan DAST sur une URL cible via OWASP ZAP, de calculer un score de risque agrégé, de gérer le cycle de vie des vulnérabilités (tâches assignées, commentaires, transitions d'état), de notifier en temps réel et d'obtenir des recommandations de remédiation via l'assistant IA local (Ollama). Plus précisément :

- intégrer OWASP ZAP (mode daemon, API REST) pour l'analyse dynamique ;
- étendre l'orchestration Celery/Redis au DAST et calculer le scoring de risque (RiskScore) ;
- mettre en place le workflow complet des vulnérabilités (VulnerabilityTask : états, assignation, commentaires) ;
- activer l'assistant IA local (Ollama, modèles Mistral/Qwen) via l'action *Assist with AI* ;
- générer des rapports de sécurité exportables (PDF/JSON) via SecurityReport.

6.10.2 Sprint Backlog

TABLE 6.6 – Sprint Backlog – Sprint 4

ID	User Story	ID Tâche	Tâche	Priorité
S4-01	En tant que développeur, je peux lancer un scan DAST sur une URL cible via OWASP ZAP.	T4-01	Implémenter l'endpoint POST /scans avec method = "dast" et déclenchement du worker DAST Celery.	M
		T4-02	Développer le worker Celery DAST : démarrage du contexte ZAP, spider, scan actif, collecte des alertes.	M
S4-02	En tant que Security Manager, je peux visualiser le score de risque agrégé d'un scan.	T4-03	Calculer le RiskScore (agrégation CVSS pondérée par sévérité) et persister après chaque scan.	M
		T4-04	Exposer GET /scans/{id}/risk-score et l'intégrer dans le tableau de bord.	M
S4-03	En tant que Security Manager ou développeur, je peux gérer le cycle de vie d'une vulnérabilité (état, assignation, commentaires).	T4-05	Implémenter les endpoints CRUD pour VulnerabilityTask avec transitions d'état (open/in_progress/fixed/verified).	M
		T4-06	Implémenter les commentaires (VulnerabilityTaskComment) et les notifications d'assignation (Notification).	M
S4-04	En tant que développeur ou Security Manager, je peux obtenir une recommandation de remédiation via l'assistant IA local (Ollama).	T4-07	Intégrer l'adaptateur Ollama (modèles Mistral/Qwen) pour l'action <i>Assist with AI</i> sur une vulnérabilité.	M
		T4-08	Persister la recommandation IA dans Recommendation (explication, exemple de correctif, référence CWE).	M
S4-05	En tant que Security Manager, je peux générer et exporter un rapport de sécurité PDF/JSON.	T4-09	Implémenter la génération de SecurityReport avec ReportLab (PDF) et sérialisation JSON.	S

6.11 Technologies et concepts utilisés

6.11.1 OWASP ZAP (API REST daemon)

Description. OWASP Zed Attack Proxy (ZAP) est un outil d'analyse dynamique web open source. En mode daemon (`zap.sh -daemon`), il expose une API REST permettant de piloter les phases de scan : création de contexte, spider (exploration de l'application), scan actif (injection de payloads), collecte des alertes.

Utilisation dans InvisiThreat. Le worker Celery DAST interagit avec ZAP via son API REST. Il crée un contexte dédié par scan, déclenche le spider sur l'URL cible, lance le scan actif et collecte les alertes. Les alertes ZAP sont normalisées vers le modèle Vulnerability avec mapping des sévérités (High/Medium/Low/Informational → critical/high/medium/info).

Justification. ZAP est la référence open source pour le DAST web. Son API REST permet une intégration programmatique complète sans interface graphique, adaptée à une orchestration depuis Celery.

6.11.2 python-socketio (Socket.IO temps réel)

Description. python-socketio est l'implémentation Python du protocole Socket.IO, permettant une communication bidirectionnelle en temps réel entre le serveur et les clients web via WebSocket (avec fallback long-polling).

Utilisation dans InvisiThreat. Les workers Celery publient des événements Socket.IO (`scan.status_changed`, `vulnerability.found`, `task.assigned`) via un canal Redis partagé. Le gateway Socket.IO (intégré à FastAPI) les diffuse aux clients abonnés. Les salles (*rooms*) sont scopées par `scan_id` pour éviter les émissions croisées entre projets. La gestion des connexions simultanées et la scalabilité horizontale représentent les contraintes principales.

Justification. Le suivi en temps réel des scans DAST (qui peuvent durer plusieurs dizaines de minutes) est essentiel pour l'expérience utilisateur. Socket.IO assure la rétrocompatibilité navigateur et la gestion automatique des reconnections.

6.11.3 Ollama (LLM local — Mistral/Qwen)

Description. Ollama est un serveur d'inférence local qui permet d'exécuter des modèles LLM (Large Language Models) tels que Mistral et Qwen sur du matériel standard sans accès à des API cloud. Il expose une API HTTP locale (`POST /api/generate`) pour soumettre des prompts et récupérer les réponses générées.

Utilisation dans InvisiThreat. L'action *Assist with AI* déclenche un appel à l'adaptateur Ollama, qui soumet un prompt enrichi avec le titre, la description et le code vulnérable de la Vulnerability sélectionnée. La réponse est parsée et persistée dans Recommendation (explication, exemple de correctif, référence CWE). L'inférence locale garantit la confidentialité totale du code analysé, sans transmission vers des serveurs externes.

Justification. L'utilisation d'un LLM local (Ollama) est un différenciateur majeur d'InvisiThreat par rapport aux solutions SaaS. Elle permet d'analyser du code propriétaire

sensible sans risque de fuite vers des fournisseurs cloud, répondant aux exigences de confidentialité de Mobelite.

6.11.4 ReportLab (génération PDF)

Description. ReportLab est une bibliothèque Python permettant la génération programmatique de documents PDF, avec support des tableaux, graphiques et mise en page complexe.

Utilisation dans InvisiThreat. La génération de SecurityReport utilise ReportLab pour produire des rapports PDF structurés contenant : résumé exécutif, score de risque, liste des vulnérabilités par sévérité, statistiques de remédiation et recommandations IA. Le contenu est également sérialisé en JSON pour les intégrations API.

6.12 Analyse et spécification des besoins

6.12.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 4 (figure 6.4) représente les interactions entre les acteurs **Developer**, **Security Manager** et les systèmes externes **OWASP ZAP** et **Ollama**. Le cas *Lancer un scan DAST* inclut *S'authentifier* et communique avec OWASP ZAP. Le cas *Gérer le workflow de vulnérabilités* regroupe les sous-cas *Assigner une tâche*, *Changer l'état* et *Ajouter un commentaire*. Le cas *Obtenir une recommandation IA* étend *Consulter une vulnérabilité* et communique avec Ollama. Le cas *Générer un rapport* est disponible uniquement pour le security_manager.



FIGURE 6.4 – Diagramme de cas d'utilisation – Sprint 4.

6.12.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Lancer un scan DAST

TABLE 6.7 – Description du cas d'utilisation – Lancer un scan DAST

Titre	Lancer un scan DAST (OWASP ZAP)
Acteur	Développeur (developer), Administrateur (admin)
Résumé	Permet de soumettre une URL cible pour analyse dynamique via OWASP ZAP, orchestrée de façon asynchrone par Celery.
Préconditions	L'utilisateur est authentifié et membre du projet. Le projet est configuré avec <code>analysis_type = "dast"</code> ou <code>"both"</code> . L'URL cible est accessible depuis le réseau du worker. Le daemon ZAP est démarré.
Postconditions	Un Scan est créé avec <code>method = "dast"</code> , <code>status = "pending"</code> . Les <code>Vulnerability DAST</code> sont persistées après exécution. Un <code>RiskScore</code> est calculé et persisté.
Scénario nominal	1. L'utilisateur soumet <code>POST /scans</code> avec <code>method = "dast"</code> et l'URL cible. 2. Le système crée le Scan avec <code>status = "pending"</code> et publie la tâche dans Redis. 3. Le worker Celery crée un contexte ZAP, configure l'URL cible et déclenche le spider. 4. Après le spider, le scan actif est lancé (injection de payloads). 5. Les alertes ZAP sont collectées, normalisées et persistées comme <code>Vulnerability</code>. 6. Le <code>RiskScore</code> est calculé (agrégation CVSS pondérée). 7. Le scan passe à <code>status = "completed"</code> et les événements <code>Socket.IO</code> sont émis.
Scénario alternatif	3.a URL cible inaccessible : le worker met le scan à <code>failed</code> avec le message d'erreur dans <code>job_state</code>. 4.a Scan ZAP timeout : les alertes déjà collectées sont sauvegardées et le statut passe à <code>failed</code>.

b) Cas d'utilisation : Gérer le workflow d'une vulnérabilité**TABLE 6.8 – Description du cas d'utilisation – Workflow de vulnérabilité**

Titre	Gérer le workflow d'une vulnérabilité (états, assignation, commentaires)
Acteur	Security Manager (security_manager), Développeur (developer)
Résumé	Permet de créer une tâche de remédiation à partir d'une vulnérabilité, de l'assigner à un développeur, de faire évoluer son état (open → in_progress → fixed → verified) et d'y ajouter des commentaires.
Préconditions	La vulnérabilité existe dans un scan completed. L'utilisateur est Security Manager ou développeur assigné.
Postconditions	Un VulnerabilityTask est créé ou mis à jour. Une Notification est envoyée à l'assigné. Un AuditLog trace la transition d'état.
Scénario nominal	1. Le Security Manager sélectionne une vulnérabilité et crée une tâche (POST /vulnerability-tasks). 2. Il assigne la tâche à un développeur (assignee_id) et fixe une échéance (due_date). 3. Une Notification est créée pour le développeur assigné. 4. Le développeur reçoit la notification, consulte la tâche et change l'état à in_progress. 5. Après correction, le développeur change l'état à fixed et ajoute un commentaire. 6. Le Security Manager vérifie la correction et change l'état à verified.
Scénario alternatif	4.a Développeur non disponible : le Security Manager réassigne la tâche à un autre développeur; une nouvelle notification est envoyée. 6.a La correction est insuffisante : le Security Manager remet l'état à open avec un commentaire explicatif.

c) Cas d'utilisation : Obtenir une recommandation IA

TABLE 6.9 – Description du cas d'utilisation – Recommandation IA (Ollama)

Titre	Obtenir une recommandation IA (<i>Assist with AI</i>)
Acteur	Développeur (developer), Security Manager (security_manager)
Résumé	Permet d'obtenir une explication contextuelle et un exemple de correctif pour une vulnérabilité donnée, via un LLM local (Ollama, modèles Mistral/Qwen), garantissant la confidentialité du code.
Préconditions	La vulnérabilité existe. L'utilisateur est membre du projet. Le serveur Ollama est démarré et le modèle sélectionné est disponible.
Postconditions	Une Recommendation est persistée avec explanation, fix_example et cwe_reference.
Scénario nominal	1. L'utilisateur clique sur <i>Assist with AI</i> depuis le détail de la vulnérabilité. 2. Le système soumet un prompt enrichi à l'adaptateur Ollama (titre, description, extrait de code vulnérable, contexte CWE). 3. Ollama génère une explication et un exemple de correctif dans la langue du prompt. 4. La réponse est parsée et persistée dans Recommendation. 5. La recommandation est affichée dans l'interface avec la référence CWE.
Scénario alternatif	3.a Timeout Ollama (modèle non chargé ou surcharge) : le système retourne un message d'erreur et invite à réessayer. Aucune Recommendation n'est persistée.

6.13 Modélisation conceptuelle

6.13.1 Diagramme de classes

Le diagramme de classes du Sprint 4 (figure 6.5) modélise les entités du pipeline DAST, du workflow de vulnérabilités et de l'IA. RiskScore est lié à Scan (1..1) et encapsule le score agrégé calculé après chaque scan. VulnerabilityTask est lié à Vulnerability (via fingerprint) et à User (assignation, assignee_id). VulnerabilityTaskComment est lié à VulnerabilityTask (1..N). Recommendation est lié à Vulnerability (0..1). Notification est lié à User (1..N). SecurityReport est lié à Scan et Project.

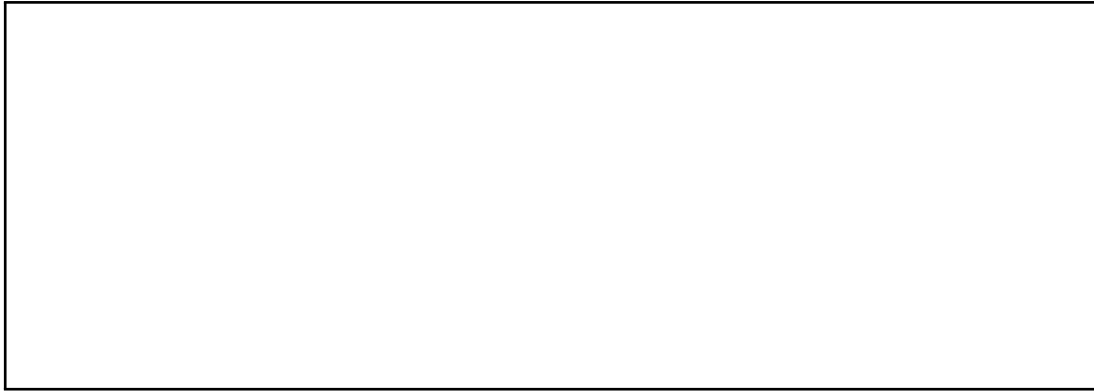


FIGURE 6.5 – Diagramme de classes – Sprint 4 (entités RiskScore, VulnerabilityTask, VulnerabilityTaskComment, Recommendation, Notification, SecurityReport).

a) Dictionnaire de données

TABLE 6.10 – Dictionnaire de données – Sprint 4

Classe	Code	Désignation	Type
RiskScore	id	Identifiant unique du score de risque	UUID
RiskScore	scan_id	Référence au scan évalué	UUID (FK → Scan)
RiskScore	score	Score de risque agrégé (0.0–10.0, pondération CVSS)	Float
RiskScore	level	Niveau de risque qualitatif (critical/high/medium/low)	String (enum)
VulnerabilityTask	id	Identifiant unique de la tâche de remédiation	UUID
VulnerabilityTask	project_id	Référence au projet concerné	UUID (FK → Project)
VulnerabilityTask	scan_id	Référence au scan source	UUID (FK → Scan)
VulnerabilityTask	fingerprint	Empreinte unique de la vulnérabilité (déduplication)	String
VulnerabilityTask	title	Titre descriptif de la tâche	String
VulnerabilityTask	severity	Sévérité héritée de la vulnérabilité source	String (enum)
VulnerabilityTask	status	État de la tâche (open/in_progress/fixed/verified)	String (enum)
VulnerabilityTask	assignee_id	Référence au développeur assigné	UUID (FK → User, nullable)
VulnerabilityTask	due_date	Date limite de correction	Date (nullable)
VulnerabilityTaskComment	id	Identifiant unique du commentaire	UUID
VulnerabilityTaskComment	task_id	Référence à la tâche concernée	UUID (FK → VulnerabilityTask)
VulnerabilityTaskComment	author_id	Référence à l’auteur du commentaire	UUID (FK → User)
VulnerabilityTaskComment	comment	Contenu textuel du commentaire	Text
VulnerabilityTaskComment	created_at	Horodatage de création du commentaire	DateTime
Recommendation	id	Identifiant unique de la recommandation IA	UUID
Recommendation	vulnerability_id	Référence à la vulnérabilité concernée	UUID (FK → Vulnerability)
Recommendation	explanation	Explication contextuelle générée par le LLM	Text
Recommendation	fix_example	Exemple de code corrigé généré par le LLM	Text
Recommendation	cwe_reference	Référence CWE associée à la vulnérabilité	String (nullable)
ISIMS SFAX			
Notification	id	Identifiant unique de la notification	UUID
Notification	user_id	Référence à l’utilisateur destinataire	UUID (FK → User)

6.13.2 Diagrammes de séquence

a) Diagramme DS-4a : Lancer un scan DAST via OWASP ZAP

Le diagramme DS-4a (figure 6.6) modélise le flux DAST selon la notation BCE. **DASTScanForm** (boundary) soumet la requête à **ScanController** (control — POST /scans). Après création de l'entité **Scan** (entity), le contrôleur publie la tâche dans Redis. Le **CeleryWorker** (control) interagit avec **OWASP ZAP** (système externe) via API REST : création du contexte, spider, scan actif, collecte des alertes. Les entités **Vulnerability** et **RiskScore** (entity) sont persistées en fin d'exécution.

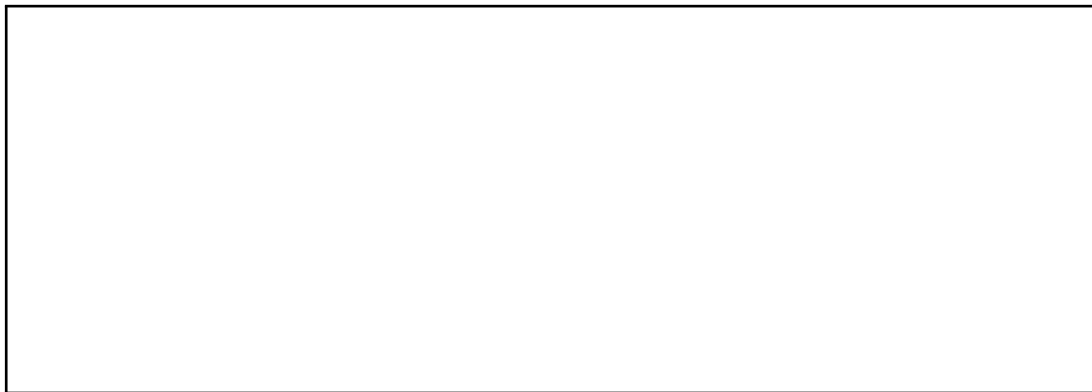


FIGURE 6.6 – Diagramme de séquence DS-4a – Scan DAST via OWASP ZAP.

b) Diagramme DS-4b : Workflow de vulnérabilité (assignation et transition d'état)

Le diagramme DS-4b (figure 6.7) décrit le workflow de gestion d'une vulnérabilité. **VulnerabilityPanel** (boundary) soumet les actions à **VulnWorkflowController** (control — endpoints POST/PUT /vulnerability-tasks). Le contrôleur met à jour l'entité **VulnerabilityTask** (entity), crée le **VulnerabilityTaskComment** (entity) et persiste une **Notification** (entity) pour l'utilisateur assigné. L'événement Socket.IO `task.assigned` est émis en temps réel.



FIGURE 6.7 – Diagramme de séquence DS-4b – Workflow de vulnérabilité (assignation, transition d'état).

c) Diagramme DS-4c : Recommandation IA via Ollama

Le diagramme DS-4c (figure 6.8) décrit le flux de l'action *Assist with AI*. **VulnDetailPanel** (boundary) déclenche la requête vers **LLMController** (control — endpoint POST /vulnerabilities/{id}/assist). Ce contrôleur appelle **LLMAdapter/Ollama** (control — adaptateur HTTP interne vers le serveur Ollama) qui soumet le prompt enrichi et reçoit la réponse. L'entité **Recommendation** (entity) est persistée et retournée à l'interface.

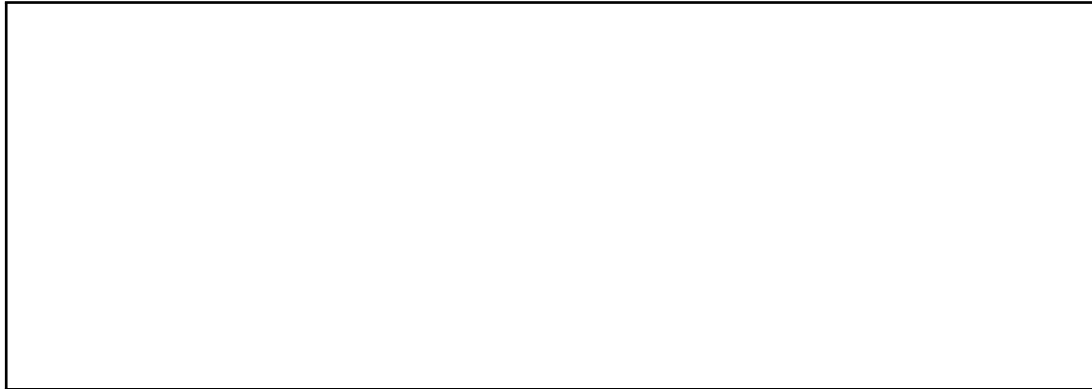


FIGURE 6.8 – Diagramme de séquence DS-4c – Recommandation IA via Ollama (action Assist with AI).

6.14 Réalisation

6.14.1 Intégration DAST (OWASP ZAP)

Le worker Celery DAST démarre une session ZAP via API REST, configure le contexte avec l'URL cible et les paramètres d'authentification si nécessaire, déclenche le spider puis le scan actif. Les alertes ZAP sont collectées après le scan et normalisées par l'adaptateur DAST, qui mappe les sévérités ZAP (High/Medium/Low/Informational) vers l'enum `VulnerabilitySeverity`.

6.14.2 Orchestration et workflow de vulnérabilités

Les `VulnerabilityTask` modélisent le cycle de vie de la remédiation : `open` → `in_progress` → `fixed` → `verified`. L'assignation déclenche une `Notification` et un événement `Socket.IO`. Les commentaires (`VulnerabilityTaskComment`) permettent un échange structuré entre le Security Manager et le développeur assigné sans quitter la plateforme.

6.14.3 Assistant IA (Ollama)

L'adaptateur Ollama construit un prompt structuré incluant le contexte technique de la vulnérabilité (titre, description, extrait de code, CWE). La réponse du modèle (Mistral ou Qwen selon la configuration) est parsée pour extraire l'explication et l'exemple de correctif, puis persistée dans `Recommendation`. L'inférence locale garantit qu'aucun code source ne quitte l'infrastructure de Mobelite.

6.15 Test et validation

6.15.1 Tests réalisés

TABLE 6.11 – Tableau de test – Scan DAST et recommandation IA (Sprint 4)

Cas de Test	TC-S4-01 – Scan DAST sur application cible de test
Auteur du cas de test	Security Manager / Testeur QA
Description	Vérifier qu'un scan DAST sur une application de test (DVWA — Damn Vulnerable Web Application) détecte les failles OWASP Top 10 exposées et calcule un RiskScore cohérent.
Précondition	Une instance DVWA est accessible à l'URL cible. Le worker Celery et le daemon ZAP sont opérationnels. Un projet avec <code>analysis_type = "dast"</code> est configuré.
Scénario	1. Envoyer POST <code>/scans</code> avec <code>method = "dast"</code> et l'URL de DVWA. 2. Observer les événements Socket.IO : <code>pending</code> → <code>running</code> → <code>completed</code>. 3. Appeler GET <code>/scans/{id}/vulnerabilities</code> et vérifier la détection des vulnérabilités XSS et SQL injection avec sévérité <code>high</code> ou <code>critical</code>. 4. Appeler GET <code>/scans/{id}/risk-score</code> et vérifier que le score est supérieur à 7.0 pour DVWA.
Postcondition	Les vulnérabilités OWASP connues de DVWA sont détectées. Le RiskScore reflète le niveau de risque élevé de l'application cible.

TABLE 6.12 – Tableau de test – Workflow de vulnérabilité (Sprint 4)

Cas de Test	TC-S4-02 – Cycle complet de remédiation d'une vulnérabilité
Auteur du cas de test	Security Manager / Développeur / Testeur QA
Description	Vérifier le cycle complet : création d'une VulnerabilityTask, assignation à un développeur, notification, transitions d'état et vérification finale.
Précondition	Un scan <code>completed</code> existe avec au moins une vulnérabilité. Un utilisateur avec le rôle <code>developer</code> est membre du projet.
Scénario	1. Le Security Manager crée une tâche (POST <code>/vulnerability-tasks</code>) et l'assigne au développeur. 2. Vérifier la création d'une Notification pour le développeur assigné. 3. Le développeur change l'état à <code>in_progress</code> et ajoute un commentaire. 4. Le développeur change l'état à <code>fixed</code>. 5. Le Security Manager change l'état à <code>verified</code>. 6. Vérifier que l'historique des transitions est traçable dans <code>audit_logs</code>.
Postcondition	La tâche est à l'état <code>verified</code> . Toutes les transitions sont auditées. Les notifications ont été créées à chaque étape clé.

6.15.2 Validation

La validation bout en bout a combiné des scans SAST et DAST sur des projets de test, vérifiant la cohérence du pipeline complet : normalisation, scoring, workflow, notifications temps réel et recommandations IA. Les tests d'intégration de l'adaptateur Ollama ont utilisé des réponses mockées pour garantir l'indépendance des tests vis-à-vis de la disponibilité du modèle LLM.

6.15.3 Gestion des versions et commits

Le Sprint 4 a été développé sur la branche `feature/sprint4-dast-ia`. Les migrations Alembic ajoutant les tables `risk_scores`, `vulnerability_tasks`, `vulnerability_task_comments`, `recommendations`, `notifications` et `security_reports` ont été versionnées et testées en isolation.

6.16 Conclusion

Le Sprint 4 clôture la chaîne DevSecOps d'InvisiThreat en intégrant l'analyse dynamique DAST (OWASP ZAP), le scoring de risque agrégé, le workflow complet de gestion des vulnérabilités, les notifications temps réel (Socket.IO) et l'assistance IA locale (Ollama, Mistral/Qwen). La plateforme dispose désormais d'un pipeline de sécurité unifié, traçable et enrichi par intelligence artificielle, garantissant la confidentialité du code grâce à l'inférence locale.

Chapitre

7

Sprint 5 – Dashboard, Recommandation LLM et Rapport

7.1 Introduction

Ce sprint clôture le développement fonctionnel d'InvisiThreat en apportant trois dimensions complémentaires : un tableau de bord analytique consolidant les métriques de sécurité, un module de recommandation par intelligence artificielle locale (Ollama, modèles Mistral/Qwen) permettant d'expliquer et de contextualiser chaque vulnérabilité via l'action *Assist with AI*, et un système de génération de rapports de sécurité exportables (PDF et JSON) à destination des équipes de direction et d'audit. L'ensemble de ces fonctionnalités repose sur les données accumulées lors des sprints précédents.

7.2 Objectifs du sprint et planification des tâches

7.2.1 Objectifs attendus

À l'issue de ce sprint, la plateforme doit offrir :

- un tableau de bord consolidé par projet avec agrégations de sécurité (scores de risque, répartition par sévérité, taux de remédiation, tendances temporelles) ;
- un assistant IA local (*Assist with AI*) capable d'expliquer une vulnérabilité et de proposer un exemple de correctif, en inférant sur un modèle Ollama local (confidentialité totale du code) ;
- un module de génération de `SecurityReport` exportable en PDF (ReportLab) et JSON, filtrable par projet, période et sévérité.

7.2.2 Sprint Backlog

TABLE 7.1 – Sprint Backlog – Sprint 5

ID	User Story	ID Tâche	Tâche	Priorité
S5-01	En tant que Security Manager ou développeur, je peux consulter un tableau de bord avec les métriques de sécurité de mes projets.	T5-01	Implémenter <code>GET /dashboard/stats</code> avec agrégation des RiskScore, compteurs par sévérité et taux de remédiation.	M
		T5-02	Implémenter <code>GET /dashboard/trends</code> retournant les scores de risque sur les N derniers scans par projet.	M
S5-02	En tant que développeur ou Security Manager, je peux obtenir une recommandation IA contextualisée pour une vulnérabilité via l'action <i>Assist with AI</i> .	T5-03	Implémenter l'adaptateur Ollama (<code>POST /api/generate</code>) avec construction du prompt enrichi (titre, description, extrait de code, CWE).	M
		T5-04	Exposer <code>POST /vulnerabilities/{id}/assist</code> et persister la réponse dans <code>Recommendation</code> .	M
S5-03	En tant que Security Manager, je peux générer un rapport de sécurité PDF ou JSON filtré par projet, période et sévérité.	T5-05	Implémenter la génération PDF avec ReportLab (résumé exécutif, score de risque, liste des vulnérabilités par sévérité, statistiques de remédiation).	M
		T5-06	Implémenter la sérialisation JSON du rapport et l'endpoint <code>POST /security-reports</code> .	S
		T5-07	Persister le <code>SecurityReport</code> et exposer <code>GET /security-reports/{id}/download?format=pdf json</code> .	S

7.3 Technologies et concepts utilisés

7.3.1 Ollama — LLM local (Mistral, Qwen)

Description. Ollama est un serveur d'inférence local permettant d'exécuter des modèles LLM (Large Language Models) tels que Mistral et Qwen sur du matériel standard sans accès à des API cloud. Il expose une API HTTP locale (`POST /api/generate`) pour soumettre des prompts et récupérer les réponses en streaming ou en mode complet.

Utilisation dans InvisiThreat. L'action *Assist with AI* déclenche un appel à l'adaptateur Ollama. Celui-ci construit un prompt structuré intégrant le titre, la description, l'extrait de code

vulnérable et la référence CWE de la vulnérabilité sélectionnée, puis soumet ce prompt au modèle local. La réponse est parsée pour extraire l'explication contextuelle (`explanation`) et l'exemple de code corrigé (`fix_example`). Ces données sont persistées dans l'entité `Recommendation`.

Justification. L'utilisation d'un LLM local (Ollama) est le différenciateur principal d'InvisiThreat par rapport aux solutions SaaS concurrentes : aucun code source ni donnée sensible ne transite vers des serveurs externes. Le choix des modèles Mistral et Qwen repose sur leur équilibre entre qualité de génération et performances sur du matériel standard (quantification 4-bit ou 8-bit).

7.3.2 ReportLab (génération PDF)

Description. ReportLab est une bibliothèque Python permettant la génération programmatique de documents PDF, avec support des tableaux, graphiques vectoriels, en-têtes/pieds de page et mise en page complexe.

Utilisation dans InvisiThreat. La génération du `SecurityReport` utilise ReportLab pour produire un document PDF structuré en plusieurs sections : résumé exécutif (score de risque global, nombre de vulnérabilités par sévérité), liste détaillée des vulnérabilités (titre, sévérité, localisation, CVSS), statistiques de remédiation (taux de résolution, vulnérabilités en attente) et recommandations IA si disponibles. Le document intègre le logo et les couleurs de charte de Mobelite.

Justification. La génération PDF côté serveur garantit une mise en page cohérente et reproductible, indépendante du navigateur ou du système d'exploitation du destinataire.

7.3.3 Agrégations PostgreSQL (tableau de bord)

Description. Les métriques du tableau de bord (scores de risque, tendances, taux de remédiation) requièrent des requêtes SQL agrégées efficaces sur les tables `scans`, `vulnerabilities`, `risk_scores` et `vulnerability_tasks`.

Utilisation dans InvisiThreat. Les endpoints du tableau de bord s'appuient sur des requêtes SQLAlchemy ORM avec `func.count()`, `func.avg()` et des sous-requêtes fenêtrées pour calculer les tendances temporelles. Les résultats sont mis en cache Redis (TTL configurable) pour garantir la réactivité de l'interface même sur des projets avec de nombreux scans historiques.

Justification. Le cache Redis évite les requêtes agrégées répétitives sur des données qui ne changent que lors d'un nouveau scan, préservant la performance de la base de données sous charge.

7.4 Analyse et spécification des besoins

7.4.1 Diagramme de cas d'utilisation du sprint

Le diagramme de cas d'utilisation du Sprint 5 (figure 7.1) représente les interactions entre le **Security Manager** (consultation du dashboard, génération de rapports) et le **Developer**

(consultation du dashboard, action *Assist with AI*). Le cas *Obtenir une recommandation IA* communique avec le système externe **Ollama**. Le cas *Générer un rapport* est accessible uniquement au `security_manager`, avec choix du format (PDF ou JSON) et des filtres.

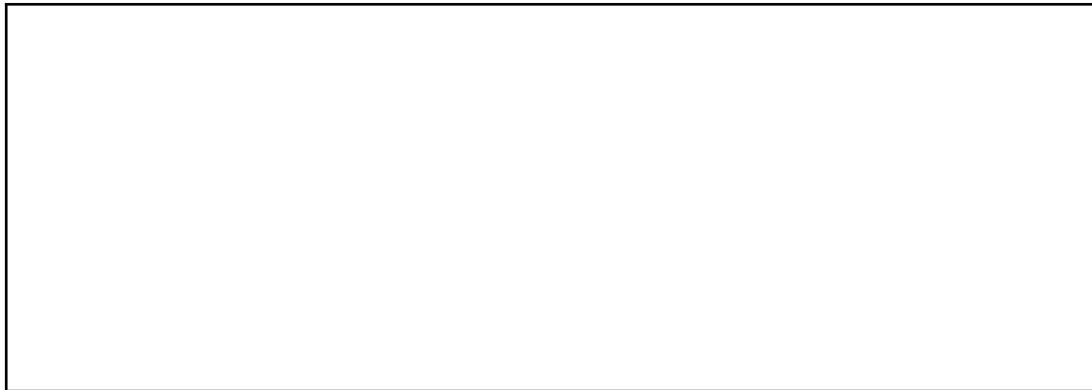


FIGURE 7.1 – Diagramme de cas d'utilisation – Sprint 5 (Dashboard · Recommandation LLM · Rapport).

7.4.2 Description détaillée des cas d'utilisation

a) Cas d'utilisation : Consulter le tableau de bord de sécurité

TABLE 7.2 – Description du cas d'utilisation – Consulter le tableau de bord

Titre	Consulter le tableau de bord de sécurité
Acteur	Développeur (<code>developer</code>), Security Manager (<code>security_manager</code>), Administrateur (<code>admin</code>)
Résumé	Consulter une vue agrégée des métriques de sécurité des projets auxquels l'utilisateur appartient : score de risque actuel, répartition des vulnérabilités par sévérité, taux de remédiation et tendances temporelles.
Préconditions	L'utilisateur est authentifié. Au moins un scan <code>completed</code> existe dans ses projets.
Postconditions	Les métriques agrégées sont retournées depuis le cache Redis ou calculées à la volée.
Scénario nominal	1. L'utilisateur accède au tableau de bord (<code>GET /dashboard/stats?project_id={id}</code>). 2. Le système vérifie le cache Redis ; si valide, retourne les métriques en cache. 3. Sinon, exécute les requêtes SQL agrégées sur PostgreSQL et met à jour le cache. 4. Les métriques sont affichées : score de risque, compteurs par sévérité, taux de résolution, tendance sur les 10 derniers scans.
Scénario alternatif	1.a Aucun scan <code>completed</code> pour ce projet : le dashboard affiche un état vide avec message informatif.

b) Cas d'utilisation : Obtenir une recommandation IA (*Assist with AI*)**TABLE 7.3 – Description du cas d'utilisation – Recommandation IA (Ollama)**

Titre	Obtenir une recommandation IA (<i>Assist with AI</i>)
Acteur	Développeur (<code>developer</code>), Security Manager (<code>security_manager</code>)
Résumé	Obtenir une explication contextuelle et un exemple de code corrigé pour une vulnérabilité donnée, via un LLM exécuté localement (Ollama, Mistral/Qwen), garantissant la confidentialité du code.
Préconditions	La vulnérabilité existe dans un scan completed. L'utilisateur est membre du projet. Le serveur Ollama est démarré et le modèle est disponible.
Postconditions	Recommandation persistée avec explication, <code>fix_example</code> et <code>cwe_reference</code> .
Scénario nominal	1. L'utilisateur clique sur <i>Assist with AI</i> depuis le détail d'une vulnérabilité. 2. Le système soumet un prompt enrichi à l'adaptateur Ollama (titre, description, code vulnérable, contexte CWE). 3. Ollama génère l'explication et l'exemple de correctif. 4. La réponse est parsée et persistée dans <code>Recommandation</code>. 5. La recommandation est affichée dans l'interface avec la référence CWE.
Scénario alternatif	3.a Timeout Ollama (modèle non chargé ou surcharge) : HTTP 503, message d'erreur ; aucune <code>Recommandation</code> créée. 3.b Recommandation déjà existante : le système retourne la <code>Recommandation</code> existante sans appel supplémentaire à Ollama.

c) Cas d'utilisation : Générer un rapport de sécurité

TABLE 7.4 – Description du cas d'utilisation – Générer un rapport

Titre	Générer un rapport de sécurité (PDF ou JSON)
Acteur	Security Manager (security_manager)
Résumé	Générer un rapport de sécurité exportable (PDF ou JSON) pour un projet sur une période donnée, incluant le score de risque, les vulnérabilités par sévérité et les statistiques de remédiation.
Préconditions	L'utilisateur est Security Manager et membre du projet. Au moins un scan completed correspond aux critères de filtre.
Postconditions	Entité SecurityReport créée. Fichier PDF ou JSON disponible au téléchargement via GET /security-reports/{id}/download.
Scénario nominal	1. Le Security Manager soumet POST /security-reports avec project_id, plage de dates et format = "pdf" ou "json". 2. Le système agrège les données des scans concernés. 3. Si PDF : ReportLab génère le document structuré et le stocke temporairement. 4. Si JSON : les données sont sérialisées via Pydantic. 5. L'entité SecurityReport est persistée avec data (JSON sérialisé). 6. Le lien de téléchargement est retourné au Security Manager.
Scénario alternatif	2.a Aucun scan dans la période sélectionnée : HTTP 400, message explicite.

7.5 Modélisation conceptuelle

7.5.1 Diagramme de classes

Le diagramme de classes du Sprint 5 (figure 7.2) modélise les entités analytiques. Recommendation est lié à Vulnerability (0..1) : une vulnérabilité peut avoir au plus une recommandation IA. SecurityReport est lié à Project (N..1) et à Scan (0..N via une table de liaison) : un rapport peut consolider plusieurs scans sur une période. La classe utilitaire DashboardStats (non persistée, produit d'une requête agrégée) encapsule les métriques retournées par GET /dashboard/stats.

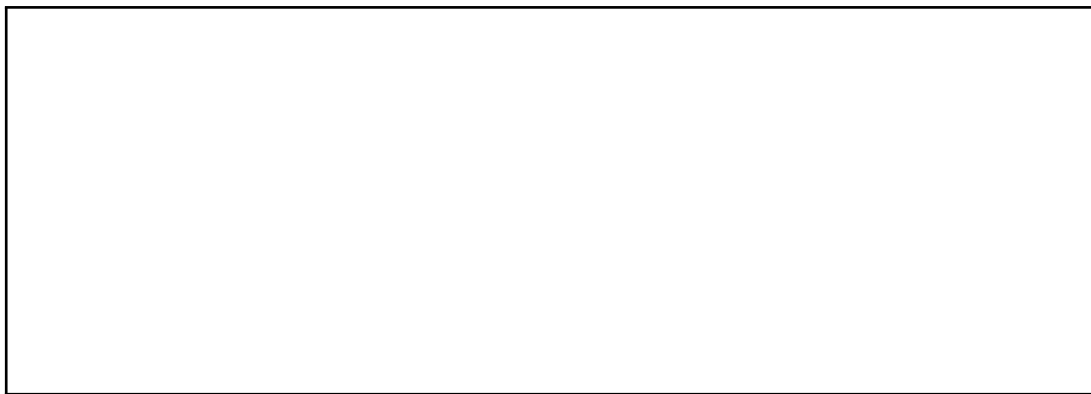


FIGURE 7.2 – Diagramme de classes – Sprint 5 (Recommendation, SecurityReport, DashboardStats).

a) Dictionnaire de données

TABLE 7.5 – Dictionnaire de données – Sprint 5

Classe	Code	Désignation	Type
Recommendation	id	Identifiant unique de la recommandation IA	UUID
Recommendation	vulnerability_id	Référence à la vulnérabilité concernée	UUID (FK → Vulnerability)
Recommendation	explanation	Explication contextuelle générée par le LLM	Text
Recommendation	fix_example	Exemple de code corrigé généré par le LLM	Text
Recommendation	cwe_reference	Référence CWE associée à la vulnérabilité	String (nullable)
Recommendation	model_used	Nom du modèle Ollama utilisé (ex. : mistral:7b)	String
Recommendation	created_at	Horodatage de génération	DateTime
SecurityReport	id	Identifiant unique du rapport	UUID
SecurityReport	project_id	Référence au projet concerné	UUID (FK → Project)
SecurityReport	generated_by	Référence au Security Manager ayant généré le rapport	UUID (FK → User)
SecurityReport	period_start	Début de la période couverte	Date
SecurityReport	period_end	Fin de la période couverte	Date
SecurityReport	format	Format du rapport (pdf ou json)	String (enum)
SecurityReport	generated_at	Horodatage de génération	DateTime
SecurityReport	data	Données du rapport sérialisées (JSON)	JSON
DashboardStats	project_id	Identifiant du projet concerné	UUID
DashboardStats	current_risk_score	Score de risque du dernier scan complété	Float
DashboardStats	risk_level	Niveau qualitatif (critical/high/medium/low)	String
DashboardStats	total_open	Nombre de vulnérabilités à l'état open	Integer
DashboardStats	by_severity	Décompte par sévérité (JSON : {critical: N, high: N, ...})	JSON
DashboardStats	remediation_rate	Taux de vulnérabilités résolues (en pourcentage)	Float
DashboardStats	trend	Historique des scores de risque (N derniers scans)	JSON

7.5.2 Diagramme de séquence

a) Diagramme DS-5 : Dashboard · Recommandation LLM · Rapport

Le diagramme DS-5 (figure 7.3) consolide les trois flux du sprint selon la notation BCE. **DashboardPage** (boundary — interface React) appelle **DashboardController** (control — GET /dashboard/stats) qui interroge le cache **Redis** avant de requêter **PostgreSQL** (entity). L'action *Assist with AI* depuis **VulnDetailPanel** (boundary) déclenche **LLMController** (control — POST /vulnerabilities/{id}/assist) qui appelle **OllamaAdapter** (control) et persiste l'entité **Recommendation** (entity). La génération de rapport depuis **ReportForm** (boundary) déclenche **ReportController** (control — POST /security-reports) qui orchestre les agrégations et délègue la mise en forme à **ReportLabGenerator** (control) avant de persister **SecurityReport** (entity).

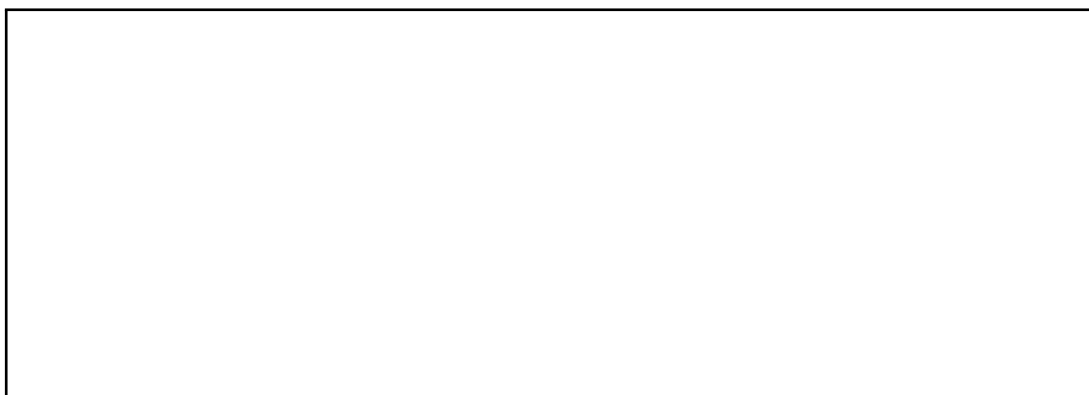


FIGURE 7.3 – Diagramme de séquence DS-5 – Dashboard · Recommandation LLM (Ollama) · Génération de rapport (ReportLab).

7.6 Réalisation

7.6.1 Tableau de bord analytique

Les endpoints du tableau de bord agrègent les données des entités **RiskScore**, **Vulnerability** et **VulnerabilityTask** pour chaque projet de l'utilisateur. Les métriques (score de risque courant, répartition par sévérité, taux de remédiation) sont mises en cache **Redis** avec un TTL de 5 minutes, invalidé automatiquement à la complétion d'un nouveau scan. L'endpoint GET /dashboard/trends retourne les scores de risque des N derniers scans, permettant la visualisation de l'évolution de la posture sécurité dans le temps.

7.6.2 Assistance IA locale (Ollama)

L'adaptateur **Ollama** construit un prompt structuré en trois sections : contexte de la vulnérabilité (titre, description, sévérité, CWE), extrait de code vulnérable (si disponible depuis le champ *evidence*) et instruction explicite de génération. La réponse du modèle est parsée pour extraire les sections *Explication* et *Correctif proposé*. Si une **Recommendation** existe déjà pour la vulnérabilité, elle est retournée directement sans nouvel appel à **Ollama**.

7.6.3 Génération de rapports (ReportLab + JSON)

Le module de génération de rapports prend en entrée les filtres (`project_id`, `period_start`, `period_end`, `min_severity`) et produit un document structuré. Le PDF généré inclut : page de garde (projet, période, Security Manager), résumé exécutif avec score de risque et indicateurs clés, tableau des vulnérabilités triées par sévérité décroissante, graphique à barres de répartition par sévérité, statistiques de remédiation et section recommandations IA. La version JSON expose les mêmes données pour intégration avec des outils tiers (SIEM, ticketing).

7.7 Test et validation

7.7.1 Tests réalisés

TABLE 7.6 – Tableau de test – Recommandation LLM (Sprint 5)

Cas de Test	TC-S5-01 – Génération d’une recommandation IA via Ollama
Auteur du cas de test	Développeur / Testeur QA
Description	Vérifier que l’action <i>Assist with AI</i> produit une Recommandation persistée avec les champs requis, et que les appels redondants retournent la recommandation existante.
Précondition	Une vulnérabilité open existe dans un scan completed. Le serveur Ollama est démarré avec le modèle <code>mistral:7b</code> .
Scénario	1. Envoyer <code>POST /vulnerabilities/{id}/assist</code> avec <code>Authorization: Bearer <token></code>. 2. Vérifier la réponse HTTP 200 contenant <code>explanation</code> (non vide), <code>fix_example</code> (non vide) et <code>cwe_reference</code>. 3. Vérifier la création d’une entité Recommendation en base avec <code>model_used = "mistral:7b"</code>. 4. Renvoyer la même requête ; vérifier qu’aucun nouvel appel Ollama n’est effectué (retour immédiat de la Recommendation existante).
Postcondition	Recommandation persistée. Idempotence garantie : aucun double appel LLM.

TABLE 7.7 – Tableau de test – Génération de rapport PDF (Sprint 5)

Cas de Test	TC-S5-02 – Génération d'un rapport PDF de sécurité
Auteur du cas de test	Security Manager / Testeur QA
Description	Vérifier qu'un Security Manager peut générer un rapport PDF valide pour un projet avec plusieurs scans complétés.
Précondition	Un projet avec au moins deux scans completed et des vulnérabilités de sévérités variées. L'utilisateur est Security Manager du projet.
Scénario	1. Envoyer POST /security-reports avec project_id, period_start, period_end et format="pdf". 2. Vérifier la réponse HTTP 201 avec l'ID du rapport et le lien de téléchargement. 3. Appeler GET /security-reports/{id}/download?format=pdf. 4. Vérifier que le fichier retourné est un PDF valide (en-tête %PDF) contenant les sections attendues.
Postcondition	Rapport PDF généré et téléchargeable. SecurityReport persisté en base.

7.7.2 Validation

La validation du tableau de bord a utilisé des jeux de données avec des historiques de scans simulés pour vérifier la cohérence des agrégations. Les tests de l'adaptateur Ollama ont été exécutés avec un vrai modèle Mistral local pour valider la qualité des recommandations générées sur des vulnérabilités Python connues (injection SQL, mauvaise gestion des secrets).

7.7.3 Gestion des versions et commits

Ce sprint a été développé sur la branche `feature/sprint5-dashboard-llm-report`. Les migrations Alembic ajoutant les tables `recommendations` et `security_reports` ont été versionnées. La génération PDF a été testée via des tests de snapshot comparant les structures de documents générés.

7.8 Conclusion

Le Sprint 5 parachève la plateforme InvisiThreat en y ajoutant les trois dimensions analytiques et intelligentes qui transforment un pipeline de détection en outil de gouvernance complet. Le tableau de bord offre une vision priorisée du risque par projet et dans le temps. L'assistance IA locale (Ollama, Mistral/Qwen) accélère la remédiation en fournissant des explications contextualisées et des correctifs concrets, sans compromettre la confidentialité du code. La génération de rapports PDF et JSON consolide les résultats pour la communication avec les équipes de direction et les auditeurs.

7

CONCLUSION GÉNÉRALE

Le présent projet a conduit à la conception et au développement complet de la plateforme **InvisiThreat**, une solution DevSecOps intégrée de détection automatisée de vulnérabilités. Ce travail, mené en quatre sprints Scrum au sein de l'entreprise Mobilite, a permis de formaliser un pipeline de sécurité applicative couvrant l'ensemble du cycle de détection, de remédiation et de gouvernance.

Bilan technique. Sur le plan technique, la plateforme repose sur un ensemble cohérent de composants : une API REST asynchrone FastAPI exposant plus d'une cinquantaine d'endpoints, un modèle de données SQLAlchemy structuré en dix-huit entités persistées sur PostgreSQL (User, Role, Project, GitHubRepository, ProjectMember, Scan, Vulnerability, VulnerabilityTask, VulnerabilityTaskComment, RiskScore, Recommendation, AuditLog, AuthToken, UserAPIKey, Notification, SecurityReport et les entités de configuration), un pipeline asynchrone Celery/Redis prenant en charge l'exécution non bloquante des scans SAST (Semgrep, Bandit) et DAST (OWASP ZAP), et une interface React temps réel connectée via Socket.IO. Les migrations de schéma sont versionnées avec Alembic, garantissant la reproductibilité entre environnements. L'ensemble est conteneurisé avec Docker Compose.

Différenciateur IA. L'intégration d'un assistant LLM local (Ollama, modèles Mistral/Qwen) constitue le principal élément différenciateur d'**InvisiThreat** par rapport aux solutions SaaS concurrentes. En inférant localement, la plateforme garantit la confidentialité totale du code source analysé, sans transmission vers des serveurs externes. Les recommandations de remédiation (explication contextuelle, exemple de correctif, référence CWE) sont persistées dans le modèle *Recommendation* et intégrées dans l'interface via l'action *Assist with AI*.

Gouvernance et sécurité. La plateforme implémente un socle de gouvernance robuste : RBAC à quatre rôles (admin, developer, security_manager, viewer) appliqué à chaque requête, authentification forte TOTP (pyotp, RFC 6238) compatible Google Authenticator, jetons JWT à durée limitée (access token 30 min, refresh token révocable), chiffrement des tokens GitHub avec Fernet (AES-128-CBC + HMAC-SHA256) et audit logs exhaustifs traçant chaque action sensible avec horodatage et identité de l'acteur.

Perspectives. Les travaux futurs envisagés en continuité de ce projet portent sur cinq axes principaux :

1. **Intégration CI/CD complète avec security gates** : bloquer automatiquement les merge requests contenant des vulnérabilités de sévérité critical ou high non résolues, renforçant le contrôle qualité dès la revue de code.

2. **Extension du périmètre d'analyse** : ajout du scanner **Trivy** pour l'analyse des images Docker et des manifestes Kubernetes, et de **Checkov** pour l'Infrastructure as Code (Terraform, Ansible), étendant la couverture DevSecOps à la couche infrastructure.
3. **Intégration ticketing (Jira, GitHub Issues)** : synchronisation bidirectionnelle du workflow de vulnérabilités avec les outils de gestion de projet existants, pour éviter la double saisie et améliorer l'adoption par les équipes de développement.
4. **Architecture multi-tenant** : isolation des données par organisation et facturation à l'usage, permettant le déploiement d'InvisiThreat en mode SaaS mutualisé pour des clients externes.
5. **Fine-tuning du modèle LLM** : entraînement du modèle sur un corpus de vulnérabilités spécifiques au domaine (CVE, CWE, OWASP), afin d'améliorer la pertinence des recommandations pour les langages et frameworks de l'écosystème Mobelite.

Ce projet a représenté une opportunité d'acquérir une maîtrise approfondie des pratiques DevSecOps modernes, de l'orchestration asynchrone et de l'intégration de l'intelligence artificielle dans un contexte de sécurité applicative réelle. InvisiThreat constitue désormais un outil opérationnel dont l'adoption au sein de Mobelite permettra de structurer, d'automatiser et de mesurer continuellement la posture de sécurité du portefeuille applicatif.

7 Bibliographie

Dos du rapport (à compléter si nécessaire).